Architectural Strategies for Personalization in Isomorphic Web Applications

Gustav Westin

2026

External Supervisor: Karim Mazzoni
Academic Supervisor: Keni Ren
Examiner: Thomas Mejtoft
External Partner: Nordea

Master thesis, 30 credits

Umeå University, Sweden

Department of Applied Physics and Electronics
Master of Science in Interaction Technology and Design, 300 credits

Abstract

Personalization has become a strategic priority in retail banking, but delivering personalized content within isomorphic web applications raises architectural questions that have not been systematically compared in the banking domain. This thesis investigates the design, implementation, and evaluation of two architectural strategies for personalization in an isomorphic Angular application at Nordea: an In-Application approach in which the personalization is resolved client-side after hydration, and a Backend for Frontend (BFF) approach in which a dedicated intermediary service resolves personalization server-side. The strategies are compared in terms of performance, maintainability, and security using a single embedded case study design that combines automated benchmark testing with qualitative expert interviews.

Performance was evaluated with Grafana k6 for protocol-level load testing and the k6 browser module for client-perceived rendering metrics. Maintainability, security, and an overall comparative assessment were evaluated through four semi-structured interviews with software architects and senior developers, analyzed thematically using Braun and Clarke's six-phase framework.

On performance, the results show a mixed picture at the protocol-level, where the In-Application architecture sustained marginally higher throughput through the content delivery network, while the BFF achieved lower Largest Contentful Paint, First Contentful Paint, and a substantially lower personalization time through the content delivery network at the browser-level (49.35 ms vs. 491.6 ms), with personalization time and Cumulative Layout Shift showing complete separation across runs ($\delta = 1.00, p < 0.01$). On maintainability and security, all four interview participants assessed the BFF as the preferable architecture, citing lower long-term change cost, centralized test effort, reduced exposure of the personalization mechanism at the network boundary, and better alignment with team structure. Regarding the conditions under which each strategy is preferable, all four participants recommended the BFF for production deployment, through three different rankings of the quality dimensions. The recommendation is bounded by four contingencies: the granularity of personalization, the controllability of the client, the sensitivity of the data, and the capability of the content management system. The thesis contributes an integrated architectural comparison of In-Application and BFF personalization in a banking context, and a methodological pattern in which expert judgment supports an architectural recommendation through priority-independent convergence rather than shared prioritization.

Keywords: Backend for Frontend, server-side rendering, hydration, retail banking, software architecture evaluation, mixed-method case study, web performance, personalization

Sammanfattning

Personalisering har blivit en strategisk prioritering inom bankverksamhet för privatkunder, men leverans av personaliserat innehåll i isomorfa webbapplikationer väcker arkitektoniska frågor som inte har jämförts systematiskt inom bankdomänen. Denna uppsats undersöker design, implementation och utvärdering av två arkitektoniska strategier för personalisering i en isomorf Angular-applikation hos Nordea: en In-Application-ansats där personaliseringen löses på klientsidan efter hydrering, och en Backend for Frontend-ansats (BFF) där en dedikerad mellanliggande tjänst löser personaliseringen på serversidan. Strategierna jämförs med avseende på prestanda, underhållbarhet och säkerhet med hjälp av en inbäddad fallstudiedesign som kombinerar automatiserad benchmarktestning med kvalitativa expertintervjuer.

Prestandan utvärderades med Grafana k6 för lasttestning på protokollnivå och k6:s browser-modul för klientupplevda renderingsmått. Underhållbarhet och säkerhet utvärderades genom fyra semistrukturerade intervjuer med mjukvaruarkitekter och seniora utvecklare, vilka analyserades tematiskt enligt Braun och Clarkes sexfasiga ramverk.

När det gäller prestanda visar resultaten en blandad bild på protokollnivå, där In-Application-arkitekturen upprätthöll marginellt högre genomströmning via content delivery network (CDN), medan BFF uppnådde lägre Largest Contentful Paint, First Contentful Paint, samt en väsentligt lägre personaliseringstid via CDN på webbläsarnivå (49,35 ms mot 491,6 ms). Personaliseringstid och Cumulative Layout Shift uppvisade fullständig separation mellan körningar ($\delta = 1,00, p < 0,01$). När det gäller underhållbarhet och säkerhet bedömde samtliga fyra intervjudeltagare BFF som den att föredra arkitekturen, med hänvisning till lägre långsiktig förändringskostnad, centraliserad testinsats, minskad exponering av personaliseringsmekanismen vid nätverksgränsen samt bättre överensstämmelse med teamstrukturen. Vad gäller villkoren under vilka respektive strategi är att föredra rekommenderade samtliga fyra deltagare BFF för produktionsdistribution, via tre olika rangordningar av kvalitetsdimensionerna. Rekommendationen avgränsas av fyra omständigheter: personaliseringens granularitet, klientens kontrollerbarhet, datans känslighet samt innehållshanteringssystemets kapacitet.

Uppsatsen bidrar med en integrerad arkitektonisk jämförelse mellan In-Application- och BFF-personalisering i en bankkontext, samt med ett metodologiskt mönster där expertbedömning stödjer en arkitektonisk rekommendation genom prioritetsoberoende konvergens snarare än delad prioritering.

Contents

1 Introduction **1**

2 Aim **3**

3 Background and theory **4**

3.1 Background 4
3.1.1 Partner organization 4
3.1.2 Existing architecture and personalization problem 4
3.1.3 Why the architectural comparison is relevant 5
3.2 Web application architecture and rendering 6
3.2.1 SSR, CSR, and isomorphic applications 6
3.2.2 Angular 7
3.2.3 Content management system 7
3.2.4 State management 8
3.2.5 CDN and caching 8
3.3 Personalization architectures and supporting technologies 9
3.3.1 Software architecture and personalization in web systems 9
3.3.2 In-Application personalization 10
3.3.3 Backend for frontend pattern 10
3.3.4 Edge computing 10
3.3.5 Supporting technologies 11
3.3.5.1 PostgreSQL 11
3.3.5.2 Redis 12
3.4 Security and privacy considerations 12
3.4.1 Cookies 13
3.4.2 Cookie security 13
3.4.3 Cross-site request forgery 14
3.4.4 Web cache poisoning 14
3.4.5 Principle of least privilege 14
3.4.6 GDPR and privacy considerations 15
3.5 Performance evaluation concepts and tools 15
3.5.1 Web performance metrics 15
3.5.2 Grafana k6 16
3.5.3 k6 browser module 17
3.6 Research methodology 17
3.6.1 Case study research 17
3.6.2 Thematic analysis 18
3.6.3 Statistical analysis 19

4 Method **20**
4.1 Research design . . . 20
4.1.1 Case study approach . . . 21
4.1.2 Embedded design . . . 21
4.1.3 Mixed-method design . . . 21
4.2 Architectural design . . . 22
4.2.1 In-Application design . . . 22
4.2.2 BFF design . . . 23
4.2.3 Shared design decisions . . . 23
4.3 Implementation . . . 24
4.3.1 In-Application architecture . . . 24
4.3.2 BFF architecture . . . 25
4.4 Evaluation framework . . . 27
4.5 Performance evaluation (quantitative) . . . 28
4.5.1 Test environment . . . 28
4.5.2 k6 load testing . . . 28
4.5.3 k6 browser testing . . . 29
4.5.4 Data analysis . . . 30
4.6 Qualitative evaluation: maintainability, security, and comparative assessment 30
4.6.1 Interview design . . . 31
4.6.2 Participant selection . . . 31
4.6.3 Interview procedure . . . 32
4.6.4 Data analysis . . . 32

5 Results **34**
5.1 Performance results . . . 34
5.1.1 Load testing . . . 34
5.1.2 Browser testing . . . 37
5.2 Qualitative interview findings . . . 40
5.2.1 Maintainability . . . 41
5.2.1.1 Theme M1: Logic locality and change cost grows towards the client . . . 41
5.2.1.2 Theme M2: Testing surface and centralization compresses test effort . . . 42
5.2.1.3 Theme M3: Team structure and ownership versus specialization . . . 42
5.2.1.4 Theme M4: Operational cost shifts in location, not magnitude . . . 43
5.2.2 Security . . . 44

5.2.2.1 Theme S1: Logic visibility and mechanisms versus result exposure . . . 44
5.2.2.2 Theme S2: Open data context attenuates security divergence 44
5.2.2.3 Theme S3: New endpoints, new hardening requirements . 45
5.2.3 Comparative assessment . . . 45
5.2.3.1 Theme C1: Convergent recommendation, divergent reasoning . . . 45
5.2.3.2 Theme C2: Quality dimensions priorities split by professional background . . . 46
5.2.3.3 Theme C3: Conditional recommendations and contextual contingencies . . . 46
5.3 Summary . . . 47

6 Discussion **49**
6.1 Performance does not settle the architectural choice . . . 49
6.2 Convergence without shared priorities . . . 51
6.3 The recommendation is contextual, not universal . . . 53
6.4 Limitations . . . 55
6.4.1 Single-machine benchmark environment. . . . 56
6.4.2 Limited statistical power. . . . 56
6.4.3 Small qualitative sample. . . . 56
6.4.4 Single-organization qualitative sample. . . . 56
6.4.5 Scope of the architectural comparison. . . . 57
6.4.6 Simulated CMS environment. . . . 57
6.5 Contribution and implications . . . 57
6.6 Future work . . . 59

7 Conclusion **61**

8 Acknowledgments **63**

A Interview guide **68**

B Informed consent form **78**

C Interview transcript – P1 **82**

D Interview transcript – P2 **97**

E Interview transcript – P3 **110**

F Interview transcript – P4 **134**

1 Introduction

Personalization in web applications refers to the practice of adapting content and functionality to meet the specific needs and preferences of individual users [50]. Empirical research has demonstrated that users respond positively to such adapted content, with personalization shown to improve engagement, attention, and decision outcomes [50]. Technology companies and consumer brands have demonstrated that data-driven personalization can produce interactions that feel relevant, timely, and valuable, raising the baseline for what users consider acceptable digital experience [12].

Organizations across industries have responded to these expectations by making personalization a strategic priority, recognizing that tailored experiences can improve engagement, reduce churn, and increase revenue [12]. In retail banking, the incentive is particularly strong. Boston Consulting Group estimates that for every $100 billion in assets a bank holds, personalization can yield up to $300 million in revenue growth, and that early adopters stand to gain significant competitive advantage [12]. At the same time, banking users interact with their institutions frequently, creating both an opportunity and an obligation to deliver experiences that feel individually relevant [12]. Banks face particularly high expectations for responsiveness and reliability, and any degradation in performance or availability carries immediate trust implications [31]. Achieving both personalized experiences and high system reliability therefore requires careful architectural design.

Delivering personalized content in modern web applications is closely tied to how the user interface is rendered and delivered to the client. From an architectural perspective, personalization therefore interacts directly with the rendering strategy of the application. Many contemporary web applications are implemented using client-side rendering (CSR), where the browser executes JavaScript to dynamically construct the user interface. While CSR enables rich interactivity, it can negatively impact initial load performance and search engine visibility since meaningful content is not present in the initially delivered HTML [35]. To address these limitations, server-side rendering (SSR) has been increasingly adopted. SSR generates fully rendered HTML on the server before delivery to the client, improving initial load performance and search engine visibility [4]. Applications that combine SSR with CSR are commonly referred to as isomorphic or universal JavaScript applications, and represent the current standard for performance-sensitive web platforms [26]. While this architecture improves performance and enables rich interactive experiences, introducing personalization within such environments presents additional challenges.

In an isomorphic application, code executes in both server and browser environments, which means personalization logic must account for platform-specific constraints. For example, cookies are accessed differently during SSR than in browser. During the SSR

phase, the application must access user-specific data, typically conveyed through cookies, headers, or session identifiers, and incorporate it into the rendering pipeline without exposing sensitive information to other users. The resulting personalized state must then be accurately transferred to the client so that when the application transitions to a fully client-side state, the content remains consistent. Any discrepancy can cause visual instability or redundant data fetching, a problem commonly referred to as hydration mismatch [51]. In a banking context, where user experience quality directly influences customer trust, such inconsistencies risk undermining the perceived reliability of the application [33]. Beyond these consistency concerns, personalization also introduces scalability concerns, specifically resolving and rendering user-specific content server-side increases the computational load per request, which under high concurrency can become a significant performance bottleneck if the architecture is not designed to handle it efficiently [24].

Despite the practical importance of these challenges, there is limited architectural guidance on how to best implement personalization in isomorphic web applications for the banking domain. Several distinct strategies are possible, including embedding personalization logic directly within the application, delegating it to a dedicated intermediary service, or moving it to the network edge using edge computing. Each approach involves different trade-offs in performance, operational complexity, security, and long-term maintainability. These trade-offs have not been systematically compared in the context of a real-world banking application.

This thesis addresses that gap by designing, implementing, and evaluating two of these architectural strategies, an In-Application approach and a Backend for Frontend (BFF) approach, for delivering personalized content in an isomorphic Angular application, conducted in the context of Nordea, one of the major banks in the Nordic region [49]. The strategies are compared across three quality dimensions performance, maintainability, and security. In addition, a comparative assessment was conducted in which expert participants stated an overall architectural recommendation and qualified it with contingencies. The data was collected using a combination of automated benchmark testing and qualitative expert assessment, with the goal of producing actionable architectural guidance for personalization in performance-sensitive web applications.

2 Aim

The aim of this thesis is to investigate the architectural trade-offs of personalization strategies in isomorphic web applications within a banking context. To support this investigation, two personalization architectures, an In-Application approach and a BFF approach, are designed and implemented in an isomorphic Angular application and evaluated with respect to performance, maintainability, and security. The study further examines under what contextual and architectural conditions each strategy is considered preferable for deployment. The investigation is guided by the following research questions:

RQ1: How do In-Application and BFF personalization architectures differ in terms of performance in an isomorphic Angular application?

RQ2: How do software architects and senior developers perceive the maintainability and security trade-offs between In-Application and BFF personalization?

RQ3: Under what contextual and architectural conditions is each personalization strategy considered preferable for deployment in a banking environment?

While RQ2 evaluates the perceived trade-offs of each architecture on the maintainability and security dimensions, RQ3 investigates how contextual and architectural factors influence the suitability of each strategy for deployment. The two questions are therefore complementary: RQ2 asks what each architecture is like, and RQ3 asks under what conditions each is preferable. The comparative assessment, in which participants stated an overall recommendation and ranked the quality dimensions, serves as the primary evidence base for RQ3.

3 Background and theory

This chapter establishes the background and theoretical foundation for the thesis. It begins with the organizational and technical context at Nordea, followed by the core concepts in web application architecture, personalization patterns, security, performance evaluation, and research methodology that inform the design and evaluation of the two architectural strategies under investigation.

3.1 Background

This section provides the organizational and technical context in which this thesis is situated. It describes the environment at Nordea that motivates the research, the existing application architecture, and the specific challenges that arise when introducing personalization into that environment. This contextual grounding is necessary for interpreting the architectural strategies evaluated in the thesis and for understanding the constraints and priorities that shape the comparison.

3.1.1 Partner organization

Nordea Bank Abp is the largest financial service group in the Nordic region and one of the ten largest financial institutions in Europe by market capitalization, operating across its four home markets of Sweden, Denmark, Finland, and Norway [34]. With more than 200 years of banking history and a customer base spanning millions of individuals and businesses across the region, Nordea operates at scale where digital service quality has direct implications for customer satisfaction, retention, and revenue [34].

Digital engagement has grown substantially in recent years. In 2024, customers logged into Nordea's digital services 1.5 billion times, an increase of eight percent compared to the previous year, and more than five million customers actively use Nordea's digital channels [34]. This volume of digital interaction underscores the importance of the web and mobile channel as the primary interface between the bank and its customers. Nordea is publicly recognized as a leader in digital banking, ranked among the top three banks for digitalization globally and the top performer in Europe [34].

3.1.2 Existing architecture and personalization problem

The application in question is built on an isomorphic Angular stack that combines SSR with client-side hydration to deliver fast initial page loads alongside rich interactive experiences, with content being managed and delivered through a content management system (CMS), which acts as the authoritative source for the pages and components displayed to users.

This architectural foundation is well suited to delivering static and semi-static content at scale. However, introducing personalization into such an environment raises questions that the existing architecture does not fully resolve. Specifically, it is not immediately clear at which layer of the stack personalization logic should be placed, how user identification should be handled across the server and client boundary, and how personalized responses should interact with the CDN caching layer without either sacrificing cache efficiency or serving incorrect content to users.

These questions motivate the architectural investigation in this thesis. Rather than prescribing a single solution, the study explores and evaluates two distinct strategies for integrating personalization into the existing application stack, with the goal of producing architectural guidance that is grounded in both empirical performance data and expert judgment from practitioners familiar with the environment.

3.1.3 Why the architectural comparison is relevant

The architectural decisions made when introducing personalization into an isomorphic web application have consequences that extend well beyond the initial implementation. Where personalization logic is placed in the system determines how it scales under load, how easily it can be modified and extended over time, how broadly sensitive data is exposed across components, and how the CDN caching layer interacts with user-specific content. These consequences compound over time as the application evolves, the user base grows, and personalization requirements become more sophisticated.

In a banking context these consequences carry additional weight. Users interact with their institutions frequently and hold high expectations for reliability and responsiveness, meaning that any degradation in performance or availability carries immediate trust implications [33, 31]. At the same time, the regulatory environment and the scale of customer dependency mean that architectural decisions have consequences that extend beyond typical content applications.

Despite this, there is limited guidance in the literature on how to make this architectural choice in practice. The relevant knowledge is fragmented across separate bodies of work addressing rendering strategies, backend service patterns, and personalization effectiveness independently, without comparative evaluation in the context of a real isomorphic banking application. Practitioners facing this decision therefore lack evidence-based guidance that accounts simultaneously for performance, maintainability, and security in a banking-grade deployment environment.

This thesis addresses that gap directly, by designing, implementing, and evaluating two architectural strategies for personalization in an isomorphic Angular application, and by combining quantitative measurements with qualitative expert assessment, the study produces guidance that is grounded in both empirical data and expert judgment. The comparison is not intended to produce a universal answer, but to surface the trade-offs,

identify the conditions under which each strategy is preferable, and provide a replicable methodological pattern for similar architectural decisions in other contexts.

3.2 Web application architecture and rendering

This section introduces the architectural and technical concepts that underpin the rendering and delivery of web applications, with particular focus on the patterns relevant to the application context of this thesis. Understanding how content is rendered and delivered to the client is essential for evaluating personalization architectures, since the placement of personalization logic is directly constrained by the rendering pipeline and the infrastructure through which responses are served.

3.2.1 SSR, CSR, and isomorphic applications

Rendering refers to the process of generating the code required to display a web page in a user's browser, including content preparation, logic processing, and integration of user interface components. In content-driven web applications, rendering strategies aim to optimize load speed, latency, and perceived responsiveness [20].

SSR generates fully rendered HTML on the server for each request and sends it to the client's browser. This approach enables fast initial page loads, improved search engine optimization, and reliable content delivery as users receive meaningful content immediately. However, SSR increases system complexity and server workload, requiring additional infrastructure and maintenance effort [20, 24].

Static site generation pre-renders HTML pages at build time rather than at request time. Content is processed by a static site generator to produce a complete set of static files that are deployed to a web server or CDN. Since the pages are static and cacheable, Static Site Generation provides excellent performance and scalability. However, it is best suited for content that changes infrequently and offers limited support for dynamic or personalized interactions [20].

CSR performs rendering directly in the user's browser using JavaScript. The server delivers a minimal HTML shell and application logic, after which the browser fetches data and dynamically constructs the user interface. CSR enables highly interactive experiences and fast client-side navigation but typically results in slower initial load times and reduced search engine optimization effectiveness[20, 24]. As applications grow, CSR architectures may also increase development and maintenance complexity [20, 35].

Each rendering strategy presents trade-offs between performance, scalability, interactivity, and implementation complexity. Selecting an appropriate rendering approach depends on application requirements such as content dynamics, personalization needs, and performance constraints.

Applications that combine SSR with CSR are commonly referred to as isomorphic

or universal web applications [26, 24]. In this architecture, the same application code executes both in the server and the browser environment. On the first request, the server renders a fully populated HTML response and sends it to the client, providing fast initial load performance. Once the HTML is received, the browser loads the JavaScript bundle and re-attaches event listeners and application state to the existing DOM in a process known as hydration [51], after which the application transitions to CSR for subsequent interactions. This hybrid approach combines the search engine optimization and initial load benefits of SSR with the interactivity of CSR, and represents the current standard for performance-sensitive web platforms [26, 24]. However, the dual execution environment introduces additional complexity, as code must account for platform-specific constraints that differ between the server and the browser.

3.2.2 Angular

Angular is an open source TypeScript-based platform and framework for building single-page-applications [5].

Its architecture is centered on components, which are fundamental building blocks of an application. Components define views that control what is displayed in the user interface and how it changes based on application data and logic. Components interact with services, which encapsulate background functionality such as fetching data. Services are provided to components via dependency injections promoting modularity, re-usability and efficiency [3].

Both components and services use decorators to supply metadata that instructs Angular how to instantiate and manage them. Component metadata links a class to an HTML template that combines standard markup with Angular directives and data binding to dynamically render content. Application views are organized hierarchically, and navigation between them is managed by Angular's Router, enabling structured client side navigation within the browser [3].

3.2.3 Content management system

Boukar [10] defines CMSs as software solutions that support creation, organization, and publication of digital information in enterprise environments by separating content from its format.

According to Teaganne Finn and Amanda Downie at IBM [19] a CMS is an application where users can create, manage, store, and modify digital content. The CMS is divided in two parts, the content management application, and the content delivery application. The content management application is the application where users write their data, it brings HTML, CSS, and JavaScript together to deliver the content in the correct format. The content delivery application is a back-end process that takes the content inputted in

the content management application and stores it, this process then makes it visible for web page visitors [19].

The CMS eliminates the process of needing to handle code and queries to a database manually [10]. In the context of this thesis, the CMS serves as the authoritative source for the content that is personalized and delivered to users, making it a central component in both architectural strategies under evaluation.

3.2.4 State management

State management refers to the process of tracking and controlling an application's condition by maintaining information about user data and system data over time [9]. The state of an application represents its current status based on stored data, which influences how the application behaves and processes tasks.

State management can be implemented either at the frontend, where the user interface or browser maintains the application's state, or at the backend, where servers or databases store and retrieve state information between processes. State is also categorized by scope: local state lives within a single component, inter-module state is shared between related components, and global state is shared across the entire application [32].

Effective state management provides several benefits, including improved application stability, better performance through data sharing and caching, easier debugging, and improved software design. In the context of personalization in isomorphic applications, state management is particularly relevant because the personalized state resolved on the server must be accurately transferred to the client during hydration to ensure content consistency and avoid redundant data fetching[51, 24].

3.2.5 CDN and caching

A CDN consists of an origin server, several edge servers that are physically distributed around the globe, and Domain Name System servers [1]. The edge servers are the most relevant part of the CDN for this thesis and they rely on caching to store data on each server after an initial request has been made so that subsequent requests can access the data directly from the edge servers and without needing to go all the way back to the origin server to retrieve the data [1].

Caching is the practice of temporarily storing frequently accessed data closer to the user so it can be retrieved more efficiently [18]. By reducing the number of round-trips to the origin server, caching improves both response times and system scalability under concurrent load. Caching can be applied at multiple layers of a system, including the browser, the CDN, the application layer, and the database.

However, caching introduces trade-offs that must be carefully managed. If cache keys do not accurately reflect all inputs that influence a response, cached content may be served

to the wrong user or become stale. In the context of personalization, this is particularly relevant since responses that vary by user segment must be cached in a way that prevents one segment's content from being served to another. These considerations directly affect the architectural decisions evaluated in this thesis.

3.3 Personalization architectures and supporting technologies

This section introduces the architectural patterns and concepts that are directly relevant to the design and evaluation of personalization strategies in this thesis. It begins with the role of software architecture in personalized web systems, establishing why personalization must be treated as an architectural concern rather than a feature-level one. It then describes three strategies through which personalization logic can be placed in a web system: directly within the application, in a dedicated BFF service, or at the network edge using edge computing. The two former strategies form the basis of the architectural comparison in this thesis, while edge computing is included as relevant context. Understanding these patterns provides the conceptual foundation for the architectural designs presented in the Method chapter.

3.3.1 Software architecture and personalization in web systems

Software architecture provides the structural foundation that determines how a system satisfies functional and quality requirements. ISO/IEC/IEEE 42020:2019 [25] defines architecture practices as consisting of structured processes for governance, management, conceptualization, evaluation, and elaboration . These processes ensure that architectural decisions contribute to business and system success throughout the system life cycle. The standard [25] further defines architectural patterns, tactics, and heuristics as mechanisms for achieving desired system qualities such as performance, modifiability, and reliability. Architectural tactics are applied to influence system characteristics, while patterns guide the organization and interaction of components.

Architecture must also be continuously evaluated and adapted as systems evolve, since constraints and performance impacts often emerge during implementation and operation phases [25].

In the context of personalized web applications, architectural decisions directly affect performance, scalability, security, and consistency of user experiences. As ISO/IEC/IEEE 42020:2019 [25] establishes, architectural tactics and patterns are the primary mechanisms through which desired system qualities are achieved, and personalization therefore cannot be treated solely as a feature-level concern but must be addressed at the architectural level.

3.3.2 In-Application personalization

The In-Application approach places personalization logic directly within the web application rather than delegating it to a dedicated intermediary service. In this pattern, a backend matching service handles the decision of which content to serve based on the user's segment, while the application itself is responsible for orchestrating the personalization flow. This implies querying the matching service, fetching content from the CMS, and assembling the personalized response.

The main advantage of this approach is its operational simplicity. There is no dedicated intermediary service between the application and the CMS, which reduces infrastructure overhead and keeps the system boundary contained within a manageable scope.

A few things to keep in mind when implementing this pattern are that personalization logic executed on the client side after hydration can cause visual instability, as users may briefly see default content before personalized content is displayed. Additionally, the personalization mechanism is visible at the network boundary, and changes to the orchestration logic require redeployment of the full application [24].

3.3.3 Backend for frontend pattern

The BFF pattern introduces a dedicated intermediary service between the frontend application and backend services such as a CMS or data store [40]. Rather than having the application communicate directly with these services, all requests are routed through the BFF, which handles data aggregation, caching, and response assembly before returning a compiled response to the application [40].

In the context of personalization, the BFF takes on responsibility for both resolving the user segment and orchestrating content retrieval, keeping the application itself unaware of how personalization is implemented. This separation of concerns improves maintainability and reduces the exposure of backend services and personalization logic at the application boundary[40, 54].

However, introducing a BFF adds operational overhead, as an additional service must be deployed, scaled, and maintained independently [54]. The extra network hop between the application and the BFF can also introduce latency [40], and the intermediary service must be hardened against attacks such as Cross-Site Request Forgery (CSRF) [54].

3.3.4 Edge computing

Another option for implementing personalization in the system is with the use of Edge computing.

The general architecture for edge computing can be divided into three layers [48]:

- **Terminal layer** is all of the systems connected to the edge network, they can potentially both provide and consume data.

- **Edge layer** is all the edge nodes on the edge of the internet.
- **Computing layer** is where all the storage and computing is taking place.

This architecture has the advantage of conducting fast processing and analysis of data by having physically placed servers closer to the user (edge of network)[48].

As discussed by Caprolu et al. [13], edge computing introduces several security and operational risks that are relevant when deploying computation closer to end users. The decentralization of computation increases the system attack surface, as a larger number of geographically distributed edge nodes must be secured and maintained compared to a centralized cloud environment. Virtualization technologies commonly used at the edge, such as containers and lightweight virtual machines, may introduce vulnerabilities related to misconfiguration, privilege escalation, and denial of service attacks. Furthermore, edge nodes often operate under constrained resources and may be difficult to update and patch consistently, which increases long-term maintenance complexity and potential exposure to security flaws.

In the context of this thesis, edge computing represents a third architectural option for personalization that was considered in the original research design but is not evaluated in the comparison. It is included here as relevant context, and its trade-offs relative to the two strategies under evaluation are identified as a direction for future work.

3.3.5 Supporting technologies

The following technologies were used in the implementation of the architectural strategies evaluated in this thesis. They are introduced here to provide the theoretical foundation necessary for understanding the implementation decisions described in the Method chapter.

3.3.5.1 PostgreSQL PostgreSQL is a free and open-source object-relational database management system that originated from the POSTGRES project at the University of California, Berkeley in 1986 and has been under continuous active development for nearly four decades [21]. It uses and extends the SQL language, and supports advanced features such as foreign keys, triggers, stored procedures, materialized views, and user-defined data types [21].

A defining characteristic of PostgreSQL is its full compliance with ACID properties: Atomicity, Consistency, Isolation, and Durability. Together, these guarantee that database transactions are processed reliably even in the presence of concurrent access, system failures, or power outages [47]. PostgreSQL achieves this through mechanisms such as write-ahead logging, which ensures that all changes are recorded to a log before being applied to the data files, enabling crash recovery, and MultiVersion Concurrency Control,

which allows concurrent transactions to operate on consistent snapshots of the data without blocking each other [47]. These properties make PostgreSQL particularly well-suited for domains where data integrity is critical, such as financial services, healthcare, and enterprise applications [21].

In the context of this thesis, PostgreSQL is used as the persistent data store for user segment mapping in the BFF service. Its ACID guarantees that segment resolution queries return consistent and reliable results which is essential for correct personalization behavior.

3.3.5.2 Redis Redis (Remote Dictionary Server) is an open-source, in-memory key-value data store commonly used as a database, cache, and message broker [23]. Unlike traditional disk-based databases, Redis stores all data in the server's main memory (RAM), which enables sub-millisecond response times for both read and write operations [23]. Redis supports a range of data structures beyond simple key-value pairs, including strings, hashes, lists, sets, and sorted sets, each with specialized atomic commands that reduce the number of round-trips between client and server [42].

While Redis is primarily an in-memory system, it provides optional persistence mechanisms to prevent data loss. These include Redis Database Backup snapshots, which periodically write the full dataset to disk as a binary dump, and append-only file logging, which records every write operation to enable point-in-time recovery [43]. Redis can also be configured for replication and clustering to support high availability and horizontal scalability [23].

Common use cases for Redis include application-level caching, session management, real-time analytics, rate limiting, and message queuing [23]. Its speed and support for expiration-based key eviction make it particularly effective as a caching layer in front of slower persistent databases.

In the context of this thesis, Redis is used for session management and as a caching layer within both architectural implementations. By caching segment resolutions and personalized content responses, Redis reduces the number of repeated calls to the CMS and the matching service, improving response times under concurrent load.

3.4 Security and privacy considerations

Web applications that deliver personalized content must address several security concerns that arise from the interaction between user identification, caching, and distributed service architectures. This section discusses the most relevant threats and protective mechanisms in the context of the architectures in this thesis.

3.4.1 Cookies

Cookies are small text-based data objects stored in a user's browser and exchanged with a web server to enable stateful behavior in an otherwise stateless web protocol. A cookie typically contains a unique identifier and associated metadata that allows the server to recognize returning users, maintain sessions, and personalize content across visits. Cookies usually include attributes such as name, value, domain, path, expiration time, security flags, lifetime, and transmission behavior [39].

Cookies can be classified according to origin, duration, and purpose. First-party cookies are created by the visited domain and are commonly used for core functionality such as authentication and session management, and third-party cookies are set by external domains and are frequently associated with tracking and advertising. Based on duration, cookies may be session-based, existing only during an active browser session, or persistent, remaining stored on the client for a defined period. From a functional perspective, different cookies serve different purposes, some are strictly necessary for basic site operation, while others might support analytics, or enable personalization and user preferences [39].

From a technical perspective, cookies enable web applications to store lightweight client-side state and reduce repeated server-side computation or data fetching [39]. However, improper cookie handling may introduce security and privacy risks (see Section 3.4.2).

Regulatory frameworks such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act impose requirements on transparency, consent, and data protection when cookies are used to process personal data. Websites are required to inform users about cookie usage, provide consent mechanisms for non-essential cookies, and ensure appropriate handling of collected data [39].

Overall, cookies remain a fundamental mechanism for enabling personalization, session continuity, and usability in modern web applications, while simultaneously requiring careful architectural, security, and regulatory considerations.

3.4.2 Cookie security

Because the personalization flow in this thesis relies on cookies to identify user segments, the secure handling of cookies is an important aspect. The Open Web Application Security Project (OWASP) identifies several cookie attributes that mitigate common attack vectors [38]. The `Secure` attribute ensures that cookies are only transmitted over encrypted HTTPS connections, preventing interception through man-in-the-middle attacks. The `HttpOnly` attribute prevents client-side scripts from accessing the cookie value, which limits the impact of cross-site scripting (XSS) attacks by protecting cookie confidentiality. The `SameSite` attribute controls whether the browser includes the cookie in cross-site requests, providing a defense-in-depth mechanism against CSRF [37]. OWASP recom-

mends combining all three attributes for session-sensitive cookies, with the most secure configuration being `Secure; HttpOnly; SameSite=Strict` [38]. Where and how cookies are read in the architecture determines which components must enforce these protections and how broadly the cookie value is exposed within the system.

3.4.3 Cross-site request forgery

CSRF is a class of attack in which a malicious website causes an authenticated user's browser to submit an unwanted request to a trusted site, exploiting the fact that browsers automatically include cookies with requests to the cookie's origin domain [8]. CSRF is particularly relevant in architectures where cookies carry session or personalization identifiers, as the browser will attach these cookies to any matching request regardless of the request's origin. OWASP recommends several mitigation strategies, including the use of anti-CSRF tokens, custom request headers for AJAX calls, and the `SameSite` cookie attribute as a defense-in-depth measure [36]. In architectures that introduce a dedicated backend service (such as the BFF pattern), CSRF protection becomes especially important, since the intermediary service acts as a session-aware proxy and must validate that incoming requests originate from the legitimate application frontend [54].

3.4.4 Web cache poisoning

In systems that combine caching with personalized content, web cache poisoning represents a significant risk. This attack occurs when an attacker manipulates an HTTP request so that a malicious or incorrect response is stored in the cache and subsequently served to other users [41]. In a personalization context, a related concern is content leakage, where a response intended for one user segment is cached and served to users belonging to a different segment. This can occur if cache keys do not properly account for all inputs that influence the response, such as personalization cookies or segment identifiers. Mitigation include ensuring that all user-specific inputs that affect the response are included in the cache key, using the `Vary` header to signal response variability, restricting caching to static resources where possible, and validating that no un-keyed inputs can influence response content [41].

3.4.5 Principle of least privilege

The principle of least privilege (PoLP) states that every component in a system should be granted only the minimum permission necessary to perform its intended function [46]. In the context of microservices and distributed web architectures, this principle has implications for how personalization logic is distributed across components. An architecture in which the application server directly accesses user identifiers, segment resolution logic, and the CMS concentrates responsibility and data access in a single component. By

contrast, an architecture that delegates segment resolution to a separate service allows the application to receive only the pre-assembled personalized content without requiring access to the user-identifying data or backend databases. This separation reduces the attack surface of the frontend application and limits the potential damage if any single component is compromised [16]. PoLP is widely recognized as a foundational principle in security frameworks such as Zero Trust [44].

3.4.6 GDPR and privacy considerations

Personalization inherently involves processing user-related data, which brings regulatory obligations under frameworks such as the GDPR [17]. Even when personalization relies on segment-level identifiers rather than directly personal data, the handling, storage, and transmission of cookies and associated metadata must comply with transparency and consent requirements [39]. Architectural decisions affect the privacy posture of the system. For instance, minimizing the number of components that handle user identifiers supports the GDPR principle of data minimization [17]. While full regulatory compliance analysis is outside the scope of this thesis, security and privacy considerations are treated as interrelated dimensions when evaluating the architectural approaches.

3.5 Performance evaluation concepts and tools

Evaluating the performance of web applications requires tooling that captures both user-perceived rendering behavior and server-side scalability under load. These two aspects represent fundamentally different dimensions of performance and necessitate complementary measurement approaches. This thesis employs Grafana k6 for both purposes, utilizing its protocol-level load testing capabilities alongside its browser module for client-perceived frontend metrics.

3.5.1 Web performance metrics

Web performance metrics are standardized measurements used to quantify how quickly and reliably a web page loads and becomes usable from a user's perspective. These metrics can be broadly divided into two categories: protocol-level metrics, which capture server-side behavior such as response times and throughput under concurrent load, and browser-level metrics, which capture the user-perceived rendering experience from within a real browser environment.

Protocol-level metrics are typically collected through load testing tools that simulate concurrent users making HTTP requests to a server. Key metrics in this category include response time, which measures the time elapsed between sending a request and receiving a complete response, throughput, which measures the number of requests a system can handle per unit of time, and error rate, which captures the proportion of requests that

fail under load [29]. Percentile-based response times, such as the 95th percentile, are particularly informative because they reflect worst-case behavior for the majority of users rather than average behavior, which can mask outliers [29].

Browser-level metrics capture performance as experienced by a real user loading a page in a browser. Google introduced the Core Web Vitals framework as a standardized set of user-centric metrics for measuring web experience [28]. The metrics most relevant to this thesis are Largest Contentful Paint (LCP), which measures the time until the largest visible content element is rendered and serves as a proxy for perceived load speed; First Contentful Paint (FCP), which measures the time until the first content is painted to the screen; Time to First Byte (TTFB), which measures the latency between a request being sent and the first byte of the response being received; and Cumulative Layout Shift (CLS), which measures visual stability by quantifying unexpected layout shifts during page load [28].

Google publishes scoring thresholds for these metrics to help practitioners interpret whether a measurement reflects a good, needs-improvement, or poor user experience [53]. For LCP, a value at or under 2.5 seconds is considered good and a value above 4.0 seconds is considered poor. For FCP, a value at or under 1.8 seconds is considered good and a value above 3.0 seconds is considered poor [52]. For CLS, a value at or under 0.1 is considered good and a value above 0.25 is considered poor [53]. These thresholds provide a frame of reference for evaluating whether measured differences between architectures cross the boundary into perceptibly degraded user experience, or remain within the acceptable range even when relative differences appear large in percentage terms.

In the context of personalization, CLS is particularly relevant because client-side content swaps after hydration can introduce layout shifts that degrade the perceived quality of the experience.

3.5.2 Grafana k6

Grafana k6 is an open-source load and performance testing tool designed to help developers, site reliability engineers, and quality assurance teams assess system performance and identify issues before they reach production. Its engine is written in Go, and tests are authored in JavaScript or TypeScript, offering a developer-friendly approach to creating, running, and automating performance tests across both local and cloud environments [30].

The tool supports a wide range of test types including: spike, stress, and soak tests, and is optimized for minimal resource consumption, making it suitable for high-load scenarios. Through the k6 browser API, browser-based performance tests can also be run, allowing for the collection of frontend metrics alongside server-side measurements [29].

k6 operates by executing scripts that define testing scenarios in JavaScript, specifying the number of virtual users, the HTTP requests they should perform, and any additional logic needed to simulate realistic user behavior. The tool collects performance metrics

from these executions and produces detailed result reports [29]. Key metrics available include response time percentiles, throughput, and error rate, which together provide a comprehensive picture of system behavior under concurrent load.

3.5.3 k6 browser module

The k6 browser module extends k6 with browser-level APIs that enable frontend performance testing alongside existing protocol-level tests. The module drives a real headless Chromium browser using the Chrome DevTools Protocol, allowing tests to simulate genuine user interactions and collect user-perceived performance metrics [29].

The browser module collects the Core Web Vitals metrics described in Section 3.5.1, including LCP, FCP, TTFB, and CLS, alongside Interaction to Next Paint, which measures the responsiveness to user interactions [28].

A key advantage of the k6 browser is that it has support for hybrid performance testing, which combines protocol-level tests with browser-level frontend metrics in a single test script [29]. This allows for a more complete picture of the systems behavior, measuring both how the server handles concurrent load and how the frontend performs from a real user's perspective simultaneously. Variability in browser-based measurements is a recognized limitation, as results can differ across runs due to fluctuations in rendering behavior and system load. To reduce this variability and produce more representative results each test scenario is executed multiple times and the median value is used for comparison [22].

In the context of this thesis, the k6 browser module is used to measure client-perceived rendering performance for a single user session capturing the time for a personalized page to appear to the user. The protocol-level k6 tests are used to evaluate server-side scalability and behavior under concurrent load across both architectural implementations.

3.6 Research methodology

This section introduces the research methodology concepts that underpin the study design and analysis. It covers case study research as the overarching empirical strategy, thematic analysis as the method for analyzing qualitative interview data, and the statistical tests used to evaluate quantitative performance results.

3.6.1 Case study research

Empirical research in software engineering can take several forms, including controlled experiments, surveys, action research, and case studies [45]. Case study research is particularly suited to investigating phenomena where the boundary between the phenomenon and its context is not clearly defined [55, ch. 1]. This sets it apart from controlled experiments, which isolate variables in artificial settings, and from surveys, which favor breadth

over contextual depth. Runeson and Höst [45] note that case study research is particularly well suited to software engineering research, where the phenomena under study are often embedded in complex organizational, technical, and operational contexts that cannot be meaningfully reproduced in a laboratory setting.

Yin [55, ch. 1] provides a foundational framework for designing and conducting case study research. The framework defines a case study as an empirical inquiry that investigates a contemporary phenomenon in depth and within its real-world context, and it emphasizes the importance of rigorous design to compensate for the lack of experimental control inherent in real-world investigations. Runeson and Höst [45] adapt Yin's framework specifically for software engineering, providing guidelines for planning, data collection, analysis, and reporting that reflect the particular characteristics of software research.

Yin [55, ch. 2] distinguishes between four basic case study designs, defined by two dimensions. Whether the study examines a single case or multiple cases, and whether it treats the case as a whole (holistic) or analyzes distinct sub-units within it (embedded). Embedded case designs are appropriate when the research question requires analysis at multiple sub-unit levels within a single organizational setting, allowing for structured comparisons between the sub-units while preserving the shared contextual factors that influence the overall case.

A central methodological principle in both Yin's and Runeson and Höst's frameworks is the use of multiple sources of evidence. Case studies are strengthened when findings converge from independent data sources, a principle known as triangulation [55, ch. 4]. Triangulation improves the construct validity of the study by reducing the risk that conclusions rest on a single, potentially biased, source of information. Runeson and Höst [45] identify several forms of triangulation relevant to software engineering research, including data triangulation (using multiple data sources), methodological triangulation (using multiple data collection methods), and observer triangulation (using multiple researchers). In practice, these principles often lead to mixed-method designs that combine quantitative measurements with qualitative insights.

3.6.2 Thematic analysis

Thematic analysis is a qualitative research method for systematically identifying, analyzing, and reporting patterns of meaning within a dataset [11]. Braun and Clarke [11] describe thematic analysis as a six-phase process: familiarization with the data, generating initial codes, searching for themes, reviewing themes, defining and naming themes, and producing the final report. The process is iterative rather than strictly linear, researchers move between phases as themes are refined and the relationship between codes and the broader dataset becomes clearer. A theme captures something meaningful about the data in relation to the research question and represents a patterned response or meaning across

the dataset, rather than simply being a topic that appears frequently.

A key distinction of thematic analysis is between inductive and deductive approaches [11]. In an inductive approach, themes are derived directly from the data without being constrained by preexisting theoretical categories. In a deductive approach, coding and theme development are guided by prior theoretical or conceptual frameworks. In practice, applied research often uses a hybrid approach, where themes emerge from the data but are interpreted within a broader conceptual context established by the research design.

3.6.3 Statistical analysis

The Mann–Whitney U test is a non-parametric test used to assess whether two independent samples tend to differ systematically [7]. Unlike the t-test, it does not assume normally distributed data, making it appropriate for small samples where that assumption cannot be verified. It works by ranking all observations from both groups jointly and evaluating whether one group consistently occupies higher ranks than the other.

Effect sizes are reported using Cliff's δ [15], a non-parametric measure that expresses the probability that a randomly selected value from one group exceeds a randomly selected value from the other. It ranges from -1 to $+1$, where 0 indicates no difference. Thresholds for interpretation are $|\delta| < 0.147$ negligible, < 0.33 small, < 0.474 medium, and ≥ 0.474 large [7].

4 Method

This chapter describes the methodological approach used to investigate personalization architectures in an isomorphic Angular application. It presents the overall research design, the architectural implementations evaluated in the study, the evaluation framework, and the procedures used for data collection and analysis. The study adopts a mixed-method embedded case study design, combining quantitative performance testing with qualitative expert interviews to evaluate the architectures from the perspectives of performance, maintainability, and security. To support controlled and reproducible evaluation, the implementations were tested within a simulated environment designed to reflect the relevant characteristics of a real-world banking application context.

4.1 Research design

This study is designed as a single embedded case study of personalization in an isomorphic Angular application. The case is defined as the delivery of personalized content within an isomorphic Angular web application, and the embedded units of analysis are the two architectural strategies evaluated: an In-Application approach and a BFF approach. These units are compared using a mixed-method approach that combines quantitative performance testing with qualitative expert interviews. Figure 1 provides an overview of the research design, which is described in detail in the following subsections.

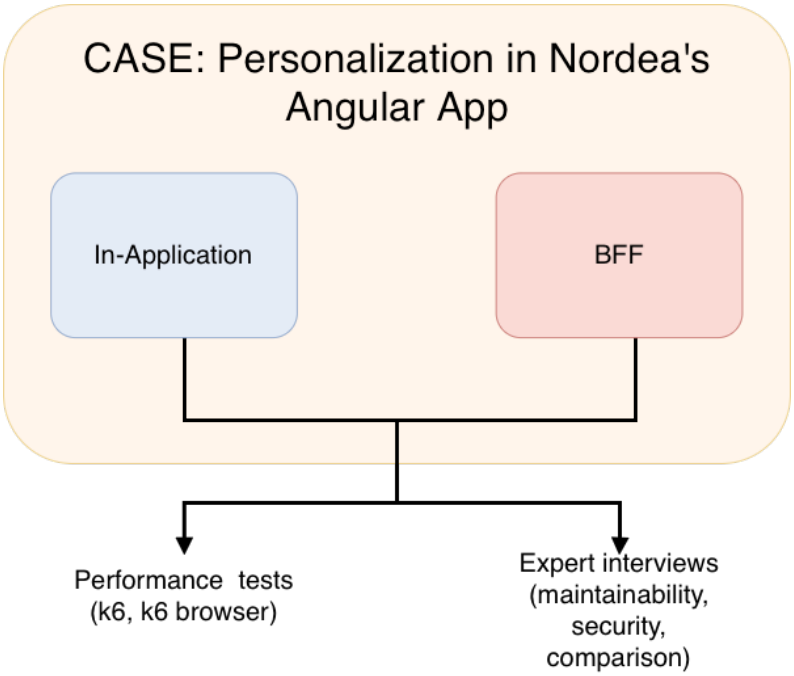


Figure 1: Overview of the research design, showing the case, the two architectural strategies under evaluation, and the two evaluation methods applied to each.

The following subsections justify each of these design decisions with reference to the

case study framework introduced in Section 3.6.1.

4.1.1 Case study approach

A case study approach was chosen because the phenomenon under investigation, architectural strategies for personalization in isomorphic web applications, cannot be meaningfully separated from the organizational, technical, and operational context in which it is deployed. The choice between different personalization architectures depends on factors such as existing team structures, deployment strategies, security requirements, and performance expectations, all of which are specific to the environment under study. This matches Yin's characterization of case study research as an empirical inquiry into a contemporary phenomenon within its real-world context [55, ch. 1], and it aligns with Runeson and Host's observation that case study research is particularly well suited for software engineering investigations where the phenomena under study are embedded in complex organizational and technical contexts [45]. A controlled experiment would not be able to capture these contextual dimensions, and a survey would lack the depth required to evaluate architectural trade-offs in any single deployment environment.

4.1.2 Embedded design

The study used a single embedded case design, following Yin's typology [55, ch. 2]. A single case is appropriate because the research question concerns personalization in one specific environment (the application stack at Nordea), and the value of the study lies in the depth of analysis within that environment rather than in generalizing across organizations.

The design is embedded because two distinct sub-units, the In-Application architecture and the BFF architecture, are evaluated independently within the overarching case. Each architectural implementation constitutes a sub-unit of analysis, while the shared application shell, simulated CMS, and deployment environment form the common context that enables meaningful comparison between them. This allows the study to isolate the effect of architectural choice while holding all other variables constant.

4.1.3 Mixed-method design

The study combines quantitative and qualitative methods because no single method can adequately assess all three quality dimensions under evaluation (performance, maintainability, and security) or capture the overall architectural preference that emerges from weighing them together.

Quantitative data was collected through automated performance tests, measuring load behavior and client-perceived rendering metrics across both architectural implementations. Performance testing provides reproducible, objective evidence about efficiency and

scalability that would be difficult to obtain through expert judgment alone.

Qualitative data was collected through semi-structured interviews with software architects and senior developers at Nordea. Maintainability and security are difficult to measure directly, as they depend on factors such as operational complexity, long-term evolvability, attack surface, and organizational fit. These are best assessed by professionals with hands-on experience in the relevant technology stack and deployment environment, whose judgment draws on tacit knowledge that automated tools cannot capture.

The two methods are complementary rather than redundant. Performance tests answer how each architecture behaves under load, while interviews surface the architectural knowledge, organizational constraints, security considerations, and the ability to put the two architectures against each other. Combining them supports methodological triangulation, one of the forms of triangulation identified by Runeson and Höst [45] as strengthening case study validity in software engineering research, and aligns with Yin's recommendation that case studies benefit from multiple sources of evidence [55, ch. 4].

4.2 Architectural design

Before implementation, each architectural strategy was designed with the goal of delivering personalized content in an isomorphic Angular application while making different trade-offs in where personalization logic is resolved. To ensure that performance and behavior differences could be attributed to architectural choices rather than unrelated factors, both designs shared the same overall system boundary: an Angular application, a CDN, and a CMS, but differed in how user segmentation and content assembly were distributed across components.

4.2.1 In-Application design

The In-Application design placed personalization logic directly within the Angular application. The default content was rendered server-side so that the browser could receive fully rendered HTML immediately and begin displaying content without waiting for personalization to be resolved. Personalization was intentionally deferred to the client side, where user identification, segment resolution, and CMS composition were performed after hydration. This avoided the latency of resolving the user segment, fetching personalized content, and assembling it server-side before any response could be sent.

To support this, a matching database was included in the design. The matching database held the mapping between user segments and the corresponding CMS paths and content variants and was queried by the client to determine which calls to make to the CMS. The design consisted of two logical components, namely the matching database acting as a decision layer, responsible for holding the mapping between user segments and corresponding CMS paths, and the Angular client acting as the orchestration layer,

responsible for querying the decision layer, executing CMS calls, and assembling the personalized response after hydration.

This approach avoided the need for a dedicated intermediary service, reducing infrastructure overhead at the cost of distributing the personalization responsibility across the matching database and the client. Because personalization occurs client-side after hydration, users may briefly see content flickering as personalized content replaces the default server-rendered content, which can affect perceived performance and visual stability.

4.2.2 BFF design

The BFF design introduced a dedicated intermediary service between the Angular application server and the CMS. Unlike the In-Application design where decision and orchestration responsibilities were distributed across separate components, the BFF consolidated both into a single service, consisting of a decision layer responsible for reading cookies and session identifiers to resolve user segments and determine which CMS paths and content variants to request, and an orchestration layer responsible for executing CMS calls, handling caching, and assembling the compiled JSON response. The Angular application required minimal modification, as personalization logic was fully encapsulated within the BFF and the application itself remained unaware of how content was assembled.

This design improved separation of concerns and allowed the personalization logic to evolve independently from the application layer, at the cost of additional infrastructure, operational overhead, and potential latency from the extra network hop.

4.2.3 Shared design decisions

The fundamental architectural difference between the two designs is where the personalization logic resides and how visible it is to the Angular application. In the In-Application design, the personalization logic lives within the Angular application itself, which directly queries the matching database and orchestrates CMS calls. In the BFF design, the personalization logic is fully encapsulated within the BFF service, and the Angular application is unaware of how segments are resolved or how content is assembled, it only consumes the pre-compiled response.

Several design elements were kept constant to ensure that observed differences could be attributed to the architectural choice rather than unrelated factors. The Angular application shell, the simulated CMS, and the mock user segmentation data were identical across both designs. PostgreSQL was used as the persistent store, holding the mapping between user IDs and their corresponding customer types. Redis was used as a session store, holding the mapping between session IDs and user IDs, enabling user identification on subsequent requests.

4.3 Implementation

Both architectural strategies described in the previous section were implemented as working systems and evaluated under identical conditions. To ensure a fair comparison and to isolate the effect of architectural choice from variability in real-world data, both implementations operated under the same set of mock data. This included a mock user segmentation dataset stored in PostgreSQL and a simulated CMS serving a fixed set of content variants, one per segment, across both architectures. Using identical mock data ensured that any observed differences in performance or behavior could be attributed to the architectural choice rather than to differences in data volume or content complexity.

4.3.1 In-Application architecture

In the In-Application approach, the orchestration part of the personalization was implemented directly within the Angular application. During SSR, the server rendered the default content and transferred it to the client. Once the application had hydrated, user identifiers were read from cookies, the appropriate content segment was resolved by querying a matching database, and multiple targeted calls were made to the CMS to compile the personalized JSON response (Figure 2). The personalized content then replaced the default content in the browser.

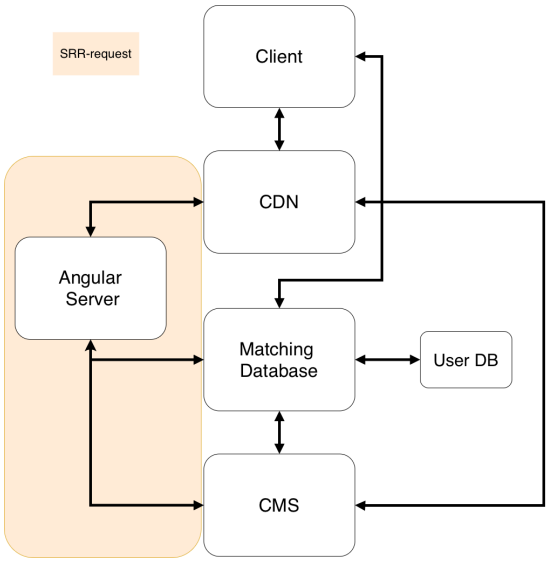


Figure 2: In-Application architecture

Figure 3 illustrates the runtime request flow for this architecture. During the SSR phase, the Angular server fetches default content from the CMS and returns a fully rendered HTML response, which is cached by the CDN for subsequent requests. Once the application has hydrated in the browser, the personalization logic executes client-side. The session cookie is read, the user segment is resolved by querying the matching database, and the targeted content is fetched from the CMS. The personalized content then replaces

the default content in the DOM. Because this content swap occurs after hydration, users may briefly see the default content before personalization resolves, which can contribute to layout shift.

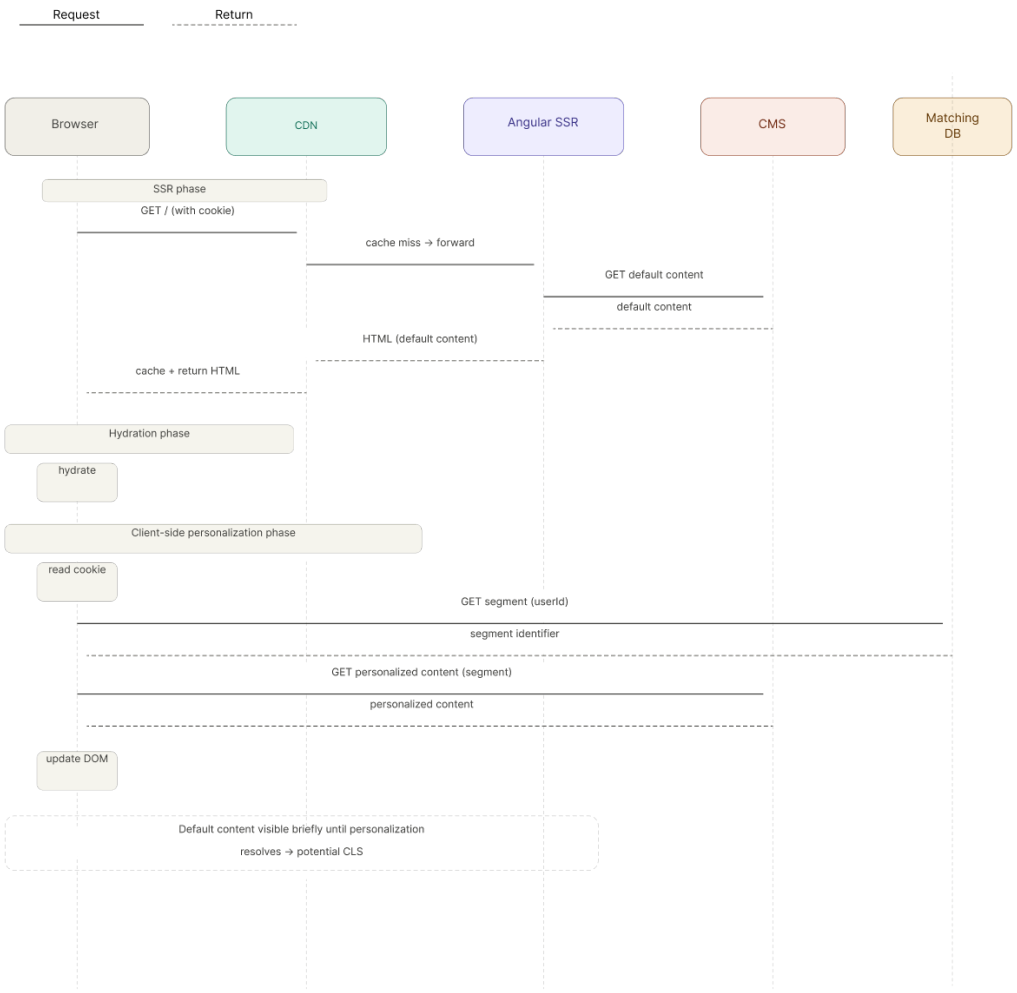


Figure 3: In-Application sequence diagram

4.3.2 BFF architecture

In the BFF approach, a dedicated intermediary service was introduced between the Angular application server and the CMS (Figure 4). The BFF consisted of a decision layer, which read cookies and sessions identifiers to resolve user segments and determine which CMS paths and content variants to request, and an orchestration layer, which executed the CMS calls, handled caching, and assembled the compiled JSON response before returning it to the application. The application then rendered the pre-assembled response.

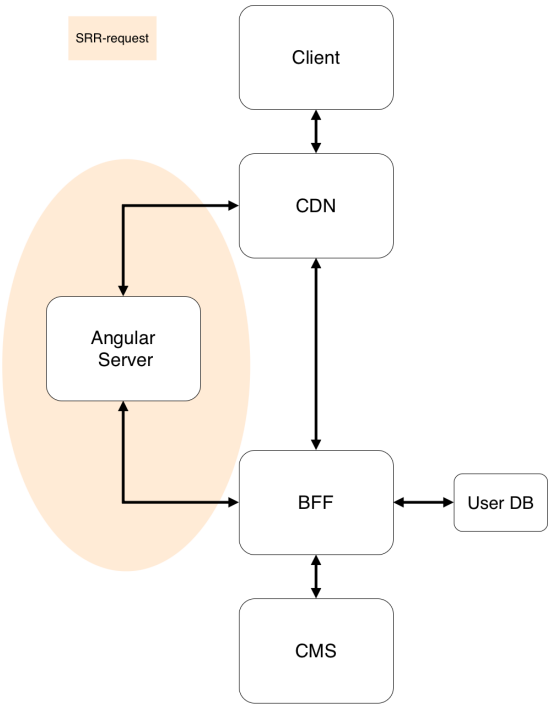


Figure 4: BFF architecture

Figure 5 illustrates the runtime request flow for this architecture. During the SSR phase, the Angular server calls the BFF service, passing along the session cookie. The BFF resolves the user segment and fetches the corresponding content from the CMS if not already cached. The BFF returns the compiled personalized content to Angular, which renders it server-side and returns a fully rendered HTML response. Because personalization is resolved on the initial request, the client receives the personalized page without the need of making further requests.

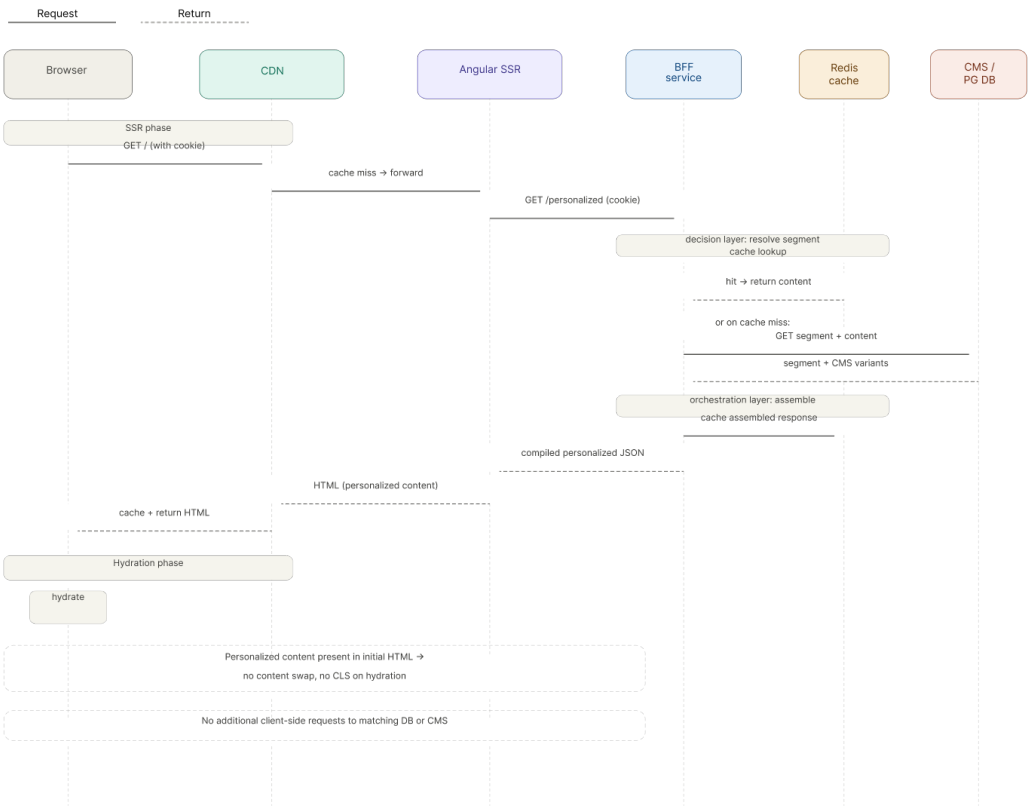


Figure 5: BFF sequence diagram

4.4 Evaluation framework

Each implementation was evaluated against the three quality dimensions performance, maintainability and security. This multi-dimensional framework was motivated by the architectural nature of the research question. Selecting a personalization strategy in a banking application involves trade-offs across all three dimensions, and optimizing for performance alone would yield an incomplete and potentially misleading comparison. In addition to evaluating each dimension independently, participants were asked to state an overall recommendation for production deployment. Table 1 summarizes the evaluation methods and tools used for each dimension.

Table 1: Evaluation framework

Dimension	**Evaluation Method**	**Primary Tools**
Performance	Automated benchmark testing	k6, k6 browser module
Maintainability	Qualitative expert assessment	Semi-structured interviews
Security	Qualitative expert assessment	Semi-structured interviews
Overall recommendation	Comparative expert assessment	Semi-structured interviews

4.5 Performance evaluation (quantitative)

Performance was evaluated using two complementary tools targeting different aspects of the system. Grafana k6 was used for protocol-level load testing to measure server-side scalability under concurrent load, while the k6 browser module was used to measure client-perceived rendering performance for a single user session.

4.5.1 Test environment

To ensure a consistent and reproducible evaluation environment, each architectural implementation was containerized using Docker. The complete system for each implementation, including the Angular application, CDN, BFF, CMS, and supporting services such as Redis and PostgreSQL, was defined as a single Docker compose configuration. This approach eliminated environmental variability between test runs and ensured that both implementations were evaluated under identical infrastructure conditions. One limitation was that the whole system ran locally on the computer, meaning that the CPU might have been a limiting factor, especially under heavy loads.

4.5.2 k6 load testing

Grafana k6 was integrated as a dedicated service within a separate Docker Compose configuration. This allowed k6 to communicate with the CDN and the Angular server through their localhost ports. The test script simulated a virtual user making a single HTTP GET request to the application, with a randomly assigned personalization cookie (cdn_key) selected from the set of available user segments. This approach ensured that the cache and personalization logic were exercised across all segment variants during each test run.

Each test scenario was executed against two endpoints per implementation: the CDN layer and the Angular application server directly. Testing against the CDN evaluated the

full request including caching behavior, which reflected the conditions a real user would experience. Testing against the Angular server directly bypassed the cache and isolated the rendering and personalization logic, revealing how each implementation performed under load without the mitigating effect of caching. This dual-endpoint approach allowed the results to be distinguished between performance gains attributable to caching and those attributable to architectural strategy itself.

The test consisted of 11 stages that progressively ramped up the number of virtual users from 15 to 1000. This approach evaluated system behavior under increasing loads and identified the breaking point at which the architecture could sustain acceptable performance. To account for measurement variability, the test was repeated five times per endpoint and implementation, and the median value was used for comparison, consistent with recommendations for reducing variance in performance measurements [22].

The following metrics were collected, and for each metric, the mean, median, and standard deviation were calculated:

- **95th percentile response time (ms)**: the response time below which 95% of requests fall, indicating worst-case behavior for the majority of users
- **Error rate**: the proportion of requests that failed during the test
- **Virtual users at failure**: the number of concurrent virtual users at which the system failed
- **Total requests**: the total number of HTTP requests completed during the test
- **Median response time (ms)**: the response time below which 50% of requests fall, indicating typical behavior

4.5.3 k6 browser testing

To complement the protocol-level load tests, the k6 browser module was used to measure client-perceived rendering performance for a single user session. The browser module drives a headless Chromium browser, allowing it to capture the same metrics as a real user would experience when loading the page. Each test simulated a single user fetching the personalized home page with a randomly assigned segment cookie.

Unlike the load tests, the browser tests were not designed to evaluate scalability under concurrent traffic. Instead, they captured the baseline rendering experience for each implementation, how quickly a personalized page was presented to the user when the system is under external load. This was relevant since the two implementations involved different number of network hops and different rendering pipelines, which might have affected the perceived performance even when the system was not under stress.

As with the protocol-level load tests, each browser test was executed both against the CDN and Angular application server directly. Testing against the CDN reflected the realistic user experience, while testing against the Angular server isolated the rendering pipeline without the influence of caching. Each test ran 50 iterations and the full test was repeated five times for each endpoint and implementation. The median value across all runs was used for comparison [22].

The following metrics were collected, and for each metric, the mean, median, and standard deviation were calculated:

- **LCP (Largest Contentful Paint)**: measures the time until largest visible content element is rendered, indicating perceived load speed
- **FCP (First Contentful Paint)**: measures the time until the first content is painted to the screen
- **TTFB (Time To First Byte)**: measures the time until the first byte reaches the client
- **CLS (Cumulative Layout Shift)**: measures the visual stability
- **Personalization Time**: measures the time until the personalized image request is initiated

4.5.4 Data analysis

Due to the small number of test runs ($n = 5$ per condition) the two-sided Mann–Whitney U test was used to compare the two architectures for each metric, supplemented by Cliff's δ as a measure of effect size. A significance threshold of $\alpha = 0.05$ was applied. Both tests are described in Section 3.6.3.

4.6 Qualitative evaluation: maintainability, security, and comparative assessment

Maintainability and security were evaluated qualitatively through semi-structured interviews with software architects and senior developers at Nordea who have experience with the application stack and deployment environment as described in this study.

In addition to evaluating each architecture independently on maintainability and security, participants were asked to compare the two approaches directly and state which they would recommend for a production deployment at Nordea. This comparative assessment included a preference with justification, a confidence rating, a ranking of the three quality dimensions by importance in a banking context, and a reflection on whether the performance data influenced their recommendation. The goal was to capture not only how

each architecture performs on individual dimensions, but which trade-off experts consider acceptable when all dimensions are weighed together.

The interviews were conducted in either Swedish or English according to the participants' preference, recorded with participants' consent, and transcribed. The data was analyzed using thematic analysis, in which recurring themes across participants were identified and mapped to the two architectural approaches. Architecture diagrams (see Figures 2 and 4) as well as sequence diagrams (see Figures 3 and 5) were presented to the participants during the interview to ensure a shared understanding of each approach before discussion.

4.6.1 Interview design

Semi-structured interviews were chosen because they combine a predefined set of topics with the flexibility to pursue follow-up questions and explore domain-specific insights that a rigid questionnaire would suppress [14, ch. 19]. This format allows for structured comparison across participants while preserving room for unexpected findings and professional nuance. As Clark et al. [14, ch. 19] notes, the semi-structured interview is particularly well suited for research where the investigator has a clear focus but also wants to allow the interviewee's perspective to shape the direction of the conversation. In the context of this study, the interviewees possess specialized architectural knowledge that could not be fully anticipated in advance, making the semi-structured format preferable to both fully structured and fully unstructured alternatives.

4.6.2 Participant selection

Interview participants were selected through purposive sampling, targeting individuals with hands-on experience in one or more of the following areas: Angular application development, CMS integration, backend service architecture, or web performance optimization. Purposive sampling is appropriate when the research requires participants with specific expertise or experience related to the phenomenon under study [14, ch. 17]. Four participants were considered sufficient for the scope of this study, given the specialized nature of the domain and the depth of each interview. The interviewees were:

- **P1:** Enterprise Chief Architect with 40 years of experience with stack-level architecture
- **P2:** Solutions Architect with 25 years of experience with content management and web
- **P3:** Developer with around 3.5 years of experience with full-stack development and a frontend focus on Angular

- **P4:** Senior Developer with 10 years of experience with full-stack development and around 2.5 years with isomorphic Angular

4.6.3 Interview procedure

Each interview followed a consistent structure. First, the participant was presented with the architectural diagrams for both implementations (see Figures 4, 2) to establish a shared understanding of each approach before discussion. The interview guide then covered the following areas, applied to each architecture:

- **Ease of understanding and on-boarding**: How easily could a new developer understand and work with this architecture?
- **Maintainability**: How easily can this approach be modified, extended, and maintained over time?
- **Testability and operational cost**: What is the perceived cost and effort of deploying and maintaining this architecture in production?
- **Security posture**: What security risks or advantages does this architecture introduce? Does it expand or reduce the attack surface compared to the alternative, and are there new trust boundaries or exposure points that need to be considered?
- **Comparative assessment**: Which architecture would the participant recommend for production deployment, under what conditions, and with what level of confidence?

4.6.4 Data analysis

Interview transcripts were analyzed using thematic analysis, following the six-phase approach described by Braun and Clarke [11] in Section 3.6.2: familiarization, initial coding, searching for themes, reviewing themes, defining and naming themes, and reporting. The analysis was conducted inductively, meaning themes were derived directly from participants' responses anchored to the three evaluation dimensions of maintainability, security, and comparative recommendation.

Coding was performed manually using a general-purpose document tool to annotate transcripts and record codes alongside the relevant quotes. Each transcript was first read in full to develop familiarity with the data. Initial coding was then performed by tagging segments of text that addressed any of the three evaluation dimensions. Codes were kept descriptive and closer to the participants' own framing at this stage, producing a larger set of granular observations than was ultimately retained in the final theme structure.

In the subsequent phase, codes were grouped and consolidated into candidate themes by identifying recurring patterns of meaning across participants. Themes were refined

and merged iteratively, codes that addressed the same underlying phenomenon under different framings were combined, and themes that initially seemed distinct were collapsed when closer reading showed that they shared a common analytic core. Themes were also reviewed against the original transcripts to ensure that the patterns claimed were genuinely supported by the data rather than imposed during consolidation.

To support pattern recognition and consolidation, the large language model Claude was used in an assistive capacity. Specifically, the model was used to produce summaries of long transcript passages and to provide a second opinion on whether candidate themes accurately reflected the underlying quotes. All analytic decisions, including which codes to retain, how themes should be named and bounded, and which quotes to associate with each theme, were made by the author. Where the model's suggestions diverged from the author's reading, the original transcripts were re-examined to verify the interpretation before accepting or rejecting the suggestion. The final ten themes presented in the Results chapter are the product of this iterative refinement process, mapped to the two architectural approaches to enable a structured comparison grounded in expert judgment.

5 Results

This chapter presents the findings from the performance tests and the qualitative expert interviews. Performance results are organized by test type (load testing and browser testing), while interview findings are structured around the two qualitative dimensions(maintainability and security) and the comparative assessment.

5.1 Performance results

Following are the results for the performance tests. To assess the consistency of results across runs and evaluate statistical significance, box plots and non-parametric statistical tests are reported alongside median comparisons. Statistical significance was assessed using the two-sided Mann-Whitney U test and Cliff's δ effect sizes, as described in Section 3.6.3.

5.1.1 Load testing

Table 2 presents the load test results when requests were made directly to the Angular application server, bypassing the CDN cache. This configuration isolates the rendering and personalization logic of each architecture under increasing concurrent load.

Table 2: Load test results – Angular server endpoint

Metric	**In-App Median**	**BFF Median**	**Difference (%)**
p95 response time (ms)	3100	3060	1.31
Error rate	0	0	0
VUs at failure	24	23	4.35
Total requests	856	761	12.48
Median response time (ms)	1770	1640	7.93

Difference computed as (In-App - BFF)/BFF * 100. For response times, positive values indicate BFF is faster (In-App took longer). For total requests and VUs at failure, positive values indicate In-App handled more.

When testing directly against the Angular server, the BFF architecture showed moderately lower median response times (1640 ms vs. 1770 ms) and slightly lower p95 response times (3060 ms vs. 3100 ms). However, the In-Application architecture handled a higher total number of requests (856 vs. 761) before failure occurred. Both architectures failed at similar virtual user counts (24 and 23 respectively), and neither produced errors before their failing point.

Table 3 presents the load results when requests were routed through the CDN, reflecting conditions closer to what a real user would experience.

Table 3: Load test results – CDN endpoint

Metric	**In-App Median**	**BFF Median**	**Difference (%)**
p95 response time (ms)	422.52	432.93	-2.40
Error rate	0	0	0
VUs at failure	1000	1000	0
Total requests	648856	614470	5.60
Median response time (ms)	4.21	5.22	-19.35

Difference computed as (In-App - BFF)/BFF * 100. For response times, positive values indicate BFF is faster (In-App took longer). For total requests and VUs at failure, positive values indicate In-App handled more.

When testing through the CDN, both architectures sustained the full 1000 virtual users without failure and produced no errors. The In-Application architecture exhibited marginally lower median response times (4.21 ms vs. 5.22 ms), a difference of approximately one millisecond that is unlikely to be perceptible in practice, and marginally lower p95 response times (422.52 ms vs. 432.93 ms), while also handling a higher total number of requests. The differences between the two architectures were substantially smaller than in the direct Angular server tests, consistent with the CDN cache serving cached responses for most of the test duration.

Figure 6 shows the distribution of results across all five runs for both scenarios. The CDN conditions produced noticeably tighter spread than the direct Angular server conditions, consistent with the cache stabilizing response times across runs.

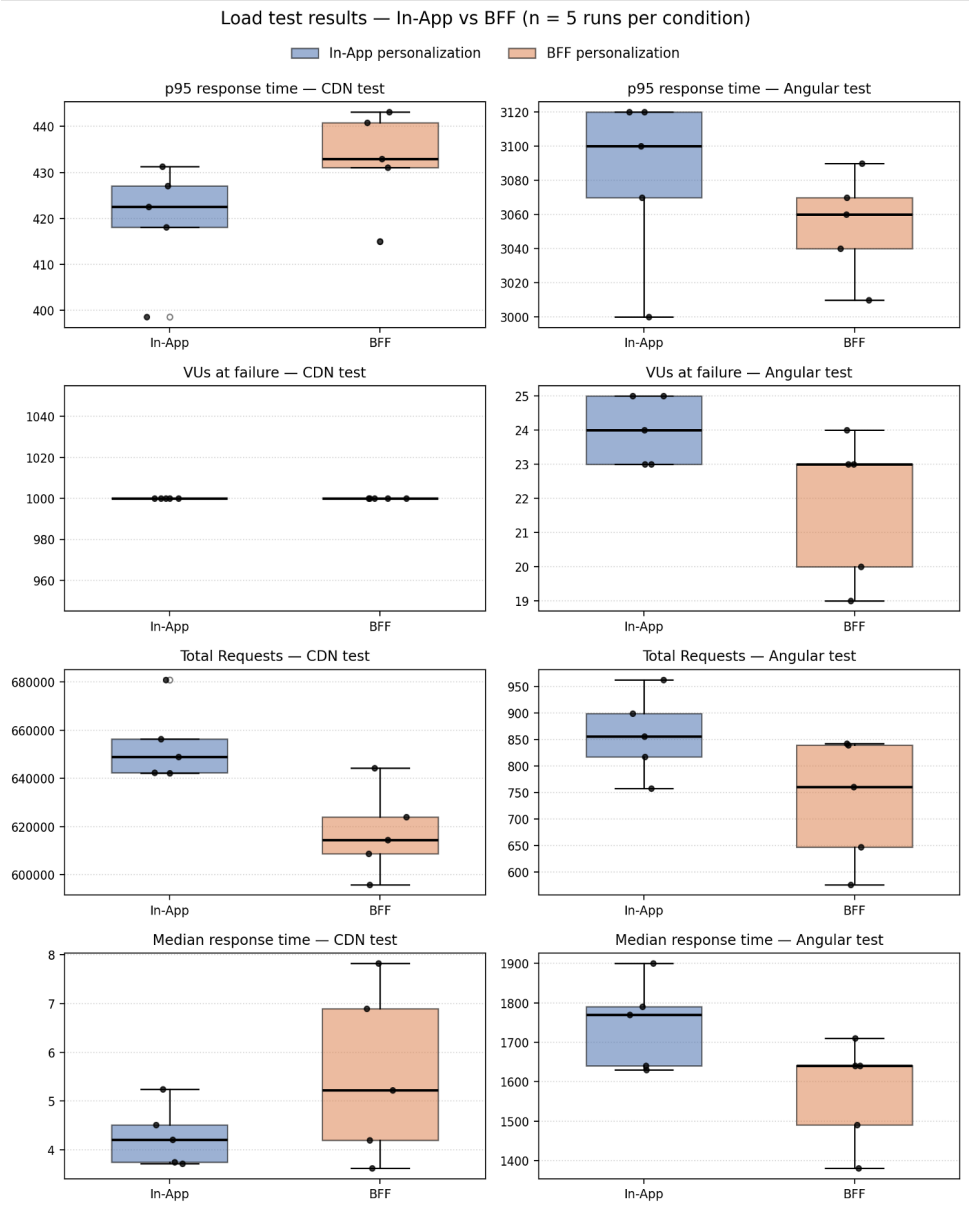


Figure 6: Load test results – distribution across 5 runs per condition, for both the CDN and Angular server scenarios. Box shows the interquartile range, the center line is the median, whiskers extend to min/max, and individual runs are shown as dots.

Table 4 and Table 5 presents the Mann-Whitney U test results and Cliff's δ effect sizes for the Angular server and CDN scenarios respectively. Error rate is excluded from both tables as it was zero in all runs for both architectures and VUs at failure is excluded from Table 5 since it reached max value for all runs.

Table 4: Statistical analysis – Angular server load test (In-App vs. BFF, $n = 5$ per condition)

Metric	**U**	**p-value**	**Cliff's δ**	**Effect size**
p95 response time (ms)	18.5	0.248	0.48	large
VUs at failure	20.5	0.104	0.64	large
Total requests	20.0	0.151	0.60	large
Median response time (ms)	20.0	0.139	0.60	large

Positive Cliff's δ indicates In-App values were higher, negative indicates BFF values were higher

Table 5: Statistical analysis – CDN load test (In-App vs. BFF, $n = 5$ per condition)

Metric	**U**	**p-value**	**Cliff's δ**	**Effect size**
p95 response time (ms)	5.0	0.151	-0.60	large
Total requests	23.0	0.032	0.84	large
Median response time (ms)	9.0	0.548	-0.28	small

Positive Cliff's δ indicates In-App values were higher, negative indicates BFF values were higher

Although none of the Angular server differences reached $p < 0.05$, all four metrics show large effect sizes ($|\delta| \geq 0.474$), indicating consistent directional differences across runs. In the CDN scenario, total requests was the only statistically significant result ($U = 23.0, p = 0.032, \delta = 0.84$), confirming that the In-Application architecture consistently served more requests through the cache. The p95 response time showed a large effect favoring BFF ($\delta = -0.60$) but did not reach significance, while the median response time difference was small in effect ($\delta = -0.28$) and non-significant. The limited statistical power of $n = 5$ should be noted.

5.1.2 Browser testing

Table 6 presents the browser test results when the page was loaded directly from the Angular server.

Table 6: Browser test results – Angular server endpoint

Metric	**In-App Median**	**BFF Median**	**Difference (%)**
LCP (ms)	736	682	7.92
FCP (ms)	384	400	-4.00
TTFB (ms)	229.25	252.35	-9.16
CLS	0.01628	0.00143	1038.46
Personalization Time (ms)	687.79	286.8	139.82

Difference computed as (In-App - BFF)/BFF * 100. For all metrics shown, lower values are better. Positive difference indicate BFF performed better (In-App's measurement was higher).

When tested against the Angular server, the BFF architecture produced a lower LCP (682 ms vs. 736 ms) and a substantially lower personalization time (286.8 ms vs. 687.79 ms). The BFF also achieved a notably lower CLS score (0.00143 vs. 0.01628), indicating better visual stability during rendering, even though both values are within Google's "good" CLS threshold of 0.1 [53]. However, the In-Application architecture recorded lower FCP (384 ms vs. 400 ms) and lower TTFB (229.25 ms vs. 252.35 ms), suggesting that the initial server response was faster.

Table 7 presents the browser test results when the page was loaded through the CDN.

Table 7: Browser test results – CDN endpoint

Metric	**In-App Median**	**BFF Median**	**Difference (%)**
LCP (ms)	530	436	21.56
FCP (ms)	180	168	7.14
TTFB (ms)	2.64	2.69	-1.56
CLS	0.01628	0.00143	1038.46
Personalization Time (ms)	491.6	49.35	896.15

Difference computed as (In-App - BFF)/BFF * 100. For all metrics shown, lower values are better. Positive difference indicate BFF performed better (In-App's measurement was higher).

When tested through the CDN, the BFF architecture outperformed the In-Application approach across nearly all metrics. LCP was lower (436 ms vs. 530 ms), FCP was lower (168 ms vs. 180 ms), and personalization time was substantially lower (49.35 ms vs. 491.6 ms). TTFB was nearly identical between the two (2.69 ms vs. 2.64 ms), which is expected since the CDN served cached responses in both cases. The CLS advantage of the BFF architecture was consistent across both test configurations, although both values remained within Google's "good" CLS threshold. All LCP and FCP values for both architectures also fell within their respective "good" thresholds.

Figure 7 shows the distribution across all five runs for both test scenarios. The personalization time particularly show clear separation between the two architectures with minimal overlap across runs.

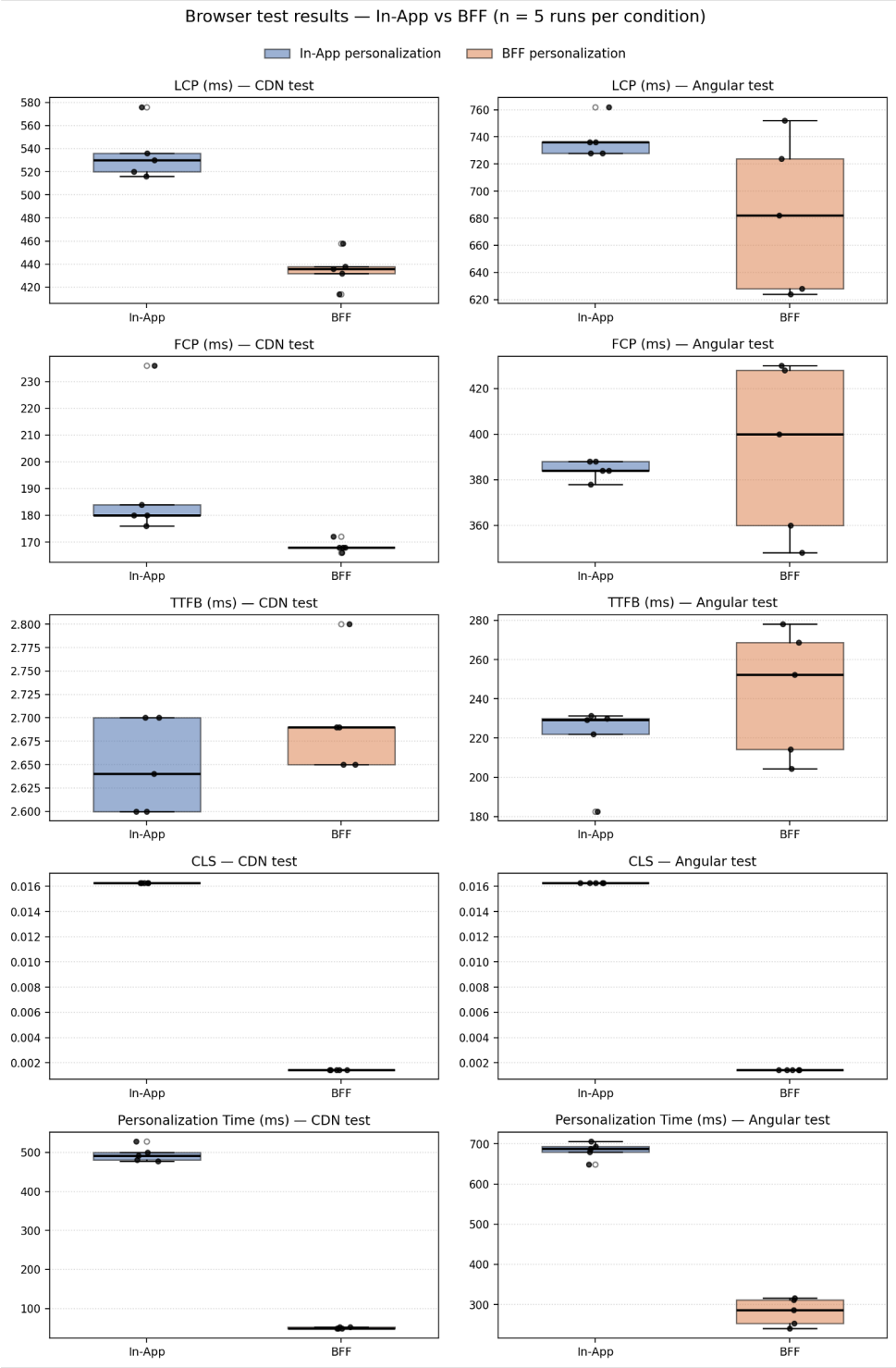


Figure 7: Browser test results — distribution across 5 runs per condition, for both the CDN and Angular server scenarios. Box shows the IQR, the center line is the median, whiskers extend to min/max, and individual runs are shown as dots.

Table 8 and Table 9 present the statistical results for the Angular server and CDN browser tests respectively.

Table 8: Statistical analysis: Angular server browser test (In-App vs. BFF, $n = 5$ per condition)

Metric	**U**	**p-value**	**Cliff's δ**	**Effect size**
LCP (ms)	21.0	0.093	0.68	large
FCP (ms)	10.0	0.674	−0.20	small
TTFB (ms)	8.0	0.421	−0.36	medium
CLS	25.0	0.004	1.00	large
Personalization Time (ms)	25.0	0.008	1.00	large

Positive Cliff's δ indicates In-App values were higher

Table 9: Statistical analysis: CDN browser test (In-App vs. BFF, $n = 5$ per condition)

Metric	**U**	**p-value**	**Cliff's δ**	**Effect size**
LCP (ms)	25.0	0.008	1.00	large
FCP (ms)	25.0	0.011	1.00	large
TTFB (ms)	8.0	0.398	−0.36	medium
CLS	25.0	0.004	1.00	large
Personalization Time (ms)	25.0	0.008	1.00	large

Positive Cliff's δ indicates In-App values were higher

Personalization time and CLS show the maximum possible effect size ($\delta = 1.00, p < 0.01$) in both scenarios, meaning In-App values exceeded BFF values in every pairwise comparison across runs. In the CDN scenario, LCP and FCP also reached $\delta = 1.00$ with $p \leq 0.011$, indicating complete separation between architectures. TTFB was the only metric without a significant difference in either scenario.

5.2 Qualitative interview findings

This section presents the findings from the semi-structured interviews, the transcribed interviews can be found in the Appendix. Four participants were interviewed: an enterprise chief architect with 40 years of experience (P1), a solutions architect with 25 years of experience in content management (P2), a full-stack developer with three and a half years of experience and a frontend focus (P3), and a senior developer with over ten years of full-stack web application experience and approximately two and a half years of work on isomorphic Angular (P4). Following Braun and Clarke's six-phase framework (see Section 3.6.2), the interviews were analyzed inductively, and codes were organized around three evaluation dimensions established by the research design. These dimensions are maintainability, security, and the comparative recommendation.

Ten themes were identified in total, with four under maintainability, three under security, and three under the comparative assessment. Figure 8 provides an overview of

the theme structure and the dimensions in which each theme manifests. The themes are presented in the following subsections.

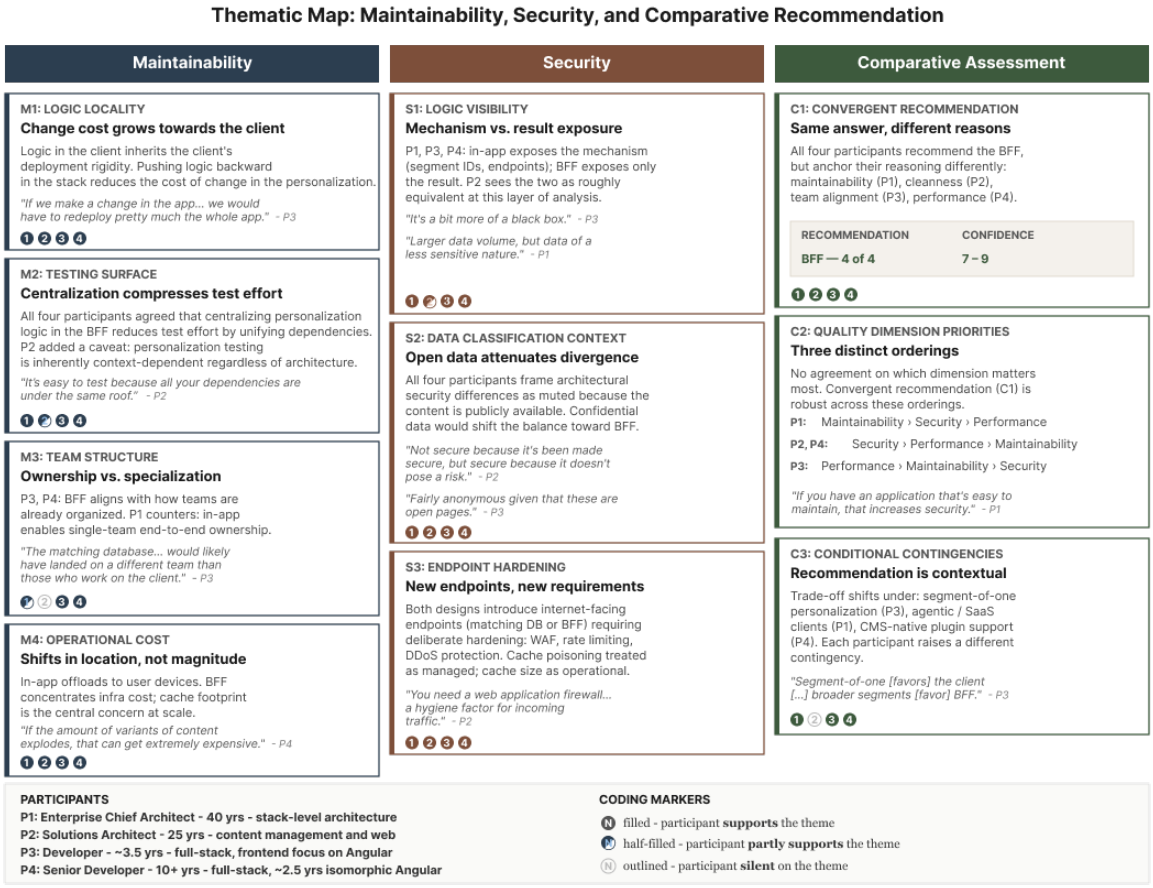


Figure 8: Thematic map of the interview findings. For each theme, filled markers indicate participants who supported the theme, half-filled markers indicate participants who partly support the theme, and outlined markers indicate participants who did not address it.

5.2.1 Maintainability

The following sections present the four themes that emerged under the maintainability dimension.

5.2.1.1 Theme M1: Logic locality and change cost grows towards the client

All four participants framed the placement of personalization logic as a central maintainability factor. The argument was consistent across the interviews. Logic placed in the client is the most expensive to change later, because the client is the hardest part of the system to redeploy. P1 expressed this as an architectural principle, arguing that *"the further back we can keep it [...] the easier it is to maintain and change"* (Appendix C, p.1), and identifying the client as *"the component that is hardest to change [...] because it has to be deployed in somewhat special ways"*(Appendix C, p.1). P3 expressed the same view

in operational terms, noting that any change to the In-Application personalization logic *"would have to redeploy pretty much the whole app"* (Appendix E, p.2). P4 highlighted the bidirectional dependency this creates, changes in the matching system *"force redeployment on your app"* (Appendix F, p.2), making compatibility maintenance a continuous obligation. P2 reached the same conclusion through the opposite framing, observing that with the BFF approach *"You don't need to release the application just because you want to change something in the BFF layer"* (Appendix D, p.2).

A related sub-pattern concerns developer cognition. When personalization logic lives in the client, every developer working on the application must hold the personalization context in mind. P2 noted that the in-application approach *"demands a greater understanding of how the application works, given that it's personalization-aware"* (Appendix D, p.1), while in the BFF design *"I think it gives better insight because you get a uniform response with everything packaged for you. You see exactly what the output is."* (Appendix D, p.3).

5.2.1.2 Theme M2: Testing surface and centralization compresses test effort

All four participants judged that centralizing personalization logic in a BFF compresses the test effort relative to client-side logic. P1 articulated this most strongly, observing that with client-side personalization *"each unique client that has to test on its own"* (Appendix C, p.2), and that *"if you suddenly have 10 clients all doing the personalisation code, then you suddenly have 10 clients that need to be updated."* (Appendix C, p.2). Centralizing the logic in a BFF allows the personalization to be *"test it once with one channel"* (Appendix C, p.2) with the assurance that it will behave identically across consumers. P3 reasoned similarly, and P4 judged the BFF *"a little bit better"* (Appendix F, p.5) on testability for the same reason. P2 agreed with the core point but raised a caveat that cut across both approaches. They rated the in-app solution a 6 on testability, reasoning that *"everything is gathered in one place"* (Appendix D, p.2) with *"all your dependencies are under the same roof"* (Appendix D, p.2), and rated the BFF a 5, noting that the layered architecture introduces trade-offs. One can test the BFF in isolation and mock its response in the application, but *"then it's not a single test"* (Appendix D, p.3). This still aligns with the centralization argument as applied to personalization-specific testing, but P2 saw the trade-off more evenly than the other three participants.

5.2.1.3 Theme M3: Team structure and ownership versus specialization

Three of the four participants (P1, P3, P4) discussed how each architecture maps onto team structure while P2 did not address this dimension explicitly. The dominant view, expressed by P3 and P4, was that the BFF's separation of concerns aligns naturally with how teams are typically organized, with frontend developers focused on the application and a separate group responsible for content and personalization logic. P3 explicitly an-

ticipated that *"the matching database and the whole part of what we present to our users would likely have landed on a different team than those who work on the client."* (Appendix E, p.10). P4 reached the same conclusion but flagged the coordination overhead that is implied with two separate teams, *"human — like management, team management kind of problem."* (Appendix F, p.4).

P1 offered a counter position that ran against their own overall recommendation in this one dimension. When asked specifically about ownership, they argued that the In-Application solution is preferable from a team-effectiveness perspective. A single team responsible for the entire personalization chain, in their view, can move faster and with less coordination risk than a setup in which delivery is segmented across teams. As they put it, *" From a team effectiveness perspective it's always easier to have total end-to-end responsibility. You get to do all the parts, because then you're responsible for entire parts of the chain."* (Appendix C, p.6). By contrast, they saw the BFF design as segmenting delivery across teams, with the BFF team needing to coordinate with the client team and *"a greater risk when you segment the delivery in that way."* (Appendix C, p.6). They further noted that *"traditional frontend development teams haven't always been great end-to-end at being skilled in backend services too."* (Appendix C, p.6), meaning that a BFF-based delivery presupposes backend competence in a team that has not historically been organized around it.

5.2.1.4 Theme M4: Operational cost shifts in location, not magnitude All four participants described each architecture as redistributing operational cost. P1 noted that the In-Application approach *"we push a lot of that logic into the client, those aren't costs that hit us"* (Appendix C, p.3). The computation is offloaded onto user devices rather than absorbed by the operator's infrastructure. P2 observed that the BFF, by contrast, allows the operator to *"dedicate our CPU to the rule engine without giving as much to the application."* (Appendix D, p.3), but this precision comes with the obligation to scale and maintain a separate service. The most concrete cost concern raised was the CDN cache footprint. P1 made the point with a worst case scenario: *"6 million customers and all of them are to have completely individual personalisation — then there's no reuse of anything that's cached anywhere at all"* (Appendix C, p.4). P3 made a similar point: *"10 times 10 times 2 on one page variants. That many pages need to be cached in the CDN."* (Appendix E, p.7). P4 framed the consequence directly: *"if the amount of variants of content [...] explodes for some reason, that can get extremely expensive"* (Appendix F, p.5). This concern was shared across all four interviews. The BFF's caching benefits hold for broadly defined segments but weaken as personalization granularity increases.

5.2.2 Security

The following sections present the three themes that emerged under the security dimension.

5.2.2.1 Theme S1: Logic visibility and mechanisms versus result exposure

Three of the four participants (P1, P3, P4) explicitly distinguished the two architectures on what each design exposes about how personalization works. The In-Application approach makes the personalization mechanism visible at the network boundary since the client requests segment information, receives endpoint URLs, and assembles content itself. P1, after initially placing both architectures at parity, revised their rating downward from 5 to 3 when this point was raised. They observed that the in-application approach exposes *"part of the personalisation data."* (Appendix C, p.4) via *"an internet-published API"* (Appendix C, p.4), whereas the BFF *"don't expose any of the data that reveals how we've personalised — instead it's only the data that goes out, the content."* (Appendix C, p.5). This distinction maps onto the PoLP. P3 observed that with the BFF *"the clients have less knowledge about how the personalization works. It becomes a bit more of a black box what it gets back from the BFF"* (Appendix E, p.10), and P4 made the same point operationally, noting that *"the personalized service is kind of transparent to the web app, the web app doesn't know that much about it, which is good."* (Appendix F, p.7). P1 summarized the trade-off as one in which the BFF produces *"larger data volume, but data of a less sensitive nature"* (Appendix C, p.5).

P2 did not place weight on this distinction. Asked to compare the two architectures on security, P2 responded that *"off the top of my head it's hard to see a major difference"* (Appendix D, p.5) and consistently framed the security comparison as governed by external factors, namely the data classification of the content being served and the hardening of the new endpoints, rather than by the visibility of personalization logic itself. The theme is therefore strongly supported by three participants but is not unanimous.

5.2.2.2 Theme S2: Open data context attenuates security divergence

A consistent qualifying observation across all four interviews was that the open-data context of the application materially reduces the stakes of the architectural security comparison. P2 expressed this most directly, stating that the system is *"not secure because it's been made secure, but secure because it doesn't pose a risk."* (Appendix D, p.4). The data being exposed in both architectures is publicly available content, and the segment identifiers themselves are not business-sensitive. P3 echoed this when noting that the data is *"fairly anonymous given that these are open pages."* (Appendix E, p.8). P4 observed that *"if this data is not sensitive or anything like that, then it's perfectly fine."* (Appendix F, p.6), and P1 placed the security comparison at the midpoint, describing it as *"neither good nor bad"* (Appendix C, p.4), before adjusting for logic-visibility considerations.

This attenuation was qualified, by recognition that the architectural security difference would matter substantially in a confidential-data context. P2 made the conditional explicit, observing that with confidential data, *"The solution would look different with respect to authentication."* (Appendix D, p.4). The client would not be permitted to query the matching database directly, and segment resolution would have to be mediated by an authenticated server-side component.

5.2.2.3 Theme S3: New endpoints, new hardening requirements All four participants stressed that both architectures introduce new internet-facing endpoints, and that neither is secure by default. The matching database in the In-Application approach, and the BFF in the BFF approach, each become attack surfaces that require deliberate hardening. P2 was the most explicit about the BFF case, identifying the connection between the CDN and the BFF as the relevant new exposure point and recommending that *"some form of web application firewall - a WAF - that at least does some basic scanning of the incoming data"* (Appendix D, p.5) be placed in front. P4 raised the same concern with respect to the direct connection from the client to the matching database in the In-Application design, noting that without intermediate protection *"if someone wanted to DDoS that thing then without CDN it's way simpler"* (Appendix F, p.6). P1 noted that the standard checklist of protections *"are always applicable"* (Appendix C, p.5) regardless of which architecture is chosen, and P3 observed that opening the matching database to the internet *"It is not a problem to have it, but it does require that you think about it and protect yourself from it."* (Appendix E, p.8).

Cache-related risks were treated more lightly. P4 dismissed cache poisoning as a managed concern in CDN mediated architectures, since reputable providers *"are doing due diligence"* (Appendix F, p.8), and P2 characterized stale-content risk as *"not a security risk, it's a business risk"* (Appendix D, p.5). The more substantive caching concern raised was operational rather than adversarial. P3 noted that the larger cache surface of the BFF means *"more things that can go wrong [...] when you have a lot of data that has to be stored in the CDN and segmented for different users"* (Appendix E, p.10).

5.2.3 Comparative assessment

The following sections present the three themes that emerged under the comparative recommendation dimension.

5.2.3.1 Theme C1: Convergent recommendation, divergent reasoning All four participants recommended the BFF architecture for production deployment. The confidence rating varied, with P1 at a nine *"I'd probably always recommend the BFF setup"* (Appendix C, p.6), P2 at an eight *"quite clearly - an 8"* (Appendix D, p.5), P3 at *"a 7 maybe, 8"* (Appendix E, p.11), and P4 at *"somewhere with like seven"* (Appendix F,

p.9), but all four converged on the same recommendation. What is more revealing is that the participants reached this consensus through distinct chains of reasoning. P1 anchored on long-term architectural maintainability and reusability across channels observing that *"BFF services become more reusable for many different types of clients - I think that's the biggest advantage"* (Appendix C, p.6). P2 cited operational and scalability cleanliness, judging the BFF *"the cleaner approach on paper* (Appendix D, p.5), and citing *"I get the operational and scalability benefits I typically look for"* (Appendix D, p.5), contingent on resolving the CDN-to-BFF security exposure. P3 emphasized organizational fit and separation of concerns, anticipating that *"the matching database and the whole part of what we present to our users would likely have landed on a different team than those who work on the client"* (Appendix E, p.10). P4 focused on the simpler integration path and the personalization-time performance gap measured in the benchmarks, noting that *"the BFF would be maybe the easier to start with - especially because the integration would be presumably simpler, also based on the numbers"* (Appendix F, p.8).

5.2.3.2 Theme C2: Quality dimensions priorities split by professional background When asked to rank performance, maintainability, and security in order of importance, the four participants produced three distinct orderings. P1 placed maintainability first, security second, and performance third, arguing that *"if you have an application that's easy to maintain, that increases security"* (Appendix C, p.6). P2 and P4 produced the same ordering: security first, performance second, and maintainability third, both citing banking-domain expectations as the basis for foregrounding security. P3 placed performance first, framing it as *"the important one for creating a good user experience"* (Appendix E, p.11), followed by maintainability and security.

5.2.3.3 Theme C3: Conditional recommendations and contextual contingencies Three of the four participants qualified their BFF recommendation with conditions under which the analysis would shift. A fourth contingency, concerning the sensitivity of the data being personalized, emerged from P2's contribution to the security dimension in Theme S2 rather than the comparative assessment directly. The remaining three arose directly from the comparative discussion. P3 explicitly stated that for segment-of-one personalization *"it is easiest to personalize in the client"* (Appendix E, p.11), whereas broader segments favor the BFF. This position was implicitly shared by P1 and P4 in their cache-cost discussions. P1 raised a second contingency concerning the future evolution of clients toward agentic-AI, in which centralizing logic in a BFF would become substantially more important. They reasoned that with agent-mediated interfaces *"you might not even have a client as a traditional client"* (Appendix C, p.4), and that in such a scenario *"we lose almost all control over the client layer"* (Appendix C, p.4). P1 also observed that the surrounding landscape increasingly involves SaaS components that the

client would otherwise have to integrate with directly, judging that *"it would become very difficult from the client to have to incorporate a lot of those SaaS functions"* (Appendix C, p.6) and that these are easier to manage centrally. The BFF's value, in this reading, increases as the client layer becomes less stable and less controllable, and as more functionality is sourced from external services that benefit from a consolidated integration point. P4 raised a third contingency, that the recommendation might shift if the CMS itself supported personalization as a native plugin, eliminating the need for an external BFF entirely. They observed that *"in the best case scenario the CMS itself would have some kind of easy way to build the personalization into it"* (Appendix F, p.9), framing the BFF as a workaround for CMSs that lack native personalization support. P2 did not raise comparable contingencies in their comparative discussion.

5.3 Summary

The performance results showed a mixed picture between the two architectures. At the protocol level, both implementations sustained the full 1000 virtual users through the CDN without errors, and the differences in median and p95 response times were small and inconsistent in direction. The In-Application architecture served more requests through the cache, while the BFF was marginally faster on p95. When tested directly against the Angular server, the architectures failed at virtually identical virtual user counts (24 and 23 respectively), with the In-Application architecture handling more total requests before failure.

The browser-side results showed a clearer separation. Through the CDN, the BFF outperformed the In-Application approach on LCP, FCP, CLS, and personalization time, with all four metrics reaching the maximum effect size ($\delta = 1.00, p \leq 0.011$). The most pronounced difference was in personalization time, where the BFF was approximately ten times faster (49.35 ms vs. 491.6 ms). CLS also showed a consistent advantage for the BFF across both test configurations.

The interview findings produced a clearer convergence. All four participants recommended the BFF architecture for production deployment, with confidence ratings ranging from seven to nine. Ten themes were identified across the three evaluation dimensions: four under maintainability, three under security, and three under the comparative assessment. The maintainability themes centered on the cost of placing personalization logic close to the client, the centralization of test effort in the BFF, the alignment between architecture and team structure, and the redistribution of operational cost between the two approaches. The security themes addressed the visibility of the personalization mechanism at the network boundary, the attenuating effect of the open-data context, and the need to harden new internet-facing endpoints introduced by both architectures. The comparative themes captured the convergent recommendation, the divergent priority orderings under-

lying it, and four contingencies under which the recommendation would shift, namely the granularity of personalization, the controllability of the client, the sensitivity of the data, and the capability of the CMS.

Taken together, the quantitative results show a partial performance advantage for the BFF concentrated in browser-side rendering metrics, while the In-Application architecture sustained marginally higher throughput at the protocol level. The qualitative results show unanimous expert preference for the BFF across three different priority orderings, with the recommendation bounded by four contingencies under which the analysis would shift. These findings are interpreted and discussed in detail in the following chapter.

6 Discussion

The integrated reading of the performance benchmarks and expert interviews points to a conclusion that neither dataset would warrant on its own. The benchmarks showed a mixed picture, with the BFF architecture producing consistently lower LCP, FCP, CLS, and personalization time values when measured through the CDN, while the In-Application architecture sustained marginally higher throughput in the same conditions. The interviews, by contrast, produced a unanimous recommendation in favor of the BFF, with each of the four participants reaching it through a distinct chain of reasoning. The convergent finding is that the BFF is the appropriate architecture for personalization in this application, and that the reasons are not primarily performance-related. This chapter examines why an architectural recommendation can be defensible in a banking domain that conventionally prioritizes performance and security, why that recommendation is strengthened rather than weakened by the fact that participants prioritized the quality dimensions differently, and under what conditions it would shift.

6.1 Performance does not settle the architectural choice

The performance results do not, on their own, support a clear preference between the two architectures. Through the CDN, both implementations sustained the full 1000 virtual users without failure or errors, and the differences in median and p95 response times were small and inconsistent in direction. The In-Application architecture served more total requests through the cache and was marginally faster on the lower-percentile response times (4.21 ms versus 5.22 ms), a difference of approximately one millisecond that, while statistically detectable in some comparisons, falls well below any threshold of human perception and has no practical implication for user experience. The BFF was slightly faster on p95. When tested directly against the Angular server, the architectures failed at virtually identical virtual-user counts (24 and 23), and the In-Application architecture again handled more total requests before failure. The CDN mediated most of the architectural load difference in both directions, which is consistent with what P1 and P2 separately predicted, namely that in a CDN-mediated setup, the architectural choice has limited impact on raw scalability so long as the cache is configured appropriately. This behavior is consistent with the role of CDN edge servers described in Section 3.2.5, where caching absorbs repeated requests without reaching the origin server [1], effectively flattening the throughput difference between architectures once cache hit rates stabilize.

Where the BFF showed a clear advantage was in the browser-side metrics measured through the CDN. LCP, FCP, CLS, and personalization time reached maximum effect size ($\delta = 1.00, p \leq 0.011$), meaning the BFF outperformed the In-Application architecture in every pairwise comparison across runs. Not all of these effects carry equal practical weight. The LCP and FCP differences (530 ms versus 436 ms and 180 ms versus 168 ms

respectively) are statistically robust but small in absolute terms, and both architectures fall comfortably within Google's "good" thresholds for these metrics (LCP under 2.5 s, FCP under 1.8 s, as defined in Section 3.5.1 [52, 53]). A user is unlikely to perceive a meaningful difference between an LCP of 436 ms and 530 ms in isolation, even though the BFF is consistently faster across runs.

The CLS and personalization time differences are more architecturally revealing, and both warrant closer examination than the percentage figures alone suggest. The reported CLS difference of 1038 % is striking at first glance, but is best understood by reading the absolute values: 0.00143 for the BFF versus 0.01628 for the In-Application architecture, a difference of roughly 0.015 on a scale where Google's "good" threshold sits at 0.1 and the "poor" threshold at 0.25 [53]. Neither architecture would be classified as poor on this metric by any external benchmark, and a user loading either page in isolation would be unlikely to consciously register the difference in layout stability. The large percentage gap arises from the small absolute scale on which CLS is measured rather than from a meaningful divergence in user experience. The architectural significance of the CLS difference is therefore qualitative rather than quantitative. The In-Application architecture produces a visible content swap after hydration, as personalized content replaces default content in the DOM, while the BFF returns personalized content in the initial render and avoids this swap entirely. This is precisely the behavior anticipated in the theory, Section 3.3.2 noted that personalization logic executed client-side after hydration can cause visual instability, as user may briefly see default content before personalized content is displayed [24], and Section 3.5.1 identified CLS as particularly relevant in personalization context for exactly this reason [28]. The CLS metric captures this structural difference even when the absolute scores remain within acceptable bounds, and the complete separation across all five runs ($\delta = 1.00, p < 0.01$) confirms that this is a reliable property of the architecture rather than a measurement artifact. The CLS finding did not drive the expert recommendation directly, but it corroborates the personalization time result by confirming that the content swap is both measurable and consistent.

Personalization time is the metric that carries the most architectural weight, and the 896 % difference reported in the table reflects a near-order-of-magnitude gap in absolute terms: 49.35 ms for the BFF versus 491.6 ms for the In-Application architecture, measured through the CDN. Unlike CLS, this gap maps directly onto a difference in perceived user experience. At 491 ms, the personalization resolution is long enough for a user to register that the page has loaded and then shifted, a sequence that erodes the sense of a coherent, immediate response. At 49 ms, the personalization happens below the threshold at which most users would notice it occurring at all. The complete separation across all five runs ($\delta = 1.00, p < 0.01$) confirms that this gap is not an outlier but a structural consequence of the two architectures: as established in Sections 3.3.2 and 3.3.3, the BFF resolves segments and assembles content server-side before the response leaves the origin [40], while the In-

Application architecture defers this work to the client and incurs an additional round of network requests after hydration [24]. This difference cannot be tuned away, as it is a direct consequence of where personalization is resolved rather than of how efficiently either implementation executes. This distinction between server-side and client-side executions is foundational to the isomorphic framework described in Section 3.2.1, where the timing of rendering and data resolution determines the sequence of events the user experiences [26, 24]. Personalization time was also the metric most directly referenced by participants P3 and P4 when explaining their recommendation, making it the single performance result with the clearest line to the architectural conclusion.

This advantage is real but bounded. As P2 observed, perceived performance is governed primarily by underlying response times and TTFB, *"Time to First Byte is perhaps the more important metric over CLS."* (Appendix D, p.6), a view consistent with Section 3.5.1's characterization of TTFB as measuring the latency between a request being sent and the first byte of the response being received [28], on which the two architectures are essentially tied through the CDN. The BFF's browser-side wins matter for perceived quality and visual stability, particularly the elimination of layout shift and the much faster appearance of personalized content, but they do not constitute a performance case strong enough to settle the architectural choice on their own. If performance had been the only evaluation dimension, the comparison would have been close, with the In-Application architecture's throughput advantage offsetting the BFF's rendering advantage on most metrics. The two metrics where the BFF's advantage is clearly meaningful, namely CLS and personalization time, were also the two metrics that participants P3 and P4 referenced when explaining their recommendation. The recommendation in favor of the BFF therefore has to rest on more than the benchmark numbers, and that further support is what the interviews provide.

6.2 Convergence without shared priorities

The strongest evidence in favor of the BFF comes from a structural feature of the interview data rather than from any single participant's argument. All four participants recommended the BFF for production deployment, and all four expressed high confidence in that recommendation. What makes the consensus analytically robust is that the participants reached it through three different rankings of the quality dimensions, with P1 prioritizing maintainability, P2 and P4 prioritizing security, and P3 prioritizing performance. No shared priority structure underlies the convergent recommendation.

This pattern is methodologically more informative than a unanimous recommendation under a shared priority structure would be. If all four participants had ranked the dimensions identically and recommended the BFF, the recommendation would be vulnerable to the objection that it merely reflects a particular professional value system. Because

the participants disagreed on what matters most and still arrived at the same conclusion, each of them found independent reasons within their own priority ordering to favor the BFF. The recommendation is therefore not an artifact of any one ranking, and it does not depend on the reader sharing any particular view of which dimension is the most important. This outcome reflects the triangulation principle that Yin [55, ch. 4] identifies as central to case study validity. When independent sources of evidence converge on the same conclusion, the confidence that can be placed in that conclusion, because it is less likely to be an artifact of any single data source or perspective. Here, the four reasoning chains function as four independent sources of expert judgment, and their convergence on the BFF strengthens the recommendation in a way that agreement under shared priorities would not.

The four reasoning chains underwrite the recommendation from different directions. P1's maintainability-first argument rested on long-term architectural flexibility and cross-channel reusability. The BFF can serve multiple frontends and evolve independently of any particular client, which keeps the cost of future change low. This aligns with ISO/IEC/IEEE 42020:2019's characterization of architectural tactics and patterns as the primary mechanisms through which desired quality properties (including modifiability) are achieved across the system life cycle [25], and with the BFF pattern's documented role as a separation-of-concerns mechanism that allows personalization logic to evolve independently of the frontend [40, 54]. P1 further argued that maintainability is not a tension with security but a precondition for it, observing that *"if you have an application that's easy to maintain, that increases security"* (Appendix C, p.6) and reasoning that an architecture which is easy to change is also easier to patch, audit, and harden over time. This reasoning connects maintainability and security in a way that is consistent with the ISO/IEC/IEEE 42020:2019 standard's emphasis on continuous architectural evaluation and adaptation [25], an architecture that is structurally easier to modify is also one in which security vulnerabilities can be identified and addressed more readily as the system evolves.

P2's security first argument framed the BFF as the cleaner operational and scalability pattern, contingent on resolving the CDN-to-BFF exposure with a web application firewall. This contingency directly instantiates the hardening requirement described for the BFF in Section 3.4.3, where CSRF protection was identified as especially important for intermediary services that act as session-aware proxies [53], and the web application firewall recommendation maps onto the broader principle that internet-facing endpoints introduced by either architecture require deliberate protection regardless of which approach is chosen (Section 3.4.3) [27, 54].

P3's performance-first argument grounded the BFF preference in observed rendering behavior, namely the personalization time gap, the elimination of post-hydration content swap, and the resulting stability of the rendering pipeline. As established in Section 3.3.2,

client-side personalization executed after hydration predictably introduces visual instability because users may briefly see default content before personalized content is rendered [24], and Section 3.5.1 identified CLS as the metric most directly sensitive to exactly this structural property [28]. P3's performance reasoning therefore engages directly with the architectural mechanism that the theory predicted would differentiate the two approaches on rendering behavior.

P4 emphasized the simpler integration path and the team-structural fit, anticipating that the personalization logic and the application would in practice land on different teams. This concern is a practical expression of the separation-of-concerns property that the BFF pattern is designed to provide [40, 54] by isolating personalization logic in a dedicated service, the BFF allows teams with different competences to own different parts of the system without requiring continuous coordination at the code level.

The four arguments do not contradict each other but they triangulate on the same recommendation through structurally different paths. Taken together with the performance results, this triangulation explains why a mixed quantitative comparison can still produce a confident architectural recommendation. The mixed-method design adopted in this study was motivated precisely by the recognition that no single method could adequately assess all three quality dimensions simultaneously (Section 4.1.3), and the convergence across both the quantitative and qualitative strands confirms that the design choice was appropriate. The performance data is sufficient to rule out the BFF being substantially worse on the dimension most likely to disqualify it (performance), but it is not strong enough to settle the choice. The qualitative evidence does the settling, and does so robustly because the convergence is not contingent on which quality dimension the reader weighs most heavily.

6.3 The recommendation is contextual, not universal

Three of the four participants qualified their BFF recommendation with conditions under which the analysis would shift, and a fourth contingency emerged from P2's discussion of data sensitivity. These contingencies are not peripheral. They are the boundary conditions that locate the recommendation in time and context, and without them the convergent finding risks being read as a general architectural claim that the data does not support.

The first contingency concerns the granularity of personalization. P3 stated explicitly that for segment-of-one personalization *"it is easiest to personalize on the client"* (Appendix E, p.11), while broader segments favor the BFF. P1 and P4 reached the same conclusion through their cache-cost discussions, captured in Theme M4 (Section 5.2.1.4). The BFF's performance and operational advantages depend on the cache being able to absorb load across users within the same segment, and this absorption degrades rapidly

as the number of distinct content variants grows. P3's worked example *"10 times 10 times 2 on one page variants."* (Appendix E, p.7) and P1's worst-case framing *"6 million customers and all of them are to have completely individual personalization, then there's no reuse of anything that's cached"* (Appendix C, p.4) both identify the same threshold, namely when personalization granularity approaches one variant per user, the BFF's centralization stops compressing load and starts multiplying it. This threshold is a direct consequence of the caching trade-offs described in Section 3.2.5 324. If cache keys must reflect each user's individual segment, the CDN can no longer serve repeated requests from a stored response, and the efficiency advantage that caching provides under broader segmentation disappears entirely [1, 18]. The recommendation in this study therefore holds for segment-based personalization, and would need to be revisited if the personalization strategy were to move towards per-user content.

The second contingency concerns the future of the client itself. P1 raised the possibility that the boundary between client and server may shift substantially as agentic-AI and SaaS-mediated interaction become more relevant, in a context where the application may not have a traditional client at all. In that scenario, centralizing personalization logic in a BFF becomes substantially more important rather than merely preferable, because the client-side execution environment that the In-Application architecture relies on can no longer be assumed. The In-Application architecture's dependency on client-side execution after hydration, described in Section 3.3.2 as the mechanism though which it orchestrates segment resolution and content assembly [24], is precisely what makes it vulnerable when the client environment is not a controlled browser. The BFF, by resolving personalization server-side before the response leaves the origin (Section 3.3.3)[40], is structurally indifferent to what the consuming client is, which makes it degrade more gracefully as the nature of the client evolves. This contingency runs in the BFF's favor, but it is worth mentioning because it identifies a future condition under which the In-Application option becomes substantially less viable. The In-Application approach degrades as the client becomes less controllable, while the BFF degrades much more gracefully.

The third contingency concerns the sensitivity of the data being personalized. P2 made the conditional explicit, observing that with confidential data *"The solution would look very different with respect to authentication."* (Appendix D, p.4). The client would not be permitted to query the matching database directly, and segment resolution would have to be mediated by an authenticated server-side component. This directly engages the Principle of Least Privilege introduced in Section 3.4.5. The In-Application architecture concentrates data access in the frontend component, which must query both the matching service and the CMS directly, whereas the BFF limits what the application receives to pre-assembled personalized content, reducing the exposure of user-identifying data at the network boundary [16, 46]. As soon as the data classification rises above the open-data context, this exposure becomes architecturally unacceptable, and the BFF's separation of

concerns ceases to be a preference and becomes a requirement. The GDPR principle of data minimization, discussed in Section 3.4.6 [17], reinforces this conclusion. Minimizing the number of components that handle user identifiers is not only a security concern but a regulatory one, and the BFF architecture is better aligned with that principle by design. The recommendation in this study therefore holds for the open-data context of the application under evaluation, and would tighten further toward the BFF in a confidential-data context.

The fourth contingency concerns the CMS itself. P4 observed that the BFF, in this design, functions as a workaround for a CMS that does not support personalization natively, noting that *"in the best case scenario the CMS itself would have some kind of easy way to build the personalization into it"* (Appendix F, p.9). If a CMS-native personalization plugin were available, the architectural problem would change shape entirely. The BFF would no longer live in one layer further back but inside the content system itself. This connects to the role of the CMS as defined in Section 3.2.3. Boukar [10] and Finn and Downie [19] describe the CMS as the authoritative source for content creation, storage, and delivery, and the thesis itself notes that it is a central component in both architectural strategies under evaluation. A CMS that natively supports segment-aware content delivery would effectively absorb the personalization responsibility that the BFF currently carries, collapsing two architectural layers into one. This points to a more general principle that runs through the maintainability themes (M1 in particular), and that is consistent with the ISO/IEC/IEEE 42020:2019 framework's emphasis on architectural tactics as mechanisms for controlling quality properties [25]. The further back in the stack personalization logic can be placed, the lower its long-term cost of change, because the component owning that logic is further from the most volatile layer of the system.

Each of these contingencies originates with a different participant, and together they describe a recommendation whose validity is bounded by four specific conditions, namely the granularity of personalization, the controllability of the client, the sensitivity of the data, and the capability of the CMS. The BFF preference therefore reflects the current state of the application, the current scope of personalization, the current data classification, and the current CMS. This is not a weakness of the finding but a feature of it. Architectural recommendations that do not name their boundary conditions tend to be either trivial or overstated, while the contingencies surfaced here let the recommendation be both confident within its scope and honest about its limits.

6.4 Limitations

The findings of this study should be read against the following limitations, each of which constrains the scope of the conclusion in a different way.

6.4.1 Single-machine benchmark environment.

All performance tests were executed on a single computer, with the Angular application, CDN, BFF, CMS, and supporting services running as Docker containers. This eliminated network variability between components but introduced CPU contention between the system under test and the load-generating tool. The absolute response times reported in this thesis are therefore not representative of production conditions, and the breaking-point measurements (24 and 23 virtual users for the Angular endpoint) reflect the limits of the test host rather than of the architectures themselves. The relative comparison between the two architectures remains informative since both implementations were tested under identical conditions, but readers should treat the response-time numbers as ordinal rather than absolute.

6.4.2 Limited statistical power.

Each test scenario was executed five times, which is consistent with prior recommendations for reducing variance in performance measurements but provides limited statistical power for detecting smaller differences. Several of the load-test comparisons showed large effect sizes (Cliff's $\delta \geq 0.474$) without reaching the conventional significance threshold of $p < 0.05$, which is a known limitation of small-sample non-parametric tests. The browser-test results, where the architectures separated completely across all five runs ($\delta = 1.00$), are robust to this limitation, but the load-test conclusions about throughput and p95 response time should be read as directional rather than conclusive.

6.4.3 Small qualitative sample.

The qualitative findings rest on four interviews. This sample size is consistent with the depth-over-breadth orientation of expert interviewing and was constrained by the specialized expertise required for participation, but it limits the range of perspectives and the confidence with which themes can be claimed to be saturated. A larger sample drawn from the same organization could surface dissenting views or additional contingencies not raised by these four participants. The convergence without shared priorities finding is partially robust to this concern, since it is a structural claim about the relationship between priorities and conclusions rather than a claim that depends on a particular sample being representative, but the specific themes and the thematic structure as a whole would benefit from replication with a larger sample.

6.4.4 Single-organization qualitative sample.

The four interview participants were drawn from a single organization (Nordea) and a single application context. Purposive sampling was appropriate given the depth of expertise required, but the convergent recommendation in favor of the BFF reflects the views

of practitioners working within a particular technology stack, organizational structure, and security culture. The convergence without shared priorities finding is structurally informative regardless of organizational context, because it is a claim about the relationship between participants' priorities and their conclusions rather than a claim about the priorities themselves. The specific reasoning chains are however anchored in this particular setting and may not transfer directly to organizations with different team structures, deployment infrastructures, or regulatory environments. The thematic analysis was also conducted by a single researcher, without inter-coder reliability checks, which is a recognized limitation for the reproducibility of qualitative coding decisions.

6.4.5 Scope of the architectural comparison.

The original research design included an Edge Computing arm in addition to the In-Application and BFF approaches, which was cut from scope due to implementation constraints. The comparison reported here therefore evaluates two of the three primary architectural strategies found for personalization, and its conclusion should not be read as ruling on the relative merits of edge-based personalization. Several of the trade-offs surfaced in the interviews, particularly P1's observations about agentic clients and SaaS-mediated interaction, point toward edge computing as a relevant comparison point that future work should address.

6.4.6 Simulated CMS environment.

The CMS used in this study was a simulation that did not reproduce the production-grade caching or content workflows of a mature CMS. P4's contingency about CMS-native personalization is particularly affected by this, since the simulation could not exercise the scenario in which a CMS supports personalization natively as a plugin. The comparison cannot speak directly to how a personalization-capable CMS would change the architectural choice.

6.5 Contribution and implications

The existing literature on personalization architecture treats the relevant pieces separately. Iskandar et al. [24] compare CSR and SSR on performance grounds without addressing personalization specifically. The BFF pattern is described in practitioner sources [40, 54] as separation of concerns mechanism, but without comparative evaluation against alternative placements of personalization logic. The edge-computing literature [48, 2, 13] describes a third placement option without engaging the others. The Boston Consulting group analysis [12] establishes the strategic value of personalization in banking but does not address how to deliver it. None of these sources compare personalization architectures end-to-end in a real banking application across performance, maintainability, and

security simultaneously. The architectural framework applied in this thesis, based on ISO/IEC/IEEE 42020:2019 [25] concept to treat personalization as an architectural concern governed by tactics and patterns rather than a feature-level decision, provided the conceptual basis for evaluating the two strategies across all three quality dimensions in a unified way. The contribution of this thesis is that integrated comparison, and the particular finding that the architectural recommendation rests primarily on the qualitative dimensions even in a domain where performance is conventionally treated as paramount.

The structural feature of the qualitative data, namely the convergence on the BFF reached through divergent priority orderings, is itself a methodological contribution. It demonstrates that an architectural recommendation grounded in expert judgment can be defended without committing to any particular ranking of the quality dimensions, provided the convergence is examined for its priority-independence. This finding extends the triangulation principle described by Yin [55, ch. 4] and applied in the research design (Section 4.1.3). Triangulation is typically understood as convergence across data sources or methods, but the convergence without shared priorities pattern shows that triangulation can also operate across evaluators with structurally different value orderings, providing a stronger form of robustness than convergence under shared assumptions would offer. This pattern is likely to recur in other architectural decisions where multiple quality dimensions are in tension, and the analytic move of asking whether convergent recommendations are robust to priority disagreement is transferable beyond the personalization domain.

For practitioners facing a similar architectural decision, four implications follow from the integrated findings. First, maintainability deserves more weight in the architectural decision than the performance-focused literature suggests. The performance gap between architectures is mediated heavily by the CDN, while the maintainability gap compounds over time as personalization logic evolves and clients multiply. This is consistent with the ISO/IEC/IEEE 42020:2019 standard's observation that constraints and performance impacts often emerge during implementation and operation phases [25], and with the BFF's documented advantage as a separation-of-concerns mechanism that keeps the application unaware of how personalization is implemented and allows the two to evolve independently [40, 54].

Second, the BFF preference is bounded by the granularity of personalization, and any move toward segment-of-one personalization should trigger a reevaluation of the architectural choice. The cache footprint, that makes the BFF efficient for broadly defined segments, degrades rapidly as personalization approaches one variant per user. This implication follows directly from the caching trade-offs established in Section 3.2.5. When cache keys must encode individual user variants, the CDN edge servers can no longer absorb repeated requests from stored responses, and the scalability advantage that justifies the BFF's additional operational overhead disappears [1, 18].

Third, the BFF's organizational fit depends on the team structure that will own the

personalization logic. The architecture works well either with deliberately coordinated frontend and personalization teams, or with a single team that has full-stack competence covering both the application and the backend service. Its benefits diminish if neither condition is in place, since a frontend team without backend competence is poorly positioned to own a BFF, and an uncoordinated multi-team setup introduces the coordination overhead that the architecture's separation of concerns is supposed to make manageable. The separation-of-concerns property of the BFF [40, 54] reduces coupling between the frontend and the personalization logic at the code level, but it introduces a new coordination boundary at the team level. The teams that own each side of that boundary must have either sufficient overlap or sufficient coordination to manage it effectively. If a single team will own the entire chain and is comfortable with the full stack, the In-Application approach's tighter coupling becomes less of a liability.

Fourth, neither architecture is secure by default. Both introduce new internet-facing endpoints that require deliberate hardening, and the choice of architecture determines which endpoint becomes the relevant attack surface. The In-Application approach exposes the matching database directly to the client, while the BFF concentrates the new exposure in the CDN-to-BFF connection. As established in Section 3.4, both endpoints are susceptible to the attack classes discussed in the theory. The matching database exposed by the In-Application approach is vulnerable to the direct access concerns raised in the context of the Principle of Least Privilege (Section 3.4.5) [16, 46], while the BFF's new endpoint must be hardened against CSRF (Section 3.4.3) [36, 54] and web cache poisoning (Section 3.4.4) [41], particularly where CDN caching interacts with segment-keyed responses. Practitioners should plan for endpoint protection (a web application firewall or equivalent) as part of the architectural decision rather than as a follow-up step.

6.6 Future work

Several directions for future work follow from the limitations of this study. A first direction concerns the granularity contingency identified qualitatively in the interviews. Several participants argued that the BFF's cache efficiency degrades as personalization approaches one variant per user, but the threshold at which this degradation becomes architecturally significant was not measured. A follow-up study that systematically varies the number of distinct content variants, ranging from broad segments to per-user personalization, and measures cache hit rate, CDN storage cost, and response times across that range would identify where the BFF's advantage flips into liabilities. This would transform a qualitative contingency into an empirically grounded decision boundary, giving practitioners a more precise basis for choosing between the two strategies based on their personalization requirements.

A second direction is replication at a larger scale. The performance benchmarks in

this study were executed on a single machine under simulated load, and the qualitative findings rest on four participants drawn from a single organization. Both dimensions of the study would benefit from expansion. On the quantitative side, running the benchmark suite in a distributed cloud environment would remove the CPU contention introduced by the single-machine setup and produce response-time measurements that are representative of production conditions, allowing the absolute figures to be interpreted rather than only the relative comparisons. On the qualitative side, a larger participant sample, drawn from multiple organizations or across different roles within the same organization, would increase the confidence with which the identified themes can be claimed as saturated and surface dissenting views or additional contingencies not raised by the four participants in this study. The convergence without shared priorities finding in particular would benefit from replication with a broader sample, to test whether the priority-independent recommendation holds when a wider range of professional backgrounds and organizational contexts is represented.

A third direction is replication in non-banking contexts. The qualitative findings in this study are anchored in a single organization with a particular security culture, regulatory environment, and team structure. The convergence without shared priorities finding is structurally informative regardless of context, since it is a claim about the relationship between participants' priority orderings and their conclusions rather than about the priorities themselves, but the specific themes and contingencies may not transfer directly to organizations operating under different constraints.

A fourth direction, suggested by P4's observation about CMS-native personalization, concerns the architectural implications of CMS platforms that support personalization as a native capability. The BFF in this study functions in part as a workaround for a CMS that does not natively support segment-based content assembly. If the CMS itself were capable of resolving user segments and returning targeted content, the architectural problem would change shape, potentially eliminating the need for a dedicated intermediary service entirely. Future work should evaluate whether CMS-native personalization produces equivalent or superior outcomes on the same quality dimensions, and under what conditions it represents a viable alternative to a purpose-built BFF.

Finally, while edge-based personalization was considered in the original research design, a comparative evaluation including this strategy remains an open direction for work that specifically targets latency-sensitive personalization use cases at scale.

7 Conclusion

The comparison between the two architectural strategies produced a finding that neither dataset would have warranted on its own. The performance benchmarks showed a mixed picture, with the In-Application architecture sustaining marginally higher throughput at the protocol level while the BFF achieved consistently lower LCP, FCP, CLS, and personalization time through the CDN. The most meaningful performance difference was in personalization time, where the BFF was approximately ten times faster (49.35 ms vs. 491.6 ms), a gap that reflects a structural consequence of where personalization is resolved rather than an implementation detail that could be tuned away. The protocol-level differences, by contrast, were small, inconsistent in direction, and largely mediated by the CDN cache.

The three research questions that guided the investigation can each be answered in turn. On performance (RQ1), the two architectures differed most clearly in browser-side rendering metrics rather than in protocol-level scalability. The BFF achieved complete separation from the In-Application architecture on personalization time and CLS across all test runs ($\delta = 1.00, p < 0.01$), while the In-Application architecture sustained marginally higher throughput through the CDN. Neither architecture was substantially worse than the other on protocol-level performance, but the BFF eliminated the post-hydration content swap that the In-Application architecture structurally cannot avoid. On maintainability and security (RQ2), all four interview participants assessed the BFF as the preferable architecture, citing lower long-term change cost as personalization logic placed in the client is the most expensive to redeploy, centralized test effort, reduced exposure of the personalization mechanism at the network boundary, and better alignment with typical team structures. The security assessment was attenuated by the open-data context of the application but three of the four participants identified the BFF as preferable on the PoLP, exposing only personalized content rather than the segmentation mechanism itself. On the conditions under which each strategy is preferable (RQ3), all four participants recommended the BFF for production deployment, but bounded that recommendation by four contingencies relating to the granularity of personalization, the controllability of the client, the sensitivity of the data, and the capability of the CMS. While RQ2 evaluates the perceived trade-offs of each architecture, RQ3 reveals that architectural suitability is not fixed but shifts as these contextual factors change, and the In-Application approach becomes relatively more viable as personalization granularity increases toward one variant per user, as data sensitivity decreases, and as the team owning the system has full-stack competence and end-to-end responsibility.

Beyond answering the individual research questions, the integrated findings surface a structural feature of the qualitative data that strengthens the recommendation beyond what any single argument would justify. The four participants recommended the BFF

through three different rankings of the quality dimensions, meaning the convergence does not rest on any particular view of whether performance, maintainability, or security matters most. This priority-independent agreement is more robust than a consensus built on shared priorities would be, since it cannot be dismissed as the artifact of a single professional value system.

Two contributions follow from the integrated findings. The first is an empirical one, namely an architectural comparison of In-Application and BFF personalization evaluated simultaneously across performance, maintainability, and security, a combination that does not exist in the prior literature. The second is methodological: the priority-independent convergence described above demonstrates that an architectural recommendation grounded in expert judgment can be defended without requiring participants to agree on what matters most. This pattern is transferable to other architectural decisions where multiple quality dimensions are in tension and practitioners hold genuinely different views about which dimension is paramount.

8 Acknowledgments

I would like to thank my academic supervisor Keni Ren and my external supervisor Karim Mazzoni for their guidance and feedback throughout this work.

I would also like to thank Johannes Bastmark for the good discussion on this subject.

I am grateful to the four interview participants at Nordea who gave their time and shared their expertise. Their insights were central to the findings of this thesis.

Regarding the use of artificial intelligence tools: Claude (Anthropic) [6] was used in two capacities during this work. During the thematic analysis, it was used assistively to summarize long transcript passages and to provide a second opinion on candidate themes, with all analytic decisions remaining with the author. It was also used for language assistance during the writing process, including phrasing and clarity improvements. All intellectual content, arguments, and conclusions are the author's own.

References

[1] Akamai. n.d.. *What Is a CDN (Content Delivery Network)?* Akamai. https://www.akamai.com/glossary/what-is-a-cdn Accessed: 2026-01-22.

[2] Akamai. n.d.. *What is Edge Computing?* Akamai. https://www.akamai.com/glossary/what-is-edge-computing Accessed: 2026-01-26.

[3] Angular. 2023. *Introduction to Angular concepts.* Angular. https://v17.angular.io/guide/architecture Accessed: 2026-01-22.

[4] Angular. n.d.. *Server and hybrid rendering.* Angular. https://angular.dev/guide/ssr Accessed: 2026-01-19.

[5] Angular. n.d.. *What is Angular?* Angular. https://angular.dev/overview Accessed: 2026-01-22.

[6] Anthropic. 2026. Claude (claude-sonnet-4-6). https://www.anthropic.com Large language model.

[7] Andrea Arcuri and Lionel Briand. 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. IEEE, Honolulu, Hawaii, 1–10.

[8] Adam Barth, Collin Jackson, and John C. Mitchell. 2008. Robust Defenses for Cross-Site Request Forgery. In *Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS)*. ACM, Alexandria, Virginia, 75–88. https://seclab.stanford.edu/websec/csrf/csrf.pdf

[9] Stephen J. Bigelow and Nolle Tom. 2024. *state management.* TechTarget. https://www.techtarget.com/searchapparchitecture/definition/state-management Accessed: 2026-02-05.

[10] Moussa Mahamat Boukar. 2012. Content Management System (CMS) Evaluation and Analysis. *Journal of Technical Science and Technologies* 1, 1 (May 2012), 49–57. doi:10.31578/jtst.v1i1.36

[11] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp063oa

[12] Sonia Brodski, Laurent Desmangles, Stefano Fanfarillo, Shervin Khodabandeh, Silvio Palumbo, and Maximiliano Santinelli. 2019. *What Does Personalization in Banking*

Really Mean? Boston Consulting Group. https://www.bcg.com/publications/2019/what-does-personalization-banking-really-mean Accessed: 2026-01-29.

[13] Maurantonio Caprolu, Roberto Di Pietro, Flavio Lombardi, and Simone Raponi. 2019. Edge Computing Perspectives: Architectures, Technologies, and Open Security Issues. In *2019 IEEE International Conference on Edge Computing (EDGE)*. Institute of Electrical and Electronics Engineers Inc., Milan, Italy, 116–123. doi:10.1109/EDGE.2019.00035 Accessed: 2026-01-26.

[14] Tom Clark, Liam Foster, Luke Sloan, and Alan Bryman. 2021. *Bryman's social research methods* (sixth edition ed.). Oxford University Press, Oxford.

[15] Norman Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin* 114, 3 (1993), 494–509.

[16] Shouki A. Ebad and Marwa Amara. 2026. The Principle of Least Privilege in Microservices: A Systematic Mapping Study. *Applied Sciences* 16, 3 (2026). doi:10.3390/app16031495

[17] European Parliament and Council of the European Union. 2016. Regulation (EU) 2016/679 (General Data Protection Regulation). https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679 Accessed: 2026-05-20.

[18] Fastly. n.d.. *What is caching?* Fastly. https://www.fastly.com/learning/cdn/what-is-caching Accessed: 2026-01-22.

[19] Teaganne Finn and Amanda Downie. n.d.. *What is a CMS?* IBM. https://www.ibm.com/think/topics/content-management-system Accessed: 2026-01-22.

[20] Google. 2024. *Rendering for Content-Driven Web Apps.* Google. https://developers.google.com/solutions/content-driven/hosting/rendering Accessed: 2026-01-27.

[21] The PostgreSQL Global Development Group. n.d.. *About PostgreSQL.* The PostgreSQL Global Development Group. https://www.postgresql.org/about/ Accessed: 2026-04-07.

[22] Tjaša Heričko, Boštjan Šumak, and Saša Brdnik. 2021. Towards Representative Web Performance Measurements with Google Lighthouse. In *Proceedings of the 11th International Conference on Electrical and Electronic Engineering, Telecommunication and Computer Science (IELECTRO)*. University of Maribor Press, Maribor, Slovenia, 39–42. doi:10.18690/978-961-286-516-0.9

[23] IBM. n.d.. *What is Redis?* IBM. https://www.ibm.com/think/topics/redis Accessed: 2026-04-07.

[24] Taufan Fadhilah Iskandar, Muharman Lubis, Tien Fabrianti Kusumasari, and Arif Ridho Lubis. 2020. Comparison between client-side and server-side rendering in the web development. In *IOP Conference Series: Materials Science and Engineering (IOP Conf. Ser.: Mater. Sci. Eng., Vol. 801)*. IOP Publishing, Bristol, United Kingdom, 012136. doi:10.1088/1757-899X/801/1/012136

[25] ISO/IEC/IEEE. 2019. *ISO/IEC/IEEE International Standard - Software, systems and enterprise – Architecture processes.* Technical Report ISO/IEC/IEEE 42020:2019(E). Institute of Electrical and Electronics Engineers. 1–126 pages. doi:10.1109/IEEESTD.2019.8767004

[26] Nirmal John. 2025. *Understanding Isomorphic Web Applications: The Future of Universal Web Development.* GIGENET. https://www.gigenet.com/blog/isomorphic-web-applications-guide/ Accessed: 2026-03-10.

[27] KirstenS et al. n.d. *Cross-Site Request Forgery (CSRF).* OWASP Foundation. https://owasp.org/www-community/attacks/csrf Accessed: 2026-04-07.

[28] Grafana Labs. n.d.. *Browser metrics.* Grafana Labs. https://grafana.com/docs/k6/latest/using-k6-browser/metrics/ Accessed: 2026-04-21.

[29] Grafana Labs. n.d.. *Grafana k6.* Grafana Labs. https://grafana.com/docs/k6/latest/ Accessed: 2026-03-17.

[30] Grafana Labs. n.d.. *What is Grafana k6?* Grafana Labs. https://grafana.com/oss/k6/ Accessed: 2026-03-17.

[31] Ming-Chi Lee. 2009. Factors Influencing the Adoption of Internet Banking: An Integration of TAM and TPB with Perceived Risk and Perceived Benefit. *Electronic Commerce Research and Applications* 8, 3 (2009), 130–141.

[32] Katie Mikova. 2025. *Angular State Management: Best Practices to Advance Your Project.* Infragistics. https://www.infragistics.com/blogs/angular-state-management/ Accessed: 2026-01-19.

[33] Andrea Müller, Selina Anke, Sabrina Herrmann, Pia Katz, Christina Leuchtweis, Christina Miclau, Sandra Wörner, and Oliver Korn. 2018. Measuring the Influence of User Experience on Banking Customers' Trust. In *HCI in Business, Government, and Organizations*, Fiona Fui-Hoon Nah and Bo Sophia Xiao (Eds.). Springer International Publishing, Cham, 382–395. doi:10.1007/978-3-319-91716-0_30

[34] Nordea Bank Abp. 2025. Annual Report 2024. https://www.nordea.com/en/doc/annual-report-nordea-bank-abp-2024-0.pdf Accessed: 2026-05-11.

[35] Addy Osmani and Lydia Hallie. n.d.. *Client-Side Rendering*. https://www.patterns.dev/react/client-side-rendering/ Accessed: 2026-04-14.

[36] OWASP. n.d.. *Cross-Site Request Forgery Prevention Cheat Sheet*. OWASP Foundation. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html Accessed: 2026-04-07.

[37] OWASP. n.d.. *Session Management Cheat Sheet*. OWASP Foundation. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html Accessed: 2026-04-07.

[38] OWASP. n.d.. *Testing for Cookies Attributes*. OWASP Foundation. https://owasp.org/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/06-Session_Management_Testing/02-Testing_for_Cookies_Attributes Accessed: 2026-04-07.

[39] Ognjen Pantelic, Kristina Jovic, and Stefan Krstovic. 2022. Cookies Implementation Analysis and the Impact on User Privacy Regarding GDPR and CCPA Regulations. *Sustainability* 14, 9 (2022), 14 pages. doi:10.3390/su14095015 Accessed: 2026-01-27.

[40] Jakub Pepliński. 2026. *Backend for Frontend (BFF): What You Need to Know*. Alokai. https://alokai.com/blog/backend-for-frontend Accessed: 2026-01-26.

[41] PortSwigger. n.d.. *Web Cache Poisoning*. PortSwigger. https://portswigger.net/web-security/web-cache-poisoning Accessed: 2026-04-07.

[42] Redis. n.d.. *Redis as an in-memory data structure store quick start guide*. Redis. https://redis.io/docs/latest/develop/get-started/data-store/ Accessed: 2026-04-07.

[43] Redis. n.d.. *Redis persistence*. Redis. https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/ Accessed: 2026-04-07.

[44] Scott Rose, Oliver Borchert, Stuart Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture*. Special Publication 800-207. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-207

[45] Per Runeson and Martin Höst. 2009. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164. doi:10.1007/s10664-008-9102-8

[46] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. https://www.cs.virginia.edu/~evans/cs551/saltzer/

[47] Amazon Web Services. n.d.. *What is PostgreSQL?* Amazon Web Services. https://aws.amazon.com/rds/postgresql/what-is-postgresql/ Accessed: 2026-04-07.

[48] Nour Alhuda Sulieman, Lorenzo Ricciardi Celsi, Wei Li, Albert Zomaya, and Massimo Villari. 2022. Edge-Oriented Computing: A Survey on Research and Use Cases. *Energies* 15, 2 (2022), 28 pages. doi:10.3390/en15020452 Accessed: 2026-01-26.

[49] Sveriges Riksbank. 2025. *Banksystemet.* Sveriges Riksbank. https://www.riksbank.se/sv/finansiell-stabilitet/det-finansiella-systemet/banksystemet/ Accessed: 2026-01-19.

[50] Kar Yan Tam and Shuk Ying Ho. 2006. Understanding the Impact of Web Personalization on User Information Processing and Decision Outcomes. *MIS Quarterly* 30, 4 (2006), 865–890.

[51] Vike. n.d.. *Hydration Mismatch.* Vike. https://vike.dev/hydration-mismatch Accessed: 2026-03-10.

[52] Philip Walton. 2024. First Contentful Paint (FCP). https://web.dev/articles/fcp Accessed: 2026-05-12.

[53] Philip Walton. 2024. Web Vitals. https://web.dev/articles/vitals Accessed: 2026-05-12.

[54] Paul Williams. 2025. *The Backend for Frontend (BFF) Pattern Explained: Benefits, Challenges, and Best Practices.* Duende. https://duendesoftware.com/learn/the-backend-for-frontend-bff-pattern-explained-benefits-challenges-and-best-pract Accessed: 2026-01-26.

[55] Robert K. Yin. 2018. *Case study research and applications : design and methods* (sixth edition ed.). SAGE, Thousand Oaks, California.

A Interview guide

Interview Guide

Architectural Strategies for Personalization in Isomorphic Web Applications

Gustav Westin — Umeå University, 2026

Session Information

Participant ID	
Role / Title	
Date	
Time (start – end)	
Location / Format	
Language	Swedish / English
Recording consent	Yes / No

Remind participant: This interview is part of a master's thesis at Umeå University, conducted in collaboration with Nordea. Participation is voluntary and responses will be anonymized. The interview will be recorded and transcribed if consent is given.

Part A: Introduction and Context

Thank you for participating. This interview is part of my master's thesis, which investigates architectural strategies for delivering personalized content in an isomorphic Angular application. I have implemented two approaches — an In-Application architecture and a BFF (Backend for Frontend) architecture — and I would like your expert assessment of each.

The interview will take approximately 45–60 minutes. I will first present you with the architectural diagrams for both implementations and walk you through the performance data. Once you have familiarized yourself with the material, the interview will begin. Several questions will ask you to give a rating on a 1–10 scale, but the ratings are just a starting point — the reasoning behind them is what I'm most interested in. There are no right or wrong answers.

[Present architectural diagrams for both implementations: Figure 1 (In-Application) and Figure 2 (BFF). Walk through the performance data. Allow participant time to review and ask clarifying questions before proceeding.]

QA1. Could you briefly describe your role and your experience with Angular, CMS integration, or backend service architecture?

Notes:

QA2. Are you familiar with the BFF pattern or similar middleware patterns in your current or previous work?

Notes:

Part B: Maintainability

The following questions are asked for each architecture. Start with the In-Application approach, then repeat for the BFF approach.

B.1 In-Application Architecture

[Refer to Figure 1. Quick recap: the Angular server renders default content during SSR. After hydration, the client reads the session cookie, calls the Matching DB service to resolve the user's segment, and then makes targeted CMS calls to assemble the personalized content, which replaces the default content in the browser.]

QB1.1. On a scale of 1–10, how easily could a new developer understand and start working with this architecture? (1 = very difficult, 10 = very easy)

Rating: ___ / 10

Follow-ups:

- What aspects of this specific design would be most challenging to onboard?
- Is the client-side personalization flow intuitive, or does it require significant ramp-up?

Notes:

QB1.2. On a scale of 1–10, how easily could this approach be modified or extended? (1 = very difficult, 10 = very easy)

Rating: ___ / 10

Follow-ups:

- How would you extend this design to support a new personalization segment?
- Where do you see the biggest risks for technical debt as the personalization logic grows?

Notes:

QB1.3. On a scale of 1–10, how testable do you consider this architecture? (1 = very difficult to test, 10 = very easy to test)

Rating: ___ / 10

Follow-ups:

- Can the personalization logic be tested in isolation, given that it lives in the Angular application?
- What would integration or end-to-end testing look like for the Matching DB and CMS interaction?

Notes:

QB1.4. On a scale of 1–10, how reasonable do you find the operational cost of running this architecture in production? (1 = very high cost, 10 = very low cost)

Rating: ___ / 10

Follow-ups:

- What monitoring or observability challenges do you foresee, given that personalization happens in the browser?
- How would you handle debugging when a user reports incorrect personalized content?

Notes:

QB1.5. On a scale of 1–10, how well does this approach align with existing team structures, tooling, and deployment practices at Nordea? (1 = poor fit, 10 = excellent fit)

Rating: ___ / 10

Follow-ups:

- Which parts would fit naturally, and which would require organizational change?
- Would this require changes to existing CI/CD pipelines or team ownership boundaries?

Notes:

B.2 BFF Architecture

[Refer to Figure 2. Quick recap: a dedicated BFF service sits between the Angular application and the CMS. The BFF reads the session cookie, resolves the user segment via the Matching DB, orchestrates CMS calls, and returns pre-assembled content. The Angular application is unaware of personalization — it simply renders what the BFF returns.]

QB2.1. On a scale of 1–10, how easily could a new developer understand and start working with this architecture?

Rating: ___ / 10

Follow-ups:

- Does having personalization encapsulated in a separate service make onboarding easier or harder?
- How does the separation of concerns affect a developer's mental model of the system?

Notes:

QB2.2. On a scale of 1–10, how easily could this approach be modified or extended?

Rating: ___ / 10

Follow-ups:

- Does moving personalization out of the Angular application make it easier to evolve the logic independently?
- Where do you see risks of added complexity from the separate service?

Notes:

QB2.3. On a scale of 1–10, how testable do you consider this architecture?

Rating: ___ / 10

Follow-ups:

- Does having the BFF as a separate service make it easier to test personalization logic in isolation?

- What complications does an additional service introduce for integration and end-to-end testing?

Notes:

QB2.4. On a scale of 1–10, how reasonable do you find the operational cost of running this architecture in production?

Rating: ___ / 10

Follow-ups:

- How significant is the overhead of maintaining and monitoring an additional service?
- Does the BFF give you better or worse observability into personalization behavior compared to the In-Application approach?

Notes:

QB2.5. On a scale of 1–10, how well does this approach align with existing team structures, tooling, and deployment practices at Nordea?

Rating: ___ / 10

Follow-ups:

- Does the BFF fit naturally into Nordea's current microservices landscape?
- Who would own the BFF service, and would that ownership boundary feel natural?

Notes:

Part C: Security

The following questions are asked for each architecture.

C.1 In-Application Architecture

[Refer to Figure 1. In this design, the client reads the session cookie after hydration, sends it to the Matching DB service to resolve the user's segment, and then makes targeted CMS calls from the browser.]

QC1.1. On a scale of 1–10, how secure do you consider this architecture overall? (1 = many concerns, 10 = very secure)

Rating: ___ / 10

Follow-ups:

- What concerns you most, and what do you see as the main security advantage?

Notes:

QC1.2. Given that personalization happens in the browser, the client needs to communicate with the Matching DB service and the CMS directly. What risks does this client-facing surface introduce?

Follow-ups:

- Does exposing the Matching DB endpoint to the browser concern you?
- How would you protect the CMS from being called with arbitrary parameters?

Notes:

QC1.3. How do you assess the risk of cache-related issues in this architecture, such as personalized content being cached at the CDN and served to the wrong user segment?

Follow-ups:

- What cache key strategies would you recommend, given that personalized content is assembled client-side?
- Does the CDN-level caching of the default SSR response introduce any risk?

Notes:

QC1.4. From the perspective of the principle of least privilege, how do you evaluate this architecture?

Follow-ups:

- Does the browser (and therefore the user) have more access than it needs?
- Could a malicious client enumerate or extract data from the Matching DB or CMS?

Notes:

QC1.5. Are there any other security concerns specific to this architecture that you would want to raise?

Notes:

C.2 BFF Architecture

[Refer to Figure 2. In this design, the BFF is the only component that reads cookies and resolves segments. The browser talks to the Angular server, which talks to the BFF. The Matching DB and CMS are only reachable from the BFF.]

QC2.1. On a scale of 1–10, how secure do you consider this architecture overall?

Rating: ___ / 10

Follow-ups:

- What concerns you most, and what do you see as the main security advantage compared to the In-Application design?

Notes:

QC2.2. The BFF acts as a session-aware proxy between the frontend and the backend services. How do you assess CSRF risk in this design?

Follow-ups:

- What mitigation strategies would you apply (anti-CSRF tokens, SameSite cookies, origin validation)?
- Does the BFF being an internal service reduce or shift the CSRF surface?

Notes:

QC2.3. How do you assess the cache-related security risks in the BFF architecture, particularly around content leakage between segments?

Follow-ups:

- Does centralizing caching in the BFF make it easier or harder to prevent segment-based leakage?
- What cache key design would you recommend for personalized responses?

Notes:

QC2.4. From the perspective of the principle of least privilege, how does this architecture compare to the In-Application approach?

Follow-ups:

- Does moving the Matching DB and CMS behind the BFF meaningfully reduce the attack surface?

Notes:

QC2.5. Are there any other security concerns specific to this architecture that you would want to raise?

Notes:

Part D: Comparative Assessment and Closing

QD1. Overall, which architecture would you recommend for a production deployment at Nordea?

☐ In-Application ☐ BFF ☐ Depends (explain)

Follow-ups:

- What was the deciding factor in your recommendation?
- Are there specific conditions (team size, scale, compliance, time-to-market) under which you would change your recommendation?

Notes:

QD2. On a scale of 1–10, how confident are you in your recommendation? (1 = very uncertain, 10 = very confident)

Rating: ___ / 10

Follow-ups:

- What additional information would increase your confidence?
- What, if anything, are you unsure about?

Notes:

QD3. If you had to rank the three quality dimensions — performance, maintainability, and security — by importance for Nordea in this specific context, how would you rank them?

1. _____
2. _____
3. _____

Follow-ups:

- Why that order?
- Would the order change for a different part of the Nordea platform?

Notes:

QD4. Looking at the performance data, did the results surprise you, or did they match what you expected based on the architectural diagrams alone?

Follow-ups:

- Did the performance data change your preference for one architecture over the other?

Notes:

QD5. If you could modify either architecture, what specific change would have the biggest impact on your overall assessment?

Notes:

QD6. Is there anything else you would like to add that we haven't covered?

Notes:

[Thank the participant for their time. Confirm whether they would like to review the transcript or be notified of the final results.]

B Informed consent form

Informed Consent Form

Research Interview — Master's Thesis

Umeå University — Department of Applied Physics and Electronics

Study Title

Architectural Strategies for Personalization in Isomorphic Web Applications

Researcher

Name: Gustav Westin

Affiliation: Umeå University, Department of Applied Physics and Electronics

Supervisor: Keni Ren

Email: gustav-westin@outlook.com

Purpose of the Study

This study investigates architectural strategies for delivering personalized content in an isomorphic Angular web application, conducted in the context of Nordea. The purpose of this interview is to gather expert assessments of two architectural implementations with respect to maintainability and security. Your professional experience and judgment will contribute to the qualitative evaluation of these architectures.

What Participation Involves

You will be asked to review two architectural diagrams and answer questions about each architecture's maintainability and security characteristics. The interview will take approximately 45–60 minutes and will be conducted in Swedish or English according to your preference.

Data Handling and Confidentiality

With your permission, the interview will be audio-recorded and transcribed. All data will be handled confidentially. Your name and identifying details will be anonymized in the thesis and any resulting publications. Recordings will be stored securely and accessible only to the researcher and supervisor. All recordings will be deleted after the thesis has been examined and graded. Transcripts will be anomymized and put in the appendix of the thesis.

Voluntary Participation

Your participation is entirely voluntary. You may decline to answer any question, and you may withdraw from the study at any time without providing a reason and without any consequences. If you withdraw, any data collected from your interview will be deleted upon request.

Consent

By signing below, I confirm the following:

- ☐ I have read and understood the information provided above.
- ☐ I have had the opportunity to ask questions and have received satisfactory answers.
- ☐ I understand that my participation is voluntary and that I may withdraw at any time.
- ☐ I consent to the interview being audio-recorded and transcribed.
- ☐ I understand that my responses will be anonymized in the thesis and any publications.
- ☐ I agree to participate in this study.

Participant

Name (printed)

Signature

Date

Researcher

Name (printed)

Signature

Date

This form will be stored securely by the researcher. A copy will be provided to the participant upon request.

C Interview transcript – P1

Interview Transcript

I = Interviewer P = Participant

I: So, your role and your experience — if you have any experience with Angular, CMS, and backend?

P: Role? At Nordea I'm enterprise chief architect, consultant. I have a background in the industry going back 40 years. I've worked with application development on virtually every platform, but grew up on mainframes. I was at Nordea when we introduced three-layer architecture. I was part of building all of NVF — Angular, Java — those parts, the whole phase. I've been chief architect here a number of times. I've worked with content marketing for the last 15–20 years. So yes, quite a lot of experience. Above all I have a lot of experience building this type of application.

I: If we start with the in-app solution then. If you were to put it on a scale of 1 to 10 — how easy it would be for a developer to get their head around what we're going to do — and that goes for building it and maintaining it.

P: And then we're talking about the whole thing?

I: The whole thing — we're imagining we have an app that isn't personalised.

P: Yes, exactly. The client app including building that function for the matching database parts. So in terms of building it, I don't actually think it's... Do you want it on some scale or?

I: Yes, 1 to 10. 10 being easiest.

P: 10 easiest. Yes, well I would... Yes, that's right, easiest. For me that would land somewhere around a 3 — complexity-wise. Because it becomes quite a lot of complexity. There ends up being quite a lot of code to keep track of up in the client app. And you'd ideally want to keep that as clean as possible — that's my view at least. So as little logic as possible up in the client app. The main reason being that it's the component that is hardest to change. Because it has to be deployed in somewhat special ways. So that's the big thing. Then the matching database itself — that component isn't significantly more complicated than building the equivalent BFF component. But the big difference I'd say is precisely that there's a lot of heat up in the client application. And from a maintenance perspective... we want to avoid pushing functionality out into the client layer, because that's the hardest part. The further back we can keep it in our infrastructure stack — which is purely logical — the easier it is to maintain and change. And we have from that layer downwards an established infrastructure for API variant management and so on. So that's really the biggest difference.

I: But if we take the same scale — if we needed to change how personalisation works, or we want to add new segments. How easy would you say — or difficult?

P: We're back to that. It gets knottier. It's harder. So I think I'd want to keep it up there at the same three in complexity. It depends a bit on... I assume it doesn't matter whether it's the web or mobile client. We've set aside a number of areas for personalisation messages. I assume we don't render them... Well, maybe we do in fact. If we want to be a bit precise, we'd want to be able to change the whole look and feel — purely theoretically. Normally you set aside structural areas: here I should be able to personalise, here I can do what I want, but this part is standardised. But if you think that everything is flexible, if you're going to take personalisation all the way out, then there becomes a whole lot more to keep track of all the time. Whereas in a central function... no, that's hard to change regardless I'd say. But it depends more on the degree to which you choose to personalise. If it's like what we do today — content-based, where we have banners or areas where we can push out different types of messages to members — that's a much simpler thing to manage. But say you come

into the mobile app and it's not the standard overview that's there, because I want something completely different. If you need to be able to change the main content and personalise it based on my needs — then it would become far more complex code in the client application than if you rendered it in a middleware layer as I see it.

I: But is it a bit more... customisation? If you think about wanting a dark mode and showing a different page or something like that.

P: No, I'd say it like this — things like that, absolutely. But I might not want to land on the account overview, I might want to start by landing directly in payments, because I do that nine times out of ten. So I want the app to behave completely differently when I come into it for the first time. If you need to be able to do all of that, then it needs to behave differently. And then I think there become more things to think about, the further out I push it.

I: Yes, I agree. And then I understand that the biggest risk for the technical side is that it sits far out?

P: Yes, well — the further out it sits, the... There are two things I'd say. Partly it's harder to test. That might sound a bit odd perhaps, but it becomes harder to test things that sit further out in the chain. Because it's easier to test central components — then I can use loads of different services. Here I can't test the last component, the one furthest out, at all really. That's one part. It's always more sensitive to deploy in that layer. If you take a mobile app, you have to do an app update via the App Store, if you're on Apple. That process is much more cumbersome. Deploying an internal matching database component or a BFF component that does a larger part of the job — I can deploy that three times a day if I want. Because I can version-manage it in a good way. I can't do that out there in the same way. Equally, if I want to test different types of behaviour, if I want to A/B test — is this good or bad? Then it's much easier to do that in a central component. Because then I can choose in the component based on some algorithm. If I'm going to code the same thing out in the client, then I have to code it twice out in the client. There are so many things that become hard to do in those kinds of change scenarios. That's why it's much more complicated.

I: If we go into testability on a scale, 1 to 10.

P: Testability I don't think differs that much in itself — testing the functionality, because it should be fairly transparent for development. But it's built on having the right relationship on the channel side. Because if you say you have a channel component, then you can test all your personalisation down in the BFF once. You can do that with the client and also where you render it centrally. But if you suddenly have 10 clients all doing the personalisation code, then you suddenly have 10 clients that need to be updated. Looking at the other side — that you render it centrally to one component instead — then you test it once with one channel and you know it will behave the same way. It becomes much easier to test and there's much less code to test above all. It also contributes to the fact that pushing things back in the technical layers means fewer test cases to write — a simple setup with one channel and component with fewer dependencies.

I: So it gets quite a poor test score or is it quite bad?

P: Yes, it gets quite a poor test score given that it's each unique client that has to test on its own. Since all the personalisation code sits in the client — if I have two components I have to do the same job in two places. Which means it gets a much worse test score. If you're grading it again it comes in at one or two. Partly I can't test it until the client is finished, and if I have several clients doing the same thing then all of them have to update their components. Things like that need to be compared against each other, and so on.

I: If you have the app as it is now and then you'd introduce personalisation — how do you think the operational cost would increase, and by how much in some way, no fixed figures.

P: What are we counting as operational cost then? Is it just maintenance or development?

I: It's maintenance and the fact that we need to put resources into this. We might need to have more resources in a backend layer. Do we need to scale up the Angular servers?

P: I wouldn't want to say that... If we're back to having greater personalisation sitting out in the client, then the central components aren't that much affected by it. It doesn't increase there at all — it's negligible. Then we go the other way on the scale, 7–8: this isn't more complex, it doesn't become more expensive for us to do these things — not on that side. On the client side, however, it becomes enormously more expensive depending on how many clients we have. They push in both directions. The other part of the operational cost is going from a non-personalised to a personalised state. Since we push a lot of that logic into the client, those aren't costs that hit us — it actually reduces the cost for us. The user picks that up out on their device or on their unit.

I: If we move over to the BFF architecture — basically the same questions. How easy it would be for a developer to understand and start working with this architecture?

P: If we talk about architecture here with us, it's well-established architecture — it's not either way. But there shouldn't be any complications. Since we talked about multiple clients in the previous case, there's a risk that it becomes multiple BFF clients too, since backend for frontend means it's different from different frontends. Then there's the question of how much of it is reusable. There might need to be at least one for mobile, and then the question is whether there are multiples — and then maybe one for web, since there are slightly different things to be rendered. But the main part of the logic is reusable to a large extent at least. But understanding it is probably fine. There is as I said a risk that you need to build multiple BFFs — it depends on what it is, but this becomes more or less device-specific.

I: But do you think it becomes easier to understand given that you've centralised all the personalisation logic?

P: Yes, I think so.

I: You imagine that there might have been two different teams — one team sitting doing the app and one team that would do a BFF.

P: I think that's how it'll be. I think it becomes easier to understand because there's less complexity between the teams in that way I think. So from that perspective I think it's easier to understand the architecture. It's a clearer separation and it aligns very much with many other things we do architecturally — that we move logic as far back as possible in our layers. Even though display logic belongs in the frontend layer, the rendering otherwise we want to push behind as far as it goes.

I: If we put it on the scale then?

P: Right, there it is again. Seven then.

I: If you needed to extend or modify — add more segments or wherever — how easy would that be on a scale?

P: Again I perceive it as easier with this type of architecture. It's a central component, it's easier to modify, it's easier to test because there are many other ways. In the previous scenario I have to deploy a new client in order to test it. I can use the same client. I can make a different call out there just to make sure I get a different version of it. It becomes a much less complex change to test. Then the change itself might not differ that much — it's still the same personalisation we want to achieve. It doesn't matter so much where I do something, but the deploy and the test of it is simpler. That's probably back to seven again then.

I: And then the testability that you touched on?

P: I think that's higher — it's certainly eight — precisely because it's easier. The whole setup here is easier to test for a central component. I have easier version management in a smooth way, I don't need to make changes out in the clients in the same way. I can make very small incremental changes to BFF clients in a simpler way than I can out in the client — because that has to be finished in some way. Here you can do it a bit more incrementally, so I think it's considerably easier to test it. If you've made a central error you notice it already from what the BFF client puts out, while the other components can be essentially unchanged.

I: What about a cost increase around this then?

P: Here we get resource consumption in our infrastructure, so that will go up in this case. How much that's balanced against other things is hard to say. In the worst case scenario here, users are sitting on very old clients, which means we need to have multiple versions of our central services running for much longer — because we haven't been able to get the clients to upgrade to the latest things. There's a risk here that central costs go up quite a lot — increased consumption on servers. Now the total number of calls might not actually get bigger. But there's a risk that we have to maintain multiple different versions of central components for longer. Very hard to say without measuring it. But on the other hand I think it's easier to test and maintain — so to speak — so I put it at five.

I: Something I've noticed is that the CDN cache becomes much larger since we cache entire pages.

P: That's something we talked about the other day. If we now take our scenarios — 6 million customers and all of them are to have completely individual personalisation — then there's no reuse of anything that's cached anywhere at all, except in my own traffic. In the other scenarios where we do more towards the clients there will be much more that's reusable in the cache. At the same time most sections are fairly short-lived today — it gets cleared pretty often anyway. But absolutely, costs go up there.

I: If we go into the security thinking — we go back to the in-app solution. 1–10, what's the immediate feeling of security? If we take the case that it's only open data to begin with.

P: I'd want to say that security-wise there won't be a big difference between these two, as I see it. I can't see that there should be any major difference. I'd want to compare it to five — it's neither good nor bad. We've talked about the fact that we control all our services and clients. If you look ahead now — agentic AI where agents are controlling things — you might not even have a client as a traditional client. It might be more of an interface where you can talk, or anything at all, a combination of things. In that case we lose almost all control over the client layer. If you take that extreme scenario versus having a fully rendered client — we have total control over everything and we can put sufficient security into it, so that we know this is a trustworthy client that we've generated all those things in, with the security mechanisms we can build in. It's clear that there's increased uncertainty out in the client layer. But in these two cases that's not really what we're talking about — no massive difference.

I: If you think a bit about the principle of least privilege — what you share outwards and what you don't. Now it might not be commercially sensitive that you belong to this segment. But it does become more visible to the client that this is personalised, compared to if we have a BFF solution where everything is hidden.

P: Then maybe we should actually step back to the previous scenario. When we take that scenario in the client part — when we do the rendering out there — then we'll need to expose that data outwards. In terms of data content. That's much worse. Especially if that's an internet-published API. Which spontaneously I'd say we won't be able to let just anyone call. It has to be a trustworthy client calling this. In order for us to know that the risk of exposure in the client is lower — because we're actually exposing part of the personalisation

data. So from that perspective security becomes more important. It becomes a bigger problem in the client rendering part. In-app security-wise — then we should set a lower value?

I: Yes, exactly.

P: Then we should put three on it. Since we expose more. Thanks, I'd completely missed that of course.

I: But nothing else around the in-app solution?

P: No, not as I see it. We need to know it's our client.

I: If we go to the BFF solution then?

P: That gets a better score again. That gets a six instead. There we don't expose any of the data that reveals how we've personalised — instead it's only the data that goes out, the content. Then one could, if you're a bit clever, figure out other things. It becomes less sensitive data that we send out.

I: Larger data volume?

P: Larger data volume, but data of a less sensitive nature.

I: What you've touched on is that we should know it's a client we trust?

P: That becomes less important from that perspective. Now it depends on whether this is an authenticated or properly authenticated mode. There's a slight difference. But if it's open pages it doesn't matter — then it's open information. It's a web browser, we don't have control over what it is. Is it a client where we run in logged-in mode, then we're still of the opinion that we want to know what's out there. It shouldn't be just anything. But we can flip it around — same type of thinking. This BFF might be something you'd want to reuse in an open banking context, purely theoretically. This is an open API for other payment providers where you can fetch information from us. We want to be transparent that a certain part of the personalisation for the customer travels with them regardless of which channel they use. It differs slightly depending on where you are.

I: You don't see any problem with cross-site request forgery, cache poisoning, and all those things?

P: All these measures are always applicable. We have to go through that whole checklist regardless. I don't spontaneously think it's worse in that type of setup — it's always there.

I: But as long as we secure the connection between the CDN and the BFF?

P: All connections in — whether it's to the matching database or the BFF — we need to have tight control over. And really make sure there's no one sniffing on the way. Now we're in a bank. That's exactly what the bank has built up a very high trust factor around over the years — that we don't make mistakes. We don't expose your data to anyone who doesn't have authorisation for it. We don't accidentally show your data to someone else. We ensure our services are so secure that no one else can hack them. We really do everything we can in that segment. If we were to compromise on that part, we're done as a brand. A very large part of our energy goes into ensuring we're not exposed to that.

I: If you had to choose one of these two?

P: I always choose BFF solutions like this. That's because I drew that three-layer architecture a great many years ago. Two things. Long-term and from a maintenance perspective I appreciate the simplicity of being able to change central components with greater flexibility than distributed ones — that's the biggest reason I'd say. Those aspects that are about performance and volumes — there's so much happening in the communications sector, even beyond mobile devices. With 5G, that type of infrastructure

that has arrived — we have virtually no network issues. Response times are so good in the network, even for larger data volumes. Not many people complain when walking around town streaming on YouTube. Even if those things increase proportionally — that we push costs onto ourselves — well, one can have opinions on that. I'd still always choose that type of setup. Precisely that BFF services become more reusable for many different types of clients — I think that's the biggest advantage.

I: If you were to build a new personalisation team — does ownership become easier? If you have a BFF solution here, where part of it is also out on the clients in the specific applications?

P: You know what? From that perspective I think the in-app solution is better. Because then you really become solely responsible for things. It's simpler to ensure. Because then you have to have the competence in it, and you have to have the competence with the service function that exists. You have to understand all these parts. From a team effectiveness perspective it's always easier to have total end-to-end responsibility. You get to do all the parts, because then you're responsible for entire parts of the chain. In the distributed setup when we do a BFF, then the BFF team will need to talk with those doing the things and you others sitting on the client — and test that it works. There's a greater risk when you segment the delivery in that way. Then it's possible you can succeed in getting the teams to work. But traditional frontend development teams haven't always been great end-to-end at being skilled in backend services too. Usually it's been another team that builds central services. I'd say that's a disadvantage for the BFF setup and an advantage for the in-app solution.

I: If you put your recommendation — how confident are you in it on a scale?

P: How confident am I in all my answers?

I: No, in this recommendation. You know, you might have changed a bit.

P: Low score was a good grade?

I: No, high score is a good grade, okay.

P: So how confident am I in my recommendation? I'm always completely certain. So there's a small margin for wiggle room. I'd probably always recommend the BFF setup. Because I think it's easier. I think it suits us better to build it. It places somewhat higher demands on the teams — you need a good organisation that can serve and support it. But I think there's a point in that. There's a lot of other things happening in this landscape that means this simplified picture doesn't hold. Which will also affect these things quite a lot. And several of them are also SaaS components in the landscape. And I think it would become very difficult from the client to have to incorporate a lot of those SaaS functions and call on others. I think it's easier to manage them more centrally. So I think there's a point in that. There's an overlap of personalisation when it comes to banners and surfaces — where we have different ideas about how to personalise things with slightly different engines. The whole tech world and everyone who has AI agents is shouting at us that we can do something much better here. So I think this architecture would help there too.

I: If you had to rank performance, maintainability, and security for implementing this kind of solution — where do you put the greatest weight? Fast responses to the customer, security?

P: No, 9 times out of 10 I'd want to answer with maintainability — that's ranked highest. Security second and performance third. If I have to rank them. Then we have some factors that say it can't take more than a certain amount of time, otherwise it's not a valid solution. But absolutely, I'd stand by that ranking. You can argue that security should be at the top, but I think if you have an application that's easy to maintain, that increases security.

I: When you looked at these results — was there anything that stood out? Did you find yourself surprised?

P: No, not really — if we look at the local results, but before you said they were local too, I was surprised that the difference was so small. That's probably the first spontaneous reaction. I was perhaps surprised by how big the difference was between in-app and the BFF on personalisation time — above all on the server-rendering part. There it's a factor of 10. It stood out that the difference was that large. Otherwise the results were pretty much as expected I'd say. Nothing else to that — except that the response times are too long, but that's another matter. They wouldn't pass in the tests, but that's how it is — everything has been running on a single computer.

I: But if you were to choose one of these solutions — and what major overarching modifications you would have made to them. Apart from the fact that there are lots more components in the picture — internal ones, that is.

P: No, but conceptually and logically — if we're now talking conceptual, logical, and physical architecture — then no. This looks well thought out at a reasonably high level. There are loads more components in this, but if we talk security there are lots of parts and lots of infrastructure and such that aren't in here. But it's a logical picture of it. So no, nothing.

I: Is there anything else you'd like to add that we haven't covered?

P: No, just a general comment. Personalisation has been a hot topic for a very long time. We've talked about it for a very long time. We can see there are major changes happening in precisely that area. Understanding how to do personalisation, what we mean by personalisation, which factors drive personalisation — is it content, offering, or appearance? What you can personalise, I think you need to think very hard about. What you want to achieve and above all what effects you want to get from it. We tend to talk very much in technical terms — if we just get this technical part done we'll solve all the problems. We're not good at articulating generally at IT departments: what is the business value we're trying to achieve? What effect on business do we want when we make this type of change? And above all how do we measure and evaluate it? If you make a change like this — how have you thought about measuring that this actually has the effect you want? And what effect did you want? That's something I sadly think we're not good at. Which means that very much of these types of changes we're very subjective about. And then we measure them as banallyy as — how many of these personalised messages did we click on? That's no business effect unless it was precisely clicks we wanted to achieve. But we haven't had that as a goal, and maybe that's not what we want to do. So that's what I think is generally important to really think about: what effect do you want to achieve with these things? That said, that we should personalise — I absolutely think we should. And I think that — even though we've talked about it for 20 years — we haven't achieved very large changes. A great deal of what we present to customers looks the same today as it did about 20 years ago. And there I think what's happening right now will have a very major impact. Above all that we see many of our potential customers want to consume our services in a different way — not everyone grew up with 3270 terminal screens like I did.

I: No. Thank you!

P: Brilliant!

Intervjutranskript

I = Intervjuare **P** = Deltagare

I: Ja, men din roll och din erfarenhet — om du har någon erfarenhet med Angular, CMS och backend?

P: Roll? På Nordea är jag enterprise chief architect, konsult. Har en bakgrund i branschen sedan 40 år tillbaka. Jobbat med applikationsutveckling på i princip alla plattformar, men uppväxt på stordatorer. Var med på Nordea när vi introducerade tre-lagers arkitektur. Var med när vi byggde hela NVF, så liksom Angular, Java — de bitarna, hela den etappen. Har varit chefarkitekt här ett antal gånger. Har jobbat med content marketing de senaste 15–20 åren. Ja, men ganska mycket erfarenhet. Framförallt har jag mycket erfarenhet med att bygga den här typen av applikationer.

I: Om vi börjar med den här in-app-lösningen då. Om du skulle sätta på en skala 1 till 10 hur lätt det skulle vara för en utvecklare att sätta sig in i det vi ska göra — och det gäller att bygga det och att underhålla det.

P: Och då pratar vi helheten?

I: Helheten, vi tänker att vi har en app som inte är personaliserad.

P: Ja, precis. Klientappen inklusive att bygga den funktionen för matchningsdatabasens bitar. Så byggnadsmässigt så tror jag egentligen inte att det är... Vill du ha det på någon skala eller?

I: Ja, 1 till 10. 10 lättast.

P: 10 lättast. Ja, men jag skulle... Ja just det, lättast. Den skulle för mig hamna någonstans på 3 kanske — komplexitet. För det blir hyfsat mycket komplexitet. Det blir ju rätt mycket kod att hålla reda på vad gäller upepe i klientappen. Och den vill man ju helst hålla så ren som möjligt — det är min uppfattning i alla fall. Så lite logik som möjligt egentligen uppe i klientappen. Av den stora orsaken att det är den komponenten som är svårast att ändra. För den måste deployas på lite speciella sätt. Så det är den stora saken. Sedan själva matchningsdatabasen — den komponenten är inte speciellt mycket mer komplicerad än att bygga motsvarande BFF-komponent. Men den stora skillnaden skulle jag vilja säga är just att det blir mycket het up i klientapplikationen. Och underhållsmässigt så... vi vill ju undvika att skjuta ut funktionalitet i klientlagret, för det är det svåraste. Ju längre bak vi kan hålla det i vår infrastrukturstack — som är rent logisk — desto lättare är det att underhålla och förändra. Och vi har ju från det lagret och ned en etablerad infrastruktur för varianshantering av API och sådär. Så det är den största skillnaden egentligen.

I: Men om vi tar samma skala — om vi skulle behöva ändra hur personaliseringen fungerar, eller vi ska lägga till nya segment. Hur lätt skulle du säga — eller svårt?

P: Vi är tillbaka där. Det blir ju knöligare. Det är svårare. Så jag tror att jag skulle vilja lägga kvar här uppe på samma tre i komplexitet. Det beror lite grann på... Jag antar att det spelar ingen roll om det är webb- eller mobilklienten. Vi har ju avsatt ett antal områden för personaliseringsmeddelanden. Jag antar att vi inte renderar dem... Ja, det kanske vi gör i och för sig. Om vi ska vara lite noga så vill vi kunna ändra hela utseendet — rent teoretiskt. Normalt avsätter man strukturella områden: här ska jag kunna personalisera, här ska jag kunna göra vad jag vill, men det här är standardiserat. Men om man tänker att allt är flexibelt, om man nu ska ta personaliseringen hela vägen ut, då blir det väldigt mycket mer att hålla reda på hela tiden. Medan man i en central funktion... nej, det där är svårt att ändra hur som helst skulle jag vilja säga. Men det beror mer på graden av hur mycket man väljer att personalisera. Om det är som vi gör idag med contentbaserat — att vi har banners eller områden där vi ska kunna lägga ut olika typer av meddelanden till medlemmar — det är en mycket enklare sak att hantera. Men säg att du kommer in i mobilappen och det är inte standardöversikten som är där, för jag vill ha något helt annat. Om man ska kunna ändra huvudcontentinnehållet och personalisera det utifrån mina behov — då skulle det bli väldigt mycket mer komplex kod i klientapplikationen än om man renderar det i ett mellanlager som jag ser det.

I: Men är det lite mer... customization? Om du tänker att man vill ha ett dark mode och ska visa någon annan sida eller så.

P: Nej, jag skulle säga så här — sådana saker absolut. Men jag kanske inte vill komma till kontoöversikt, jag kanske vill börja med att jag direkt hamnar i betalningar, för jag gör det nio gånger av tio. Så jag vill att appen ska bete sig helt annorlunda när jag kommer in i den första gången. Om man ska kunna göra hela den biten, då ska det bete sig annorlunda. Och då tror jag att det blir mer saker att tänka på, ju längre ut jag skjuter det.

I: Ja, det tycker jag. Och då förstår jag att den största risken för det tekniska är att det ligger långt ut?

P: Ja, alltså — ju längre ut det ligger, desto... Det är två saker jag skulle säga. Dels är det svårare att testa. Det kan låta lite krångligt kanske, men det blir svårare att testa saker och ting som ligger längre ute i kedjan. För det är lättare att testa centrala komponenter — då kan jag använda massor med olika services. Här kan jag inte testa den sista komponenten, den som är längst ut, överhuvudtaget egentligen. Det är den ena biten. Det är alltid känsligare att deploya i det lagret. Om du tar en mobilapp ska du göra en appuppdatering via App Store, om du kör på Apple. Den processen är mycket krångligare. Att deploya en intern matching-database-komponent eller en BFF-komponent som gör en större del av jobbet — den kan jag deploya tre gånger om dagen om jag vill. För att jag kan versionshantera den på ett bra sätt. Det kan jag inte göra där ute på samma sätt. Likadant om jag vill testa olika typer av beteende, om jag vill A/B-testa — är det här bra eller dåligt? Då är det mycket lättare att göra det i en central komponent. För då kan jag välja i komponenten baserat på någon algoritm. Ska jag koda samma sak ut i klienten, då får jag koda dubbelt ut i klienten. Det är så många saker som blir svårt att göra i sådana förändringsscenarier. Därför är det mycket mer komplicerat.

I: Om vi går in på testbarheten på en skala, 1 till 10.

P: Testbarhet tycker jag inte skiljer så mycket i sig självt — att testa funktionaliteten, för den ska ju vara hyfsat transparent för utveckling. Men det bygger på att det är rätt förhållande just på kanalsidan. För om du säger att du har en kanalkomponent, då kan du testa all din personalisering nere i BFF:en en gång. Det kan du göra med klienten och även där du renderar det centralt. Men har du helt plötsligt 10 klienter som alla ska göra personaliseringskoden, då får du plötsligt 10 klienter som ska uppdateras. Tittar man på den andra sidan — att du renderar det centralt till en komponent istället — då testar du den en gång med en kanal och så vet du att den kommer att bete sig likadant. Det blir mycket lättare att testa och det blir mycket mindre kod att testa framförallt. Det bidrar också till att trycka tillbaka saker i de tekniska lagren gör att det blir färre testfall att göra — en enkel setup med en kanal och komponent med mindre beroenden.

I: Så den får ett ganska bra testvärde eller är det ganska dåligt?

P: Ja, den får ett ganska dåligt testvärde utifrån att det är varje unik klient som måste testa för sig. Eftersom all den personaliseringskoden ligger i klienten — har jag två komponenter måste jag göra samma jobb på två ställen. Vilket gör att den får ett mycket sämre testvärde. Om du ska gradera den igen så kommer den på ett eller två. Dels kan jag inte testa den förrän klienten är färdig, och har jag flera klienter som ska göra samma sak måste alla uppdatera sina komponenter. Sådant ska de jämföra med varandra och så vidare.

I: Om man har appen som den är nu och sen skulle man introducera personalisering — hur tror du att driftkostnaden för det skulle öka, och hur mycket på något sätt, inga fasta pengar.

P: Vad räknar vi med driftkostnaden då? Är det bara underhåll eller utveckling?

I: Det är underhållet och att vi behöver lägga resurser hit. Vi behöver kanske ha fler resurser i ett backend-lager. Behöver vi skala upp Angular-servrarna?

P: Jag skulle inte vilja säga att... Om vi nu är tillbaka på att vi har större personalisering som ligger ute i klienten, då är de centrala komponenterna inte så mycket påverkade av det. Där ökar det inte på något sätt — det är försumbart. Då ska vi gå åt andra hållet på skalan, 7-8: det här är inte mer komplext, det blir inte dyrare för oss att göra de här sakerna — inte på den sidan. Däremot på klientsidan blir det oerhört mycket dyrare beroende på hur många klienter vi har. De slår åt båda hållen. Den andra delen av driftkostnaden är om man går från ett icke-personaliserat till ett personaliserat läge. Eftersom vi trycker ut mycket av den logiken i klienten så är det inte kostnader som drabbar oss — det minskar ju kostnaden för oss. Den får användaren ta ute i sitt device eller i sin enhet.

I: Om vi går över till BFF-arkitekturen — samma frågor egentligen. Hur lätt det skulle vara för en utvecklare att förstå sig på och börja arbeta med den här arkitekturen?

P: Om vi pratar om arkitektur här hos oss så är det väletablerad arkitektur — det är inte båda sätten. Men det ska inte vara några krångligheter. Eftersom vi pratade om multipla klienter i det förra fallet finns det risk att det blir multipla BFF-klienter också, eftersom backend för fronten betyder att det är olika från olika fronten. Sen är det frågan om hur mycket av det som är återanvändbart. Det kanske behöver bli åtminstone en för mobil, och sen är det frågan om det blir multipla — och så kanske någon för webb, eftersom det är lite olika saker som ska renderas. Men huvuddelen av logiken är återanvändbar till stor del i alla fall. Men att förstå den är väl okej. Det finns som sagt risk att man behöver bygga multipla BFF:er — det beror på vad det är, men det här blir mer eller mindre device-specifikt.

I: Men tror du att det blir lättare att förstå det i och med att man har centraliserat all personaliseringslogik?

P: Ja, det tror jag.

I: Man tänker att det kanske hade varit två olika team — ett team som sitter och gör appen och ett team som skulle göra en BFF.

P: Det tror jag att det blir. Jag tror att det blir lättare att förstå för det blir mindre komplexitet mellan teamen på det viset tror jag. Så ur det perspektivet tror jag att det är lättare att förstå arkitekturen. Det är en tydligare uppdelning och det stämmer väldigt mycket med många andra saker som vi gör arkitekturmässigt — att vi flyttar logik så långt tillbaka som möjligt i våra lager. Även om just displaylogik hör hemma i frontendlagret, renderingen för övrigt vill vi ju pusha bakom så långt det går.

I: Om vi sätter det på skalan då?

P: Just det, nu var det där igen då. Sju då.

I: Om man skulle behöva förlänga eller modifiera — stoppa in fler segment eller var det nu var — hur lätt hade det varit då på en skala?

P: Återigen så uppfattar jag det som att det är lättare med den här typen av arkitektur. Det är en central komponent, den är lättare att modifiera, den är lättare att testa för det finns mycket andra sätt. I det förra scenariet måste jag deploya en ny klient för att kunna testa den. Jag kan använda samma klient. Jag kan göra ett annat anrop där ute bara för att se till att jag får en annan version av den där. Det blir mycket mindre komplex ändring att testa. Sedan själva ändringen i sig kanske inte skiljer sig speciellt mycket — det är fortfarande samma personalisering vi vill åstadkomma. Det spelar inte så stor roll vad jag gör någonstans, men själva deployen och testet av den är enklare. Där är det nog tillbaka till sju igen då.

I: Och sen testbarheten som du var lite inne på?

P: Den tror jag är högre — den är säkert åtta — just för att det är lättare. Hela setupen i det här är lättare att testa en central komponent. Jag har lättare versionshantering på ett smidigt sätt, jag behöver inte göra ändringar ute i klienterna på samma sätt. Jag kan göra väldigt

små stegvisa förändringar i BFF-klienter på ett enklare sätt än vad jag kan göra ute i klienten — för den måste bli klar på något sätt. Här kan man göra lite mer stegvis, så jag tror att det är avsevärt lättare att testa den. Har man gjort något centralt fel märker man det redan på det BFF-klienten släpper ifrån sig, medan de övriga komponenterna kan vara i princip oförändrade.

I: Om lite kostnadsökning kring det här då?

P: Här får vi resursförbrukning i vår infrastruktur, så den kommer gå upp i det här fallet. Hur mycket den balanseras mot andra saker, det är svårt att säga. I värsta fallscenariot av det här sitter användare på väldigt gamla klienter, vilket gör att vi behöver ha multipla versioner av våra centrala services körande under väldigt mycket längre tid — för att vi inte har fått klienterna att uppgradera till de senaste sakerna. Här finns risk att centrala kostnader går upp ganska mycket — ökad förbrukning på servrar. Nu kanske det totala antalet anrop inte blir större. Men det finns risk att vi under längre tid måste underhålla flera olika versioner av centrala komponenter. Jättesvårt att säga utan att man mäter det där. Men å andra sidan tror jag att det är lättare att testa och underhålla — så att säga — så jag lägger fem på den.

I: Något som jag har märkt är att CDN-cachen blir mycket större eftersom vi cachar hela sidorna.

P: Det pratade vi om häromdagen. Om vi nu tar våra scenarier — 6 miljoner kunder och alla ska ha helt egen personalisering — då finns det ingen återanvändning av någonting som är cachat överhuvudtaget någonstans, förutom i min egen trafik. I de andra scenarierna när vi gör mer mot klienterna kommer det att vara mycket mer saker som är återanvändbara i cachen. Samtidigt är de flesta sektioner ganska kortlivade idag — den rensas ju ganska ofta ändå. Men absolut, där blir det högre kostnad.

I: Om vi går in på lite säkerhetstänket — vi går tillbaka till in-app-lösningen. 1–10, vad är den spontana känslan av säkerhet? Om man tar fallet att det bara är öppen data till at börja med.

P: Jag skulle vilja säga att säkerhetsmässigt blir det ingen stor skillnad på de här två, som jag ser det. Jag kan inte se att det ska vara någon stor skillnad. Jag skulle vilja jämföra det med fem — det är varken bra eller dåligt. Vi har ju pratat om att vi kontrollerar alla våra services och klienter. Om man tittar framåt nu — agentic AI där det är agenter som styr saker och ting — man kanske inte ens har en klient som en traditionell klient. Det kanske mer är ett interface där man kan prata, eller vad som helst, en kombination av saker och ting. I det fallet tappar vi nästan all kontroll över klientlagret. Om man tar det ytterlighetsscenariot mot att vi har en fullrenderad klient — vi har total kontroll på allt och vi kan sätta in tillräckligt mycket säkerhet i den, så att vi vet att det här är en trovärdig klient som vi har genererat alla de sakerna i, med de säkerhetsmekanismer vi kan bygga in. Det är klart att det finns en ökad osäkerhet ute i klientlagret. Men i de här två fallen är inte det riktigt det vi pratar om — ingen jättestor skillnad.

I: Om man tänker lite på principle of least privilege — vad man delar med sig utåt och inte. Nu kanske det inte skulle vara affärskänsligt att du tillhör det här segmentet. Men det blir ju mer synligt för klienten att det här är personaliserat jämfört med om vi har en BFF-lösning där man döljer allt.

P: Då kanske vi egentligen ska backa tillbaka till föregående scenariot. När vi tar det scenariot i klientdelen — när vi gör renderingen där ute — då kommer vi behöva exponera det datat ut. Datainnehållsmässigt. Det är mycket sämre. Framförallt om det där är ett internet-publicerat API. Vilket jag spontant skulle säga att det där kommer vi inte kunna låta vem som helst anropa. Det måste vara en trovärdig klient som måste anropa den här. För att vi ska veta att risken för exponering ute i klienten är mindre — för att vi faktiskt exponerar en del av personaliseringsdatat. Så ur det perspektivet blir säkerheten viktigare. Det blir ett större problem i klientrenderingsdelen. In-app säkerhetsmässigt — då borde vi sätta ett lägre värde?

I: Ja, precis.

P: Då borde vi sätta tre på den. Eftersom vi exponerar mer. Tack, den hade jag helt missat förstås.

I: Men inget annat kring in-app-lösningen?

P: Nej, inte som jag ser det. Vi måste ju veta att det är vår klient.

I: Om vi går till BFF-lösningen då?

P: Den får vi återigen ett bättre score. Den får vi sex istället. Där exponerar vi inte någonting av det datat som talar om hur vi har personaliserat — utan där är det bara datat som går ut, innehållet. Sen kan man väl, om man är lite klurig, klura ut andra saker. Det blir mindre känslig data som vi skickar ut.

I: Större datamängd?

P: Större datamängd, men data av en mindre känslig natur.

I: Det som du har varit inne på är att vi ska veta att det är en klient som vi litar på?

P: Det blir mindre viktigt ur det perspektivet. Nu handlar det om det här är ett autentiserat eller riktigt autentiserat läge. Det finns lite skillnad. Men om det är öppna sidor har det ingen betydelse — då är det öppen information. Det är en webbläsare, vi har inte kontroll på vad det är. Är det en klient där vi kör i inloggat läge, då är vi fortfarande av den åsikten att vi vill veta vad det är där ute. Det ska inte vara vad som helst. Men vi kan vända på det — samma typ av tänk. Den här BFF:en kanske man vill återanvända i ett open banking-kontext, rent teoretiskt. Det här är ett öppet API för andra payment providers där ni kan hämta information från oss. Vi vill kunna vara transparanta på att en viss del av personaliseringen för kunden medföljer oavsett vilken kanal man använder. Det skiljer lite grann beroende var man befinner sig.

I: Du ser inget problem med cross-site request forgery, cache poisoning och alla de där bitarna?

P: Alla de här åtgärderna är alltid applicerbara. Hela den checklistan måste vi ändå gå igenom. Spontant tycker jag inte att det är värre i den typen av setup — den finns alltid där.

I: Men så länge vi säkrar kopplingen mellan CDN:et och BFF:en?

P: Alla kopplingar in — oavsett om det är till matching-databasen eller BFF:en — måste vi ha dö koll på. Och verkligen säkerställa att det inte är någon som sniffar på vägen. Nu är vi i en bank. Det är just det som banken under åren har jobbat upp en väldigt hög tillitsfaktor kring — att vi inte gör fel. Vi exponerar inte ditt data till någon som inte har behörighet till det. Vi visar inte ditt data av misstag för någon annan. Vi säkerställer att våra tjänster är så säkra att ingen annan ska kunna hacka dem. Vi gör verkligen allt vi kan i det segmentet. Skulle vi kompromissa på den delen så är vi som varumärke rökta. Väldigt mycket av vår energi går åt till att säkerställa att vi inte utsätts för det.

I: Om du skulle behöva välja någon av de här två?

P: Jag väljer ju alltid sådana här BFF-lösningar. Det är för att jag har ritat den där tre-lagriga arkitekturen för en massa år sedan. Två saker. Långsiktigt och underhållsmässigt uppskattar jag enkelheten med att kunna förändra centrala komponenter med större flexibilitet än distribuerade — det är den största orsaken skulle jag vilja säga. De bitar som är just med performance och volymer — det händer så pass mycket inom kommunikationssektorn, även utöver mobila devices. Med 5G, den typen av infrastruktur som har kommit — vi har i princip väldigt lite nätverksproblematik. Svarstiden är så bra i nätet, även för större datamängder. Det är inte många som klagar när man går runt på stan och streamar på Youtube. Även om de bitarna går upp förhållandevis — att vi skjuter in kostnader till oss själva — ja, det kan

man ha åsikter om. Jag skulle nog ändå alltid välja den typen av setup. Just att BFF-services blir mer återanvändbara för många olika typer av klienter — det tror jag är den största fördelen.

I: Om man skulle bygga ett nytt personaliseringsteam — blir ägandeskapet lättare? Om man har en BFF-lösning här, där en del av det också är ute på klienterna på de specifika applikationerna?

P: Vet du vad? Ur det perspektivet tror jag att in-app-lösningen är bättre. För då blir du verkligen ensam ansvarig för saker och ting. Det är enklare att se till. För då måste du ha kompetensen med det, och du måste ha kompetensen med den servicefunktionen som är. Du måste förstå alla de här delarna. Ur teamets effekt är det alltid lättare att ha total end-to-end-ansvar. Du får göra alla delar, för då är du ansvarig för hela delar av kedjan. I den distribuerade setupen när vi gör en BFF, då kommer BFF-teamet behöva prata med de som ska göra grejerna och ni andra som sitter på klienten — och testa att det funkar. Det finns en större risk med att man segmenterar leveransen på det viset. Sedan är det möjligt att man kan lyckas få teamen att fungera. Men traditionella frontend-utvecklingsteam har inte alltid varit jättebra på end-to-end på att vara duktiga på backend-services också. Oftast har det varit ett annat team som bygger centrala services. Det skulle jag säga är en nackdel för den BFF-setupen och en fördel för in-app-lösningen.

I: Om du sätter din rekommendation — hur säker är du på den på en skala?

P: Hur säker jag är på alla mina svar?

I: Nej, på den här rekommendationen då. Nu vet du, du kanske har ändrat lite.

P: Lågt score var bra betyg?

I: Nej, högt score är bra betyg, okej.

P: Så hur säker är du på din rekommendation? Jag är alltid tvärsäker. Så det finns en liten marginal för wiggle room. Jag skulle nog alltid rekommendera den BFF-setupen. För jag tror att det är lättare. Jag tror att det passar oss bättre att bygga det. Det ställer lite högre krav på teamen — man har en bra organisation som ska serva och supporta det. Men jag tror att det finns en poäng i det där. Det händer ju väldigt mycket andra saker i det här landskapet som gör att den här förenklade bilden inte håller. Som också kommer påverka de här sakerna en hel del. Och flera av dem är ju också SaaS-komponenter i landskapet. Och jag tror att det skulle bli väldigt svårt från klienten att behöva lägga in väldigt mycket av de SaaS-funktionerna och anropa andra. Jag tror att det är lättare att hantera dem mer centralt. Så jag tror att det finns en poäng i det. Där finns det ett overlap av personalisering vad gäller banners och ytor — där vi har olika idéer om hur vi ska personalisera saker med lite olika engines. Hela techvärlden och alla som har AI-agenter ropar på oss att vi kan göra något mycket bättre. Så jag tror att det skulle underlätta med den här arkitekturen också.

I: Om du hade rankat performance, maintainability och security för att implementera en sån här lösning — vart lägger du den största vikten? Snabba svar du till kunden, säkerhet?

P: Nej, 9 gånger av 10 skulle jag vilja svara med maintainability — det är högst rankat. Säkerhet tvåa och performance trea. Om jag måste ranka dem. Sen har vi lite faktorer som säger att det inte får ta mer än en viss tid, för då är det inte en giltig lösning. Men absolut, den rankningen håller jag. Man kan argumentera att säkerhet borde vara överst, men jag tror att om man har en applikation som är lätt att underhålla så ökar det säkerheten.

I: När du tittade på de här resultaten — var det något som stack ut? Tyckte du att du var förvånad?

P: Nej, egentligen — om vi tittar på de lokala resultaten, men innan du sa att de var lokala också, så var jag förvånad över att det var så pass liten skillnad. Det är nog den första spontana. Jag kanske var förvånad över att det var så stor skillnad mellan in-app och

BFF:en på personaliseringstiden — framförallt på det som är server-renderings-delen. Där är det faktor 10. Det stack ut att det var så pass stor skillnad. Annars var det nog ganska förväntade resultat skulle jag vilja säga. Inget annat till det här — förutom att svarstiderna är för långa, men det är en annan sak. De skulle inte passera godkänt i testen, men det är så här — allting har gått på en dator.

I: Men om du skulle välja någon av de här lösningarna — och vilka stora övergripande modifieringar du hade gjort på dem? Förutom att det är en massa mer komponenter som är med i bilden — interna, liksom.

P: Nej, men konceptuellt och logiskt — om vi nu pratar konceptuellt, logiskt och fysisk arkitektur — så nej. Det här ser väl genomtänkt ut på en hyfsat hög nivå. Det finns ju en massa mer komponenter i det här, men om vi pratar säkerhet finns det ju en massa delar och massa infrastruktur och sånt som inte är med. Men det är en logisk bild på den. Så nej, inget.

I: Är det något annat som du vill lägga till som vi inte har?

P: Nej, alltså en generell kommentar. Personaliseringen har ju varit ett hot topic väldigt länge. Vi har pratat om det väldigt länge. Vi ser att det är stora förändringar på gång inom just det området. Förståelse för hur man ska göra personalisering, vad man menar med personalisering, vilka faktorer driver personalisering — är det content, erbjudande eller utseende? Vad man kan personalisera tror jag att man behöver fundera väldigt mycket på. Vad man vill åstadkomma och framförallt vad man vill få för effekter av det. Vi tenderar ofta att prata väldigt mycket i tekniska termer — får vi bara göra den här tekniska delen så löser vi alla problem. Vi är dåliga på att artikulera generellt sett på IT-avdelningar: vad är det för affärsnyttan vi ska försöka uppnå? Vad vill vi få för effekt på business när vi gör den här typen av förändringar? Och framförallt hur mäter vi och utvärderar det? Om man gör en sån här förändring — hur har man tänkt att man ska mäta att det här faktiskt får den effekt man vill ha? Och vilken effekt ville man ha? Det tycker jag tyvärr vi är dåliga på. Vilket gör att väldigt mycket av de här typerna av förändringar vi är väldigt subjektiva. Och så mäter vi på dem så banalt som — hur många av de här personaliserade meddelanden har vi klickat på? Det är ju ingen affärseffekt om det inte var just klick vi ville åstadkomma. Men vi har inte haft det som mål, och kanske är det inte det vi vill göra. Så det är väl det som jag tycker är generellt sett viktigt att verkligen fundera på: vad vill man uppnå för effekt med de här sakerna? Sen att vi ska personalisera, det kan jag absolut tycka. Och jag tror att vi — även om vi har pratat om det i 20 år — inte har åstadkommit så stora förändringar. Väldigt mycket av det som vi visar upp för kunderna ser likadant ut idag som vi gjorde för typ 20 år sedan. Och där tror jag att det som händer just nu kommer få väldigt stor påverkan. Framförallt att vi ser att många av våra potentiella kunder vill konsumera våra tjänster på ett annorlunda sätt — det är ju inte alla som är uppvuxna med 3270-terminalskärmar som jag är.

I: Nej. Tack!

P: Kanon!

D Interview transcript – P2

Interview Transcript

I = Interviewer | P = Participant

English Translation

I: Tell me about your background and your role with Angular and CMS.

P: I'm a solutions architect with 25 years of experience in content management. Very little Angular experience, but quite a lot of software development, mainly for the web and web-related work.

I: And your familiarity with BFF and similar patterns — middleware?

P: Familiar in the sense of solution patterns, absolutely. Something sitting in between as a middleware layer.

I: Right. Let's start with the in-app solution. On a scale of 1 to 10, how easy do you think it would be for a new developer — or a developer you might bring on — to start working with this and implement it?

P: This feels straightforward if you think about it from the perspective of a new Angular developer who needs to deal with the in-app matching. I'd probably give it a 7. There's nothing particularly unusual about it. Since it's just a middleware sitting there.

I: Is there any part you think would be more difficult?

P: Not difficult exactly, but it does require you as a developer to know that certain of your calls will get different responses depending on context. You need to keep in mind that you always have a variable context. Things will look different, the composition of the page will look different depending on the context.

I: It demands a greater understanding of how the application works, given that it's personalization-aware.

P: Absolutely, there's an additional layer to manage from a maintenance perspective too.

I: On a scale of 1 to 10, how easy would this solution be to modify or extend across multiple platforms, websites, or similar?

P: I'd probably say a 5, because the more rules and decisions you need to build in, the harder it becomes to scale the application.

I: And if you think about introducing new segments and personalization?

P: It's not difficult to introduce, but you'll constantly get a performance impact on the application the more you add. So it's directly proportional to how many segments you need to manage.

I: Right, but then you might need to take both the application and the matching database into account.

P: How do you mean?

I: If you're bringing in new segments you'll need to change — instead of a banner, say you want to change some other text or a longer field. Then it's required that the matching database tracks which segments are available to fetch. And the application fetches and passes its session ID. But then the application needs to be able to assemble the right output. It needs to iterate through the JSON object somehow. It needs to know what should be personalized. So it ends up in at least two places.

P: Exactly — it just becomes a bit heavier.

I: Where do you think the greatest risk lies as the personalization logic grows? What part of the system gets most affected?

P: Historically it's been the decision layer itself. It's essentially a rule engine. And the more rules you put in, the heavier it gets, the longer it takes to reach a decision. That's where it will start to strain, I think.

I: Because of having many segments?

P: Yes — you have to run everything through a lot of rules for what should be shown when. It's not just the segments; there may also be attributes that determine that a certain time of day should show something for a particular segment. That's what becomes complex to manage.

I: So it's really in the matching database?

P: In the decision layer specifically.

I: And thinking about the CDN layer, how much can you gain there?

P: You can gain quite a lot, given that you can cache specific modules. Specific JSON snippets on the page can be cached individually. So you can save some performance that way. It should be quite easy to scale in theory, especially if you have many segments.

I: Okay, let's move on. On a scale of 1 to 10, the testability of this architecture?

P: It's more testable in the sense that everything is gathered in one place, which is advantageous. Testing personalization is always problematic because you constantly need to supply a context and test against that context. That affects testability, but it does so regardless of the solution. I'd give it a 6. It can be easy to test in isolation in a closed loop, but harder to test in real-world scenarios. It's easy to test because all your dependencies are under the same roof. But if something had been moved out, complexity would be higher. And if you were to test an end-to-end flow, there are intersections. You can handle that in various ways, but it increases complexity regardless.

I: And on the same scale, 1 to 10 — what would you estimate the operational cost of this system to be, as an increased cost compared to without personalization?

P: My experience is that this type of decision-making can be quite CPU-intensive if we want answers as fast as possible. Should I rate the total cost or the cost increase? Is 1 a high cost and 10 a low cost?

I: 10 is a low cost, given what we're doing. 1 is a high cost.

P: I'd say a 5. Because when you run everything as a single unit, you have to scale everything. You don't get individual scaling layers, no segmentation. You can't scale a specific part of the application. Can I assume this runs within the application layer and in memory?

I: Yes, and in a real scenario the decision layer might be at a third-party endpoint. But then you're quite close to the BFF pattern. If the decision or matching layer sits there or at a third-party endpoint

P: Then it becomes more similar to BFF. Then you get that separation, different scalabilities. But that re-introduces complexity for testing — versus just having a database.

I: Then it's a database and a session manager.

I: Let's move on to the BFF. It's essentially the same questions. How easy is it for someone to understand and work with, or maintain?

P: I think it feels easier — I'd put this at around an 8. Because there are much clearer boundaries of responsibility. You can work on things with greater focus and don't need to think about as much ripple effect. So for an Angular application developer, you get a uniform response to work with. You don't need to think too much — you just render what you receive. And the BFF service itself is fairly clean in what it does and should be relatively easy to develop.

I: And how easy would this solution be to modify or extend?

P: I'd put a 7 there too. Because given the clear separation of responsibilities, you can work on things in isolation. It becomes easier because it's isolated. You don't need to release the application just because you want to change something in the BFF layer, for example.

I: And the testability of this system?

P: It's a bit more complex. What did I on the other one?

I: In the previous one you gave a 6.

P: That was low. If we put this one at a 5?

I: Yes.

P: Because it's difficult — there are trade-offs. Now you can test the BFF layer. You can test the application itself with a mocked BFF response, which in one sense makes it more testable. But then it's not a single test.

I: Although you can also test the whole flow?

P: Absolutely, but those are two different things. If you test the BFF layer separately, that's a unit test, yes.

I: Taking the scalability and dependencies you mentioned for the previous solution — is this easier to scale?

P: Yes, the more layered it is, the easier it is to scale generally. And now we have a more layered architecture — if the rule engine is what costs most CPU, we can dedicate our CPU to the rule engine without giving as much to the application. So we can target our resources much more precisely where they're needed for performance.

I: And what about the cost of running this?

P: I'd say similar. Actually I'd put a 7 here because I can be more precise in how I allocate. This should be cheaper.

I: Does that include the CDN layer?

P: No, I don't have visibility into that cost so I can't estimate it well.

I: So excluding CDN — just the cost of running the BFF and the application?

P: Yes.

I: Do you feel the BFF gives better or worse insight into personalization behavior compared to the in-app solution? Is the logic easier or harder to follow?

P: I think it gives better insight because you get a uniform response with everything packaged for you. You see exactly what the output is. It's worse in the sense that from the outside you can't tell exactly what's personalized on a page — you just render what you receive. I think it's clearer to just get the finished package. But you may not have much insight into what's personalized; you'd need to surface that somehow.

I: Yes, that's something I noticed when I implemented them. It's more about printing out all the JSON responses and checking whether personalization has kicked in.

P: You can easily attach a JSON response as a decoration saying it's personalized, and which segment it belongs to.

I: Let's look at the security angle. Does the in-app architecture introduce any security risks — in terms of attack vectors, dependencies, or anything like that?

P: No, we're not really introducing anything new. The main risk is leaking information about which segments we're considered to belong to. But I see no inherent security risk in the personalization itself. It partly depends on what's in the matching database. If it's segments mapped to content then there's nothing secret anywhere.

I: The database needs to be connected or fed with customer data — or rather session data — in some way.

P: But that's not covered by this architecture, so it's hard to say what we expose.

I: One thing is that here we have a more direct connection between the client and the matching database, which may not exist in the other solution.

P: But then again it comes back to what's in the matching database. Is it something we're worried about exposing? As I see it, the only data that's somewhat sensitive here is the segments themselves. And those aren't a business secret. So at that level there's no real risk. If someone were to access the matching database, it's not the end of the world. And if it went down, that's not the end of the world either.

I: So fairly secure then?

P: Well — not secure because it's been made secure, but secure because it doesn't pose a risk. There's low security and low risk, and we can live with that.

I: But we should still say it's relatively low security.

P: There's nothing here that's secured or hardened in any way. I'd say it's relatively low security.

I: It comes down to how data is transmitted. But there's nothing secret in the data.

P: Regardless of whether you encrypt it, it's still HTTPS and the client receives the decrypted data in the end that they read in the browser. There's nothing to hide there.

I: And we're not sending SQL queries or talking directly to a database, so no risk of that either. The application also talks directly to the CMS. Any concern there?

P: No, that's how we've implemented it today. It's an established pattern and we see no risk. We have security on the CMS side against DDoS and cross-site scripting and so on. But this is all about open data — we have a pattern appropriate for the information classification that exists. If we had more confidential data, things would look very different with authentication requirements and so on. Since it's open data, all endpoints are open and publicly accessible. We're not creating any unusual dependencies or exposing anything beyond what already exists.

I: Exactly. And if there had been confidential data on the site, the fundamental architecture would look completely different. Yes. And the risks with caching in the CDN layer — showing the wrong user segment, or just the size of the cache storage — that's perhaps a bigger concern.

P: Not in this specific case, but I feel fairly comfortable with it. We can also ensure some kind of eviction policy that limits the number of cached objects in the CDN. But this should generate relatively few cached objects since it's only caching personalized snippets of your data. So it's a very targeted cache.

I: Exactly — relatively small.

P: And it also means more users within the same segment share the same cached data.

I: Yes, and the principle of least privilege doesn't introduce any additional risks here. Are you familiar with least privilege from a security perspective? That each part has as little information as it possibly can.

P: Yeach, not in this case. This holds here precisely because we have such a low threshold for open information. Not much is needed. It's just content — everything lives in the CMS.

I: But if you imagine it were confidential data — some sensitive personalisation on the client — would this solution then introduce a risk?

P: Yes, you couldn't go from the client directly to the matching database. You'd always need to go via the application to verify authentication and check that you have a valid token. So you'd need to restrict the flows like that. The solution would look different with respect to authentication.

I: But do I understand correctly that there are no real new security risks here?

P: Yes, exactly. We're building on existing, known patterns.

I: Now moving back to the BFF part. Does it introduce any new security risks?

P: This is an logical diagram and what I would like to say now that it's a separate service, we are business-critical. There are requirements for how we set this up. From an operational reliability perspective — not just security as a strict question, but still an important aspect — this needs to be able to run in multiple data centers with multiple copies in each. And they all need to be synchronised. That's an aspect we haven't factored in when looking at this.

I: It becomes a different kind of scaling problem?

P: Not scaling as such, but a foundational matter. It's not how you scale it, but that you need a base to stand on. If we build a table with two legs, we need to be able to remove one leg and have the table stay standing. It's more about that — not from a performance perspective but from an operational reliability perspective. But otherwise from a general security standpoint, the

flow becomes client → CDN → BFF → something, and you need to be a bit careful about what guarantees the traffic from CDN to BFF and ensures it's not malicious or a DDoS. That part isn't covered here.

I: How do you think about that — would you need some form of authentication?

P: Not authentication as such, but some form of web application firewall — a WAF — that at least does some basic scanning of the incoming data or requests. These also detect if you're getting too many requests from the same IP or a certain IP range, classify it as a DDoS, and drop the traffic. So it never reaches the BFF and doesn't need to bear the load. That's more of a hygiene factor for incoming traffic if you're going to expose it directly to CDN.

I: So I understand correctly that this opens a risk?

P: Yes, you're opening a service to the internet and then every endpoint is exposed without authentication, so you need to harden the chain in. Make sure you filter as much as possible before your service. And the service itself should of course be hardened too.

I: And on the CDN side, we have a bit more of a problem with caching — from a security perspective, both in terms of attacks and in terms of correctness, making sure we show what we want to show. Not old content. Not content meant for another user. You could potentially construct or guess a cache key that sneaks in.

P: Yes — a cache polution attack. I don't think that's a major concern with CDN, since they secure traffic between the CDN and origin. CDNs tend to be quite good at ensuring they only ingest data from acceptable sources. And as for stale data — that's not a security risk, it's a business risk. It's up to the business how much of a problem it is to have old information. It's not an IT problem per se.

I: Comparing the principle of least privilege — how does the BFF solution look against the in-app solution?

P: Off the top of my head it's hard to see a major difference. The only thing is opening up the traffic at the BFF level. It sits a bit deeper in the request chain. You kind of want to go via the application server. That serves as a guarantor — it takes the incoming request and routes traffic downward. Service layers are usually more sensitive than applications based on how they're operated. And BFF should also be a service layer.

I: The security risk you see then is really the direct connection between the CDN and the BFF?

P: That can of course be handled and solved, but as it stands in our logical architecture it's not resolved. So we'd need to add that.

I: If you had to choose one of these two. You're presented with both and can then make modifications as needed.

P: Spontaneously it's the BFF. For me it's the cleaner approach on paper — I get the operational and scalability benefits I typically look for.

I: Even accounting for CDN?

P: No — that's something that needs to be resolved. How do we get an optimal cache-friendly solution with a BFF?

I: You'd need to think differently — combine things more, perhaps offer more granular responses from the BFF instead of one whole package.

P: That's an option. But it's something that needs to be thought through.

I: Because you don't need to respond with all context at once — you can push things out piece by piece.

I: How confident are you in that recommendation?

P: I'd say quite clearly — an 8.

I: If you were to rank these three — performance, maintainability, and security — for these solutions and this personalization problem, how would you rank them?

P: As someone from a banking background, I'd say security, then performance, then maintainability. Getting good performance out of a personalization solution is absolutely critical from my perspective.

I: So the ease of working with it, implementing and maintaining it, becomes secondary.

I: Would you change the order if it were a different context — if you had confidential information, or some other type of system?

P: No, I probably wouldn't. I think the customer experience is so critical that performance stays important. So I think it's exactly the same order.

I: Security, performance, maintainability?

I: When you looked at the benchmark results, was there anything that stood out, or was it roughly what you expected?

P: There was a much bigger difference than I expected — on the other results. I remember now: seeing in the browser test results such a large difference between BFF and in-app, in both cases by about one point.

I: Though you'd need more statistical significance for these numbers. Yes, there is a difference. But thinking about how the user experienced the personalization on the page — the CLS value, which represents how much content shifts on the page — is that something you'd place a lot of weight on? Is it an aspect in your choice?

P: I think it's a factor that's downstream of other things. It's relative to the response time of the services and so on.

I: CLS doesn't really matter on its own — what it becomes is content flickering.

P: I think the faster the response time of underlying systems, the less flickering you get. Or in the BFF case where you send everything at once, no flickering at all. But if there's no flickering and it takes half a second to load the page, that's not great either. So for me the underlying response time becomes extremely important. Time to First Byte is perhaps the more important metric over CLS.

I: Right. And in-app is markedly faster there.

P: 10% on one and 1.5% on the other, so not a massive difference. Presentation time was where the really big difference was.

I: Yes, exactly. I think I'm satisfied there.

Swedish Original / Svensk originaltext

Intervjutranskript

I = Intervjuare | **P** = Deltagare

I: Berätta om din bakgrund och din roll med Angular och CMS.

P: Jag är lösningsarkitekt med 25 års erfarenhet av innehållshantering. Väldigt lite Angular-erfarenhet, men ganska mycket mjukvaruutveckling, främst för webben och webbrelaterat arbete.

I: Och din bekantskap med BFF och liknande mönster — middleware?

P: Bekant i den meningen av lösningsmönster, absolut. Någonting som sitter emellan som ett middleware-lager.

I: Precis. Låt oss börja med in-app-lösningen. På en skala från 1 till 10, hur lätt tror du det skulle vara för en ny utvecklare — eller en utvecklare du kanske anställer — att börja arbeta med det här och implementera detta?

P: Det här känns enkelt om man tänker på det ur perspektivet av en ny Angular-utvecklare som ska förhålla sig till in-app-matchningen. Jag skulle nog ge det en 7. Det är inga konstigheter med det egentligen. Eftersom det bara är ett middleware som ligger där.

I: Är det någon del du tror hade varit svårare?

P: Inte svår egentligen, men det kräver att du som utvecklare vet att vissa av dina anrop får olika svar beroende på kontexten. Du måste hålla i åtanke att du alltid har en variabel kontext. Saker kommer att se annorlunda ut, kompositionen på sidan kommer att se annorlunda ut beroende på sammanhanget.

I: Det kräver en större förståelse för hur applikationen fungerar, i och med att den är medveten om personaliseringen.

P: Absolut, det finns ett lager till att förhålla sig till från underhållsperspektiv också.

I: På en skala från 1 till 10, hur lätt hade den här lösningen varit att ändra på eller bygga ut över flera plattformar, webbplatser eller liknande?

P: Jag skulle nog säga en 5, för ju mer regler och beslut du behöver bygga in, desto svårare blir det att skala applikationen.

I: Och om du tänker på att införa nya segment och personalisering?

P: Det är inte svårt att införa, men du får hela tiden en prestandapåverkan på applikationen ju mer du lägger till. Så det står i direkt relation till hur många segment du behöver hantera.

I: Precis, men då kanske du också behöver ta hänsyn till både applikationen och matchningsdatabasen.

P: Hur menar du?

I: Om du ska ta in nya segment behöver du ändra — istället för en banner, säg att du vill ändra någon annan text eller ett längre fält. Då krävs det att matchningsdatabasen håller koll på vilka segment det finns att hämta. Och applikationen hämtar och skickar med sitt sessions-id. Men sen behöver applikationen kunna bygga ihop rätt output. Den behöver loopa igenom JSON-objektet på något sätt. Den behöver veta vad som ska personaliseras. Så det hamnar på minst två ställen.

P: Precis — det blir bara lite tyngre.

I: Var tror du att den största risken ligger när personaliseringslogiken växer? Vilken del av systemet påverkas mest?

P: Historiskt sett har det varit själva beslutsskiktet. Det är egentligen en regelmotor. Och ju mer regler du stoppar in, desto tyngre blir det, desto längre tid tar det att få ett beslut. Det är där det kommer att börja klämma, tror jag.

I: För att man har många segment?

P: Ja — man måste ta allt igenom en massa regler för vad som ska visas när. Det är inte bara segmenten; det kan också finnas attribut som bestämmer att en viss tid på dagen ska visas för ett visst segment. Det är det som blir svårt att hantera.

I: Så det är egentligen i matchningsdatabasen?

P: I beslutsskiktet specifikt.

I: Och om man tänker på CDN-lagret, hur mycket kan man rädda där?

P: Du kan rädda en hel del, med tanke på att du kan cacha specifika moduler. Specifika JSON-bitar på sidan kan cachas individuellt. Så där kan du spara en del prestanda. Det borde vara ganska lätt att skala i teorin, särskilt om du har många segment.

I: Okej, vi går vidare. På en skala från 1 till 10, testbarheten på den här arkitekturen?

P: Den är mer testbar i den meningen att allting är samlat på ett ställe, vilket är fördelaktigt. Det är alltid problematiskt att testa personalisering för att du hela tiden måste tillhandahålla en kontext och testa mot den kontexten. Det påverkar testbarheten, men det gör det oavsett lösning. Jag skulle ge det en 6. Det kan vara lätt att testa i isolation i en stängd loop, men svårare att testa i verkliga scenarier. Det är lätt att testa för att alla dina beroenden är under

samma tak. Men om något hade flyttats ut hade komplexiteten blivit högre. Och om du skulle testa ett end-to-end-flöde finns det skärningspunkter. Du kan hantera det på olika sätt, men det ökar komplexiteten oavsett.

I: Och på samma skala, 1 till 10 — vad skulle du uppskatta driftkostnaden för det här systemet till, som en ökad kostnad jämfört med utan personalisering?

P: Min erfarenhet är att den här typen av beslutsfattande kan vara ganska CPU-intensivt om vi vill ha svar så snabbt som möjligt. Ska jag betygsätta den totala kostnaden eller kostnadsökningen? Är 1 en hög kostnad och 10 en låg kostnad?

I: 10 är en låg kostnad, givet vad vi gör. 1 är en hög kostnad.

P: Jag skulle säga en 5. För när du kör allting som en enhet måste du skala allt. Du får inga individuella skalningslager, ingen segmentering. Du kan inte skala en specifik del av applikationen. Får jag förutsätta att det här körs inom applikationslagret och i minnet?

I: Ja, och i ett verkligt scenario kanske beslutsskiktet finns vid ett tredjepartsendpoint. Men då är du ganska nära BFF-mönstret. Om besluts- eller matchningsskiktet finns där eller vid ett tredjepartsendpoint.

P: Då blir det mer likt BFF. Då får du den separationen, olika skalbarheter. Men det återinför komplexitet för testning — jämfört med att bara ha en databas.

I: Då är det en databas och en sessionshanterare.

I: Vi går vidare till BFF. Det är i princip samma frågor. Hur lätt är det för någon att sätta sig in i och arbeta med, eller underhålla?

P: Jag tror det känns lättare — jag skulle sätta det på ungefär en 8. För det finns mycket tydligare ansvarsgränser. Du kan jobba på saker med större fokus och behöver inte tänka på lika mycket spridningseffekter. Så för en Angular-applikationsutvecklare får du ett enhetligt svar att förhålla dig till. Du behöver inte tänka så mycket — det är bara att rendera det du får. Och själva BFF-tjänsten är ganska renodlad i vad den gör och borde vara relativt lätt att utveckla.

I: Och hur lätt hade den här lösningen varit att modifiera eller bygga ut?

P: Jag skulle sätta en 7 där också. För med tanke på den tydliga ansvarsfördelningen kan du jobba på saker i isolation. Det blir lättare för att det är isolerat. Du behöver inte göra en ny release av applikationen bara för att du vill ändra något i BFF-lagret till exempel.

I: Och testbarheten på det här systemet?

P: Det är lite mer komplext. Vad satte jag på den förra?

I: På den förra gav du en 6.

P: Det var lågt. Om vi sätter den här på en 5?

I: Ja.

P: För det är svårt — det finns avvägningar. Nu kan du testa BFF-lagret. Du kan testa själva applikationen med ett mockat BFF-svar, vilket i en mening gör det mer testbart. Men då är det inte ett enda test.

I: Även om du kan testa hela flödet?

P: Absolut, men det är två olika saker. Om du testar BFF-lagret separat är det ett enhetstest, ja.

I: Med tanke på skalbarheten och beroendena du nämnde för den förra lösningen — är det lättare att skala det här?

P: Ja, ju mer skiktat desto lättare är det att skala generellt. Och nu har vi en mer skiktad arkitektur — om regelmotorn är det som kostar mest CPU kan vi dedikera vår CPU till regelmotorn utan att ge lika mycket till applikationen. Så vi kan rikta våra resurser mycket mer precist dit de behövs för prestanda.

I: Och vad gäller kostnaden att drifta det här?

P: Jag skulle säga liknande. Faktiskt skulle jag sätta en 7 här för att jag kan vara mer noggrann i hur jag allokerar. Det här borde bli billigare.

I: Inkluderar det CDN-lagret?

P: Nej, jag har inte insyn i den kostnaden så jag kan inte estimera den på ett bra sätt.

I: Så exklusive CDN — bara kostnaden att drifta BFFen och applikationen?

P: Ja.

I: Upplever du att BFFen ger bättre eller sämre insyn i personaliseringsbeteendet jämfört med in-app-lösningen? Är logiken lättare eller svårare att följa?

P: Jag tror den ger bättre insyn för att du får ett enhetligt svar med allt paketerat åt dig. Du ser exakt vad outputten är. Det är sämre i den meningen att du utifrån inte kan se exakt vad som är personaliserat på en sida — du renderar bara det du får. Jag tycker att det är tydligare att bara få det färdiga paketet. Men du kanske inte har så mycket insyn i vad som är personaliserat; du skulle behöva lyfta fram det på något sätt.

I: Ja, det var något jag märkte när jag implementerade dem. Det handlar mer om att skriva ut alla JSON-svar och kontrollera om personalisering har skett.

P: Du kan enkelt bifoga ett JSON-svar som en dekoration som säger att det är personaliserat, och vilket segment det tillhör.

I: Låt oss titta på säkerhetsaspekten. Introducerar den här arkitekturen några säkerhetsrisker — i form av attackvektorer, beroenden eller liknande?

P: Nej, vi introducerar egentligen ingenting nytt. Den främsta risken är att läcka information om vilka segment vi anses tillhöra. Men jag ser ingen inneboende säkerhetsrisk i personaliseringen i sig. Det beror delvis på vad som finns i matchningsdatabasen. Om det är segment mappade mot innehåll finns det inget hemligt någonstans.

I: Databasen behöver på något sätt vara kopplad till eller matas med kunddata — eller snarare sessionsdata.

P: Men det täcks inte av den här arkitekturen, så det är svårt att säga vad vi exponerar.

I: En sak är att vi här har en mer direkt koppling mellan klienten och matchningsdatabasen, som kanske inte finns i den andra lösningen.

P: Men då går vi tillbaka till vad som finns i matchningsdatabasen. Är det något vi är oroliga för att exponera? Som jag ser det är den enda datan som är något känslig här segmenten i sig. Och de är ingen affärshemlighet. Så på den nivån finns det ingen egentlig risk. Att någon skulle komma åt matchningsdatabasen är inte slutet på världen. Och om den gick ner är det inte heller slutet på världen.

I: Så ganska säkert då?

P: Jo — men inte säkert för att det har gjorts säkert, utan säkert för att det inte innebär en risk. Det finns låg säkerhet och låg risk, och det kan vi leva med.

I: Men vi bör ändå säga att det är relativt låg säkerhet.

P: Det finns ingenting här som är säkrat eller härdat på något sätt. Jag skulle säga att det är relativt låg säkerhet.

I: Det handlar om hur datan överförs. Men det finns inget hemligt i datan.

P: Oavsett om du krypterar den är det fortfarande HTTPS och klienten får den okrypterade datan till slut som man läser i webbläsaren. Det finns inget att dölja där.

I: Och vi skickar inga SQL-queries eller pratar direkt med en databas, så ingen risk för det heller. Applikationen pratar också direkt med CMS:et. Finns det någon oro där?

P: Nej, det är så vi har implementerat det idag. Det är ett etablerat mönster och vi ser ingen risk. Vi har säkerhet på CMS-sidan mot DDoS och cross-site scripting och så vidare. Men det här handlar helt om öppen data — vi har ett mönster som är lämpat för den informationsklassning som finns. Om vi hade haft mer konfidentiell data hade det sett väldigt annorlunda ut med krav på autentisering och så vidare. Eftersom det är öppen data är alla endpoints öppna och publikt tillgängliga. Vi skapar inga ovanliga beroenden eller exponerar något utöver vad som redan finns.

I: Precis. Och om det hade funnits konfidentiell data på sidan hade den grundläggande arkitekturen sett helt annorlunda ut. Ja. Och riskerna med cachning i CDN-lagret — att visa fel användarsegment, eller bara storleken på cache-lagringen — det är kanske en större oro.

P: Inte i det här specifika fallet, men jag känner mig ganska bekväm med det. Vi kan också säkerställa någon form av eviction-policy som begränsar antalet cachade objekt i CDN:et. Men det här borde generera relativt få cachade objekt eftersom det bara cachar personaliserade snippets av data. Så det är en väldigt riktad cache.

I: Precis — relativt liten.

P: Och det betyder också att fler användare inom samma segment delar samma cachade data.

I: Ja, och principen om minsta privilegium introducerar inga ytterligare risker här. Är du bekant med least privilege ur ett säkerhetsperspektiv? Att varje del har så lite information som möjligt.

P: Ja, inte i det här fallet. Det fungerar här just för att vi har en så låg tröskel för öppen information. Det behövs inte så mycket. Det är bara innehåll — allt finns i CMS:et.

I: Men om du föreställer dig att det vore konfidentiell data — någon känslig personalisering på klienten — skulle den här lösningen då introducera en risk?

P: Ja, du kunde inte gå från klienten direkt till matchningsdatabasen. Du skulle alltid behöva gå via applikationen för att verifiera autentiseringen och kontrollera att du har en giltig token. Så du skulle behöva begränsa flödena på det sättet. Lösningen hade sett annorlunda ut vad gäller autentiseringen.

I: Men förstår jag rätt att det inte finns några egentliga nya säkerhetsrisker här?

P: Ja, precis. Vi bygger på befintliga, kända mönster.

I: Nu går vi tillbaka till BFF-delen. Introducerar den några nya säkerhetsrisker?

P: Det här är ett logiskt diagram och vad jag vill säga nu är att eftersom det är en separat tjänst är vi affärskritiska. Det finns krav på hur vi sätter upp det här. Från ett driftsäkerhetsperspektiv — inte bara säkerhet som en strikt fråga, men ändå en viktig aspekt — det här måste kunna köra i flera datacenter med flera kopior i varje. Och de måste alla vara synkroniserade. Det är en aspekt vi inte har tagit hänsyn till när vi tittar på det här.

I: Det blir ett annat slags skalningsproblem?

P: Inte skalning som sådant, utan en grundläggande fråga. Det är inte hur du skalar det, utan att du behöver en basplatta att stå på. Om vi bygger ett bord med två ben måste vi kunna ta bort ett ben och ha bordet stående. Det handlar mer om det — inte från ett prestandaperspektiv utan från ett driftsäkerhetsperspektiv. Men annars från ett generellt säkerhetsperspektiv blir flödet klient → CDN → BFF → något, och man måste vara lite försiktig med vad som garanterar trafiken från CDN till BFF och säkerställer att den inte är skadlig eller en DDoS. Den biten täcks inte här.

I: Hur tänker du kring det — skulle man behöva någon form av autentisering?

P: Inte autentisering som sådant, men någon form av web application firewall — en WAF — som åtminstone gör en grundläggande scanning av inkommande data eller requests. Dessa detekterar också om du får för många requests från samma IP eller ett visst IP-intervall, klassificerar det som en DDoS och droppar trafiken. Så den aldrig når BFFen och den behöver inte bära belastningen. Det är mer som en hygienfaktor för inkommande trafik om man ska exponera den direkt mot CDN.

I: Så jag förstår rätt att det öppnar en risk?

P: Ja, du öppnar en tjänst mot internet och då är varje endpoint exponerad utan autentisering, så du måste härda kedjan in. Se till att filtrera bort så mycket som möjligt innan din tjänst. Och tjänsten i sig bör förstås också härdas.

I: Och på CDN-sidan har vi lite mer problem med cachning — ur ett säkerhetsperspektiv, både vad gäller attacker och korrekthet, att se till att vi visar det vi vill visa. Inte gammalt innehåll. Inte innehåll avsett för en annan användare. Man skulle potentiellt kunna konstruera eller gissa en cachenyckel som smyger sig in.

P: Ja — en cache poisoning-attack. Jag tror inte det är ett stort problem med CDN, eftersom de säkrar trafiken mellan CDN:et och origin. CDN:er tenderar att vara ganska bra på att säkerställa att de bara tar in data från godtagbara källor. Och vad gäller inaktuell data — det är inte en säkerhetsrisk, det är en affärsrisk. Det är upp till verksamheten hur stort problem det är att ha gammal information. Det är inte ett IT-problem i strikt mening.

I: Om vi jämför principen om minsta privilegium — hur ser BFF-lösningen ut jämfört med in-app-lösningen?

P: Spontant är det svårt att se någon större skillnad. Det enda är just att öppna upp trafiken på BFF-nivå. Den sitter lite djupare i request-kedjan. Man vill egentligen gå via applikationsservern. Den fungerar som en garant — den tar den inkommande requesten och skickar trafiken nedåt. Tjänsteskikt är vanligtvis känsligare än applikationer baserat på hur de driftas. Och BFFen bör också vara ett tjänsteskikt.

I: Den säkerhetsrisk du ser är alltså egentligen den direkta kopplingen mellan CDN:et och BFFen?

P: Det kan förstås hanteras och lösas, men såsom det ser ut i vår logiska arkitektur är det inte löst. Så vi skulle behöva lägga till det.

I: Om du var tvungen att välja en av de här två. Du presenteras för båda och kan sedan göra modifieringar efter behov.

P: Spontant är det BFFen. För mig är det det renare tillvägagångssättet på papper — jag får de drifts- och skalningsfördelar jag brukar leta efter.

I: Även med hänsyn till CDN?

P: Nej — det är något som behöver lösas. Hur får vi en optimal cache-vänlig lösning med en BFF?

I: Man skulle behöva tänka annorlunda — kombinera saker mer, kanske erbjuda mer granulära svar från BFFen istället för ett helt paket.

P: Det är ett alternativ. Men det är något som behöver tänkas igenom.

I: För man behöver inte svara med all kontext på en gång — man kan skicka ut saker och ting bit för bit.

I: Hur säker är du på den rekommendationen?

P: Jag skulle säga ganska tydligt — en 8.

I: Om du skulle ranka de här tre — prestanda, underhållbarhet och säkerhet — för de här lösningarna och det här personaliseringsproblemet, hur skulle du ranka dem?

P: Som någon med bakgrund i banksektorn skulle jag säga säkerhet, sedan prestanda, sedan underhållbarhet. Att få bra prestanda ur en personaliseringslösning är absolut kritiskt från mitt perspektiv.

I: Så lättheten att arbeta med det, implementera och underhålla det, blir sekundärt.

I: Skulle du ändra ordningen om det vore i ett annat sammanhang — om du hade konfidentiell information, eller något annat slags system?

P: Nej, det skulle jag nog inte. Jag tror kundupplevelsen är så kritisk att prestanda förblir viktig. Så jag tror det är exakt samma ordning.

I: Säkerhet, prestanda, underhållbarhet?

I: När du tittade på benchmark-resultaten, var det något som stack ut, eller var det ungefär vad du förväntade dig?

P: Det var en mycket större skillnad än jag förväntade mig — på de andra resultaten. Jag minns det nu: att i webbläsartestresultaten se en så stor skillnad mellan BFF och in-app, i båda fallen med ungefär en poäng.

I: Även om man behöver mer statistisk signifikans för de här siffrorna. Ja, det är en skillnad. Men om man tänker på hur användaren upplevde personaliseringen på sidan — CLS-värdet,

som representerar hur mycket innehåll förflyttar sig på sidan — är det något du lägger stor vikt vid? Är det en aspekt i ditt val?

P: Jag tror det är en faktor som är beroende av andra saker. Det är relativt till svarstiden hos tjänsterna och så vidare.

I: CLS spelar egentligen ingen roll i sig — vad det blir är content flickering.

P: Jag tror att ju snabbare svarstiden är hos underliggande system, desto mindre flickering får du. Eller i BFF-fallet där du skickar allt på en gång, ingen flickering alls. Men om det inte finns någon flickering och det tar en halvsekund att ladda sidan är det inte heller bra. Så för mig blir den underliggande svarstiden extremt viktig. Time to First Byte är kanske det viktigare måttet framför CLS.

I: Precis. Och in-app är markant snabbare där.

P: 10% på den ena och 1,5% på den andra, så ingen jättestor skillnad. Presentation time var det som visade verkligt stor skillnad.

I: Ja, precis. Jag tror jag är nöjd där.

E Interview transcript – P3

Interview Transcript

I *= Interviewer* | **P** *= Participant*

English Translation

I: Could you briefly explain what you work with and your experience with Angular, CMS integration, and service architecture?

P: I have been an IT consultant for about three and a half years, a bit more. That is essentially my full-time career, and I have also worked additionally with full-stack development. The larger part of my consulting career has also been in full-stack development with some side trips into mobile development as well. But much of that time has been tied to Angular development and revolving around that in many ways. Primarily front-end, and some back-end too, but I would say primarily front-end.

I: Are you familiar with the BFF pattern and similar middleware patterns?

P: I would say so, yes.

I: Okay, let's begin then. First we will go into the in-app variant and then we will do it against the BFF. Then we will go a bit into the security track and then we will do a small comparison.

I: On a scale of 1 to 10, how easy would it be for a new developer to understand what to do and start working on the in-app solution? The scale goes 1 is very difficult, 10 is very easy. So the idea is that we have the application and the website. Now we want to introduce personalization via this design pattern.

P: And specifically tied to Nordea's needs?

I: You don't need to specify that. I am thinking it depends a lot on what prior experience the new developer has.

P: Sure, a new developer coming into the project, but one with experience. It is a fairly complex setup, really. It can get very complex at least. But it is perhaps not the absolute most complex thing.

I: 1 to 10, what makes it difficult?

P: 6 or 7, right?

I: It depends on whether you think of 1 as changing the colour of a button, or 1 as very difficult, 10 as very easy. Which way do you take it?

P: 10 is easy, 1 is difficult.

I: 10 is difficult.

P: 3 or 4.

I: Is there any particular part you think would be most challenging? For something like this you might need to work a bit with the CDN. That is external, but it is what you want. And then there is the segmentation as well.

P: Specifically for a public application you really see the problem: what kind of user are you, and what do you get? How should that matching work? Now we are setting aside the matching, the database, and all that?

I: It is part of the whole thing.

P: I would still describe it as complicated, but also... specifically: identifying the user uniquely, and then somehow trying to find patterns for that user and personalize based on that. That is where I feel I have the least understanding of how you would build something like that. But that is also based on my experience — I simply do not have that experience yet.

But I am mainly thinking about serving that content in a good way. How it is presented and when. And with server-side rendering there are advantages and disadvantages. But specifically, server-side needs a server that actually renders HTML. And then somehow making that scalable, because if every request needs to be rendered by a server it creates a lot of pressure on the server. Hence the CDN and so on. How would you segment that in a CDN? If you want resources to be shared between users somehow so that you can serve cached requests.

I: Exactly — in this solution the personalization always happens on the client side. So the server-side rendered part is just the default content. The matching rules still sit in the matching database. That is where we hold your state. So nothing SSR renders that is personalized.

P: Then it is more of a problem for the other solution. But then it is also considerably simpler. You can scale away quite a lot — the application is shared among all other users. You have a CDN serving generic data. There is not nearly the same amount of data that needs to be saved, and segmentation is not the problem. Then what it really comes down to is that the client receives some kind of ID with which it can fetch data from the matching database. I think that part is simpler in this solution.

But it puts more logic on the front-end and the client, which has to handle the personalization and fetch the content and so on. Compared to the BFF method where you could...

I: On the same topic — on a scale of 1 to 10, where 1 is difficult and 10 is very easy, how easy would it be to modify this solution or to add more personalization — more content things to change?

P: There are a lot of things that become coupled then. I am thinking about how Angular works on the server side and the client side — they are in the same application and then it essentially runs twice, or it hydrates an extra time and runs through everything again in the client. There is more logic that has to exist in the client to handle the personalization. When there is a lot of logic in the client, it is affected if you need to make a change to the matching database.

It feels as though there are many advantages to the SSR part of it all. Serving the SSR part feels simpler. So it does not become as hard to make changes. It becomes more coupled — the client has to be updated if you want to make changes to the personalization.

I: If we want to swap things out we need to go through the whole page and know what to look for and what to replace.

P: All of that. The authentication against the matching database. Fetching that data. Re-rendering the new data in case we have already fetched and started showing something that is not personalized. There is a lot of logic that has to exist in the client.

I: If we make a change in the app for personalization we would have to redeploy pretty much the whole app. If we place that somewhere on the scale — it is difficult to make a change here. If we

need to add a new segment, or somehow need to redo or change how personalization works in this way.

P: It obviously depends on the extent of the change. I would still say it involves more parts — both the matching database itself and possibly the CMS. The client primarily needs to rethink the logic for where personalization happens. I would say the same either way. This is something generally difficult.

I: 10 is easy. It will stay that way — 10 is easy, 1 is hard. Higher score the better.

P: I understand. Maybe a 4. If I just have to say something. If you are saying something general, if you want to make a large change — how the matching database works, how you could swap it out entirely, swapping the interface between how the client talks to the matching database, how you could swap out the CMS — I would still say there are many pieces that have to work together. The personalization becomes fairly tightly coupled into the client. If you want to remove the personalization or change it completely, very many parts become involved. 4 then. If you say that a 1 is replacing an old COBOL system at a bank that takes 20 years — absolutely.

I: We move on with some testing. How testable would you say this solution is? 1 is very difficult, 10 is very easy. Give me a rating and then we can discuss.

P: I instinctively think there are still clear roles that the different parts of the system take, which could be tested individually. The CMS can be tested individually. You can test the matching database individually. The client could also be tested individually. I would still say it is testable. A high number is good. Maybe 3? No, 7. It depends on what kind of testing.

I: Can you test the personalization logic in isolation? The logic is split up.

P: The matching database still says what data you should fetch — it holds the rules. Then you can test the rules there. Then the testing for the personalization happens in the client. I also think you could test a lot of that logic there. And also test the whole flow from the client's perspective — fake some kind of cookie and have some form of state you can test with in the matching database in a test environment or something similar. I still think it is quite decent.

I: Do you have any sense or feeling about the operational cost of running this architecture? Not implementing it, but running it.

P: Just a spontaneous feeling — I can't compare between them. CMS is CMS. It will be needed. That is a cost. I understand that CMS has a CDN too. And for a scale like Nordea, which is quite large, I think that is actually quite manageable. Pretty much everything in CMS can be cached. I don't think that cost becomes super expensive.

Since everything happens client-side, the Angular app itself does not need a huge amount of server capacity either. Because then CDN won't route to the Angular server if it is not cached. Everything can happen in the user's browser. So it does not cost as much. You don't need that many Angular servers. It is only the CDN that has to store a lot of data — or maybe that becomes the expensive part. With the matching database, requests will go directly from the client. If you have very many clients that need to personalize content, there will be a lot of requests to the matching database. That is where we will need to scale up with the number of users in a way that the Angular server and CMS with a CDN in front do not. That is most likely where the hosting costs will land. It depends a bit on how the matching is done and how much needs to be computed in real time.

I: But focusing on the matching database — if you think about comparing having the application without personalization versus with personalization, the thing that needs to scale up and becomes the new, more expensive part is the matching database.

P: That is where I see a cost that correlates with number of users and the scale of the whole thing. Then there are other costs involved. Depending on what CMS you have there can be license costs and all sorts of things. It is hard to say that will be the most expensive cost.

I: If you think about us adding personalization, where will the cost increase?

P: I would say that if we add personalization it is the matching database where the cost will land. Given that we have an SSR-rendered Angular app and CMS already in place, and that we are not making any changes there.

I: If you were to estimate it in some way — if you think about personalizing and weighing the benefit against the cost of personalizing?

P: It is a bit of a business question, really.

I: If we say that you have to weigh up where you put the computational effort, where will we need to scale? Hosting and servers? If we say this costs — I'll pull a number out of thin air — 200 million a year for us to be able to personalize all of Nordea's open pages. Is it worth it compared to having personalized and it costs 100 million?

P: My spontaneous feeling is it would cost less than that. But it is a cost calculation. Something you also have to test on certain users — do we get a higher conversion rate? Do we get more customers or earn more money from customers receiving personalized content? That is a trade-off that has to be made statistically. You have to do some form of analysis on it — does it add up that we earn enough extra money from doing the personalization that it is worth it, or are we just counting money, or do we have other goals? We just want to make the experience much better, for customers to feel it is much better. You have to investigate what the effects of the personalization actually are. But personalization in itself is usually worth it. That is my spontaneous feeling.It will be rough and approximate. It is unreasonable to do precisely. But you want a number?

I: An estimate of it. What do you think it will cost? Your feeling. Between 1 and 10 — very high cost at 1 and very low cost at 10.

P: I thought you wanted a number.

I: Not like that. It will cost 20 million — that is incredibly hard to say.

P: 10 is low cost in any case. Then I would say this is maybe... 4, 3, 4. It depends entirely on what you put on the scale. A 10 is buying a coffee. A 1 is replacing all infrastructure. It is hard how that scale goes.

I: We move on to the BFF architecture. Essentially the same questions but looking at that. Scale 1 to 10, how easy do you think a new developer would find it to understand and start working with this architecture? 10 is very easy.

P: I see the advantage in that there is a more logical separation. The client does not need to worry in the same way about content — about personalizing content. You can perhaps in a different way get up to speed... Let's say a new developer needs to get up to speed on the personalization and develop a feature tied to the personalization of the app. In this case it becomes one place. So I only have to focus on all two — if you count the database if it is required. But that is needed. Yes, you need to understand a smaller part of the system. I am thinking if you were to jump into the other solution and do an equivalent thing, you might also need to understand the Angular app and how it is structured. That might cause you to have to change parts of the personalization. Whereas this becomes very non-centralized. Or rather it is... yes, it is non-centralized in the sense that there is one part that only handles the BFF.

I: Yes, the personalization is in one place.

P: Exactly. I would estimate that it is easier. And then we will see — that is my spontaneous feeling.

I: You gave a 3–4 on the in-app thing.

P: Exactly. Say it would be a 6, maybe?

I: And you would say that the separation here is one part of what makes it easier?

P: Absolutely, I would say that. I can tie back to it a bit. So we discuss around the testing. We talked about the fact that here you can test both the matching database and the client. It becomes a bit the same thing. If I focus on personalization, I have to test things on several different systems, whereas here it is just one. The BFF may not be more complex than... and maybe as complex as if you combine the matching database and the client. But it is still separated. And I believe that it is — generally speaking it becomes easier with a solution that is divided. You can change the personalization without redeploying the client.

I: On a scale of 1 to 10, 1 difficult, 10 easy, how easy would this solution be if you need to modify or extend the personalization logic or include more segments?

P: Then I think no work is possibly required in the client to do it. It is primarily the BFF that is affected. And then it depends on whether you segment...

I: Angular is completely unaware of the personalization, really.

P: Yes, exactly. And then perhaps the CDN is also affected if you segment different pages there. So it might require a bit of understanding of the CDN which is more complex.

I: You gave a 4 on the in-app.

P: Was that a question about testability or modifiability?

I: Modifiability.

P: Yes, but then... put a 6 there as well.

I: And do you think the separation between the logic is a reason for that number too?

P: Absolutely. Having a dedicated place. I believe that. And specifically — say you have different teams working on it as well. You have a team that only does personalization and works with the BFF. Or you would need to sync two different teams where one team works on the client, or however it might be. So there are those aspects too. But I would absolutely say that the separation is the main thing.

I: So is there some form of risk of more complexity because it is separated?

P: There might be... if you think about the BFF becoming a new... that it replaces the CMS somewhere as interface. Or that the BFF becomes the new interface toward the CMS. Then you might have to... and you don't want to change anything in the client at all. So maybe you become more limited in what you can do. Because then you have to adapt to the existing client and how it structures data and so on. It will need to work according to its principles. If you build this by adding a BFF and letting the client remain largely untouched, you might need to add a few things to get some kind of cookie or some form of identifier for the user. But that might be a risk of the opportunity to make larger changes. But as long as you have everything at your disposal, it is built on the principle that we will add features for personalization. We only get to touch the BFF. In the other solution there is a clear coupling — there you get to touch both. But if you have the same access to make changes in the client, I see no greater risks compared to the other. It depends on the scenario. But I would say there are perhaps fewer risks in the maintenance part. But perhaps greater risks in what you can build if you see it as a separate part and limit yourself to only making changes there.

Here there may be more risks in maintainability. But greater opportunities if you have a team working on both.

I: Now let's go to testability. How testable would you estimate this architecture is, 1 to 10?

P: That is fairly testable. I would say the testability is quite similar. It may be a slight win in that there is even more separation and responsibility. Now we are adding personalization. Then perhaps we do not need to change the testing of the client itself at all, and it just becomes testing the BFF. I would say the same. But you can still test different parts here as well. It is the same with this — you can test it in a similar way as well. The end-to-end might be harder to test in a different way. It must be more complex. I would put the same as the other. Maybe one point higher. Yes, the same.

I: In the BFF solution it might be slightly easier to test the whole personalization.

P: That is perhaps an advantage if you focus only on testing the personalization. In the first solution you have to test personalization in two places. And then maybe the whole thing as well. Whereas here it might just be that you test it in one place. And then get an overall level with end-to-end testing. Is it a bit better, set an 8 instead.

I: Yes. If we think a bit about the cost — same scale 1 to 10 — what do you feel about that front?

P: I think since everything from the BFF also gets cached and the client only fetches data from the CDN, there is not the same scaling factor required on the BFF per user. But instead the CDN layer might need to be considerably more complex and hold much more data. So the CDN cost might go up significantly here. That would be my guess.

But if I were to estimate the cost..

I: It is a bit misleading. It always goes to the BFF and checks your state in my current implementation. It only gets bypassed through this proxy thing. And it could have done that in the other solution too. But here it is unnecessary, because there is nothing that can be cached. If you need to update your state — if you think you want to send tracking data or something like that.

P: It is hard to get around. You have to process tracking somewhere. And the data used to personalize must be updated to give you the right personalized data. But I think that is a considerably smaller part. Rather than the client fetching data from the matching database, it fetches it from cached CDN instead. Do I understand that correctly? In this case, in the first case, the client fetches... it gets told which data to fetch from the matching database and then it fetches it from the CDN. Whereas in this case it is still the same thing somewhere — it has to get some form of ID. But it does not need to know which data to fetch. So it might still be less logic for the client in the end. And it will still go through the CDN.

I: Every segment change on a page becomes an entire cached page.

P: Exactly.

I: Whereas here we can have one page and just the segments in the in-app solution. So we get quite a lot more memory or storage going here.

P: Yes, exactly. I think that somehow the BFF still has to keep up. If the client makes a side request past the CDN to update information in the BFF, that has to be handled. But spontaneously it still feels like there is less work. If you consider the database having to continuously compute personalization for each user in the first solution — that scales more with the number of visits. The more visitors, the more requests go to the matching database, and it has to compute what each user should have.

I: Yes. It is more of a business question — should it be segment-of-one, and do we want to change your personalization segment in real time?

P: Then there might be a scaling question at the BFF level too, because the client still has to update — or the BFF still has to know how to personalize for this segment. And that will ultimately come via some form of request from the client that bypasses the CDN level to the BFF. So it still has to be able to process new requests that occur.

I: Yes, it needs to govern the rules — orchestrate everything. Build together the JSONobjects.

P: But that also gets cached, I think, that part. And it does in this first solution too. Is it cached or will it be more individual?

I: It gets cached to the CDN that sits between the client there. So going to the matching database and asking — here I come with this identifier, what should I have? You should have that. Then I can send off the request toward the CMS, but it might get caught in the CDN on the way. And it works the same in the BFF. So you ask the BFF, is this what I should have? Yes, it is. Here is everything. But here, if we have 10 pages and we personalize in two places and we have 10 variants on each part we can personalise, then you have 10 times 10 times 2 on one page variants. That many pages need to be cached in the CDN.

P: That can become an insane cost — I think about that too. So cost-wise maybe not... It is hard to compare whether the scaling of the matching database corresponds to how much the BFF has to scale.

I: Given that they still only need to scale similarly, or possibly more for the BFF because it requires more computing power.

P: Yes, that is what I am wondering about. Because if the request usually goes to the CDN from the client, then the client only needs to go for a new ID — do I understand correctly? Because when you make your first request you get an ID. And then tracking and such will also need to be sent to the BFF. So it may be roughly the same. Hard to compare without going into super-high detail.

I: But you put a 4 on the in-app solution. Low was bad. Low is bad, high is cheap. Is your spontaneous feeling that this would be cheaper or more expensive?

P: Hard to say for the CDN — whether the extra logic required for CDN costs much more or whether the storage in CDN costs much more. Hard to estimate. Say... yes, spontaneously a bit higher then perhaps. A 5 then.

I: Cheaper?

P: No, worse. A 3 then. I simultaneously think it is... within the same order of magnitude in terms of costs.

I: We move on then to some security thinking, going back to the in-app solution. You send the session cookie back to the base, see what user segment you should have, send it back to the client — it gets back essentially the endpoints it should send requests to. On a scale of 1 to 10, 1 is low security, 10 is high security — does this solution introduce new security risks? You are thinking about open pages we are working with too.

P: Yes, exactly. What I see is that we are adding some form of data that we store. That might be data we already have, really. We already track users and use that to improve the page and so on. So adding the personalization extends what we do with that information. That might create a few more risks in the compliance area. And there, well — suddenly we might, if you also extend it, have more information we need to store about users. Which could be sensitive, I don't know. Even though these are open pages it still becomes that you pick out a person and give them some unique data adapted for them. It feels like you have to protect that.

But I do not see that as a huge risk or that there will be major security problems with this. What I am thinking about is the client, especially when you have a lot of client-side. And the whole personalization happening client-side means something is exposed outward — all the logic about how our personalization works. The matching database rules are hidden. So it might not matter much that the rest sits in the client. But it becomes more transparent what is happening with the personalization. But I honestly do not want to say any huge, large security problems with the solution in general. It is cookie-based, it is no super-sensitive data as far as I can imagine.

I: And the only thing I use is data protected by GDPR.

P: And we still have a database protecting everything. We are presumably running secure cookies and best practice for storing tokens and so on for clients. So it is hard to see any greater risk — comparable to any other system.

I: So fairly secure then?

P: Fairly secure I would say. But it is not...

I: Are we up at 7–8?

P: I would say 8. Do you want the next one too?

I: We'll get there. Think about this — do you feel there is anything... since the personalization happens in the browser on the client, we need to communicate with... do we increase the attack surface if we think about facing a hacker attack or something? Are we safe against forgery or whatever it might be. Do we make that surface bigger?

P: The number of attack vectors, you mean?

I: Yes, or example, we have opened a point-to-matching-database endpoint facing the internet. Exactly.

P: That is a sensitive point and then the CDN itself — if you compare to the loading today, you have to say. If you do not have the matching, it will still be there. There are only fetches. So what opens up in the environment is that we now send some form of information to the matching database. And when you can send information to something like that there is some form of attack vector there. Depending on what it is. But it does open something up — some form of attack perhaps. Depending on... it is a risk. But then — and it has to be maintained, that you protect the matching database according to best principles. So I see no direct risk of it being sensitive. But there are possibilities to, for example, run a DDoS attack. You have to protect yourself from such things. It is not a problem to have it, but it does require that you think about it and protect yourself from it. And the client exposing a bit more of how the logic works — it is hard to see how that could be exploited for... the request will still come from the CDN and you can imagine it would be protected against overload attacks and so on. And the matching database — yes, it gives information about what you should have for something, but still requires a session. And it is fairly anonymous given that these are open pages. No, it opens up an extra attack vector as we mentioned, but.

I: If we think about the risk around cache-related issues... well, actually being shown the wrong content for the wrong segment — do you see any risk in that? Or is it… we don't care because it's not our responsibility, it will not expose anything personal to anyone else. Everything is public, open.

P: That would be if you were to somehow try to cross this information with being logged in or something, but I don't think that's really in scope. Then there might be a risk where... the sensible thing, if you were to do something like that, is that the matching database gave you the information and if there is no cache there, there is no risk of it leaking to someone else directly. Yes. No, I think that should be fine, and if it is just open data in the CDN then there is no greater danger of it leaking.

I: If we take the Principle of Least Privilege — are you familiar with it?

P: Yes, feel free to refresh it for me.

I: It is essentially just that each component in a system has as little information as it needs. And then how you think about that in this solution — does the client have more access than it actually needs?If you think about it — could a malicious client extract data from the matching database in some way?

P: Given that we always operate from a lesser understanding of this — and given that you still protect that endpoint, that you have your unique identifier and your cookie ensuring you are you — it is hard to see how you could fetch someone else's data. And if everything sits publicly in the CDN — the actual data you then fetch — the matching database just says which data to fetch. Then it sits elsewhere publicly. So of course someone else could fetch data not meant for them. But that does not begin to mean it is a bad thing because it is still open data. And yes... The client ultimately receives the information about what to fetch, perhaps in a slightly different way than in the other solution. And that data has to end up in the client in the end.

I: Yes, it essentially gets exposed endpoints that are published. One of the endpoints that already sit publicly on the net like this.

P: So no greater risk. But the client does have a slightly larger area of responsibility than the other solution will have.

I: A bit more?

P: A bit more if you think about how much responsibility it has. But actually it does not have more access than the other solution does to anything.

I: Are there any other security things with this, or anything around security that you would worry about? Only related to this first one either way.

P: No, not that I can think of off the top of my head. I mean, given that it is open data and...The spontaneous feeling. I could not compare between them. Alright, then we have cookies and things that are secure and the whole part of that. It gives the feeling of adhering to best practice somewhere for that type of data. So I feel no great... I feel no greater risk.

I: Onwards, onwards. We move on to the BFF then. Scale 1 to 10, how do you perceive the security?

P: I would say about the same as the previous one.

I: An 8?

P: It is essentially the same information that is needed from the client — it needs some form of identifier, cookie or similar.

I: So there is some risk for both but the matching database and BFF are exposed — not fully optimized against the internet.

P: But you get that with the BFF too. Yes. In that... and both become like that. Yes, exactly. That is why I think they are fairly equivalent in terms of security. So maybe there are some ways like that... yes, maybe. Depending on how it is structured it might open up for different types of attacks or there are certainly other ways to attack this compared to the other. But I would still say there is design-by-obscurity in that the BFF becomes a bit more like that — it is a bit more of a black box what happens there or how the personalization works. Here you get endpoints that answer. Here is what you should fetch. That makes it clearer for someone who would want to attack — this is how our system works, this is how our personalization works. We can see we send a cookie here, we get back what we should fetch. So we know that this service we send this data to has that responsibility.

And in this way here we also send some data and we get back a cookie I assume. But we do not know what happens inside beyond that. We know that the next time we fetch something from our CDN it will be different based on what we got back. So here it becomes a bit more obscure what actually happens, and thereby perhaps a little more protected. But since there still has to be a small side channel here to update your cookie or get new information about what to fetch, there is still the same vector there — overload attack or finding some loophole to bring down the matching database or BFF. But I would still rate them fairly similar or put them the same.

I: If we take the cache-related risks.

P: I think in that case the client is responsible for building up the page and there is really only one page we get from CDN that is finished, server-side rendered. As long as that is okay it is not affected by any personalization, I think. So there is no greater risk there. And that might open up perhaps some more CDN-related attack here — someone could... I am no cyber-security expert but I think it becomes more — since the CDN becomes more advanced, there are also more things that can go wrong in this solution perhaps, when you have a lot of data that has to be stored in the CDN and segmented for different users and so on. Perhaps there could be a cache poisoning vector. I cannot think of how that would work in practice. But maybe that is where you need to think a bit more about how it should work. That there should be more security thinking around the CDN in this solution than in that one. Or some form of segmentation there. Otherwise I would still rate them fairly similar actually, both of them.

I: If we take the Principle of Least Privilege in relation to the BFF — how does it feel around that?

P: Around all solutions, you mean?

I: Yes.

P: Then as we talked about there, the clients have less knowledge about how the personalization works. It becomes a bit more of a black box what it gets back from the BFF, and it is only responsible for rendering it. But without knowledge of what is personalized and what is not, there is something in that — and that the BFF, that all logic for how things are personalized lands there, whereas here it kind of seeps out a bit to the client too, giving a bit more information to someone who might want to attack, which is open really. So it is a bit better in this one then. That is good, but I would also say again — what does that information do? It is really not sensitive. It is not personal in any way and it is completely open. It does not mean someone would get hold of it. But it is still there. It has less responsibility for it. And nothing more for it to know about.

I: Is there anything else around security that you want to raise?

P: Not really. I feel it. No.

I: We move on to the comparison part then, a bit more. If you had to choose one of these, which would you recommend for developing and implementing on Nordea's open pages? It can be one of them or "it depends".

P: I do prefer the BFF pattern perhaps. Because in the client you do not need to do as much. You can have that logic completely tied to its own system and handle it — yes. It does not become so intertwined everywhere and does not require as much integration. And I am also thinking about what it looks like in how we work now — the matching database and the whole part of what we present to our users would likely have landed on a different team than those who work on the client. And then I think roughly that the logical part should be...

I: Would you say the BFF then?

P: Yes, but it would be more spontaneous. Maybe I am overthinking.

I: No, no, that is only good. On a scale of 1 to 10, how confident are you in that recommendation? 1 being uncertain, 10 very certain.

P: I would say a 7 maybe, 8. I feel strongly about it, but as we discussed there are advantages and disadvantages with both. And depending on what you want to get out of it in the end — if you want segment-of-one, then maybe it is easiest to personalize in the client. Whereas if you can have somewhat broader and larger segments across many users, if they get the same content, I think there are great advantages to the BFF approach.

I: If you were to rank the three dimensions — performance, maintainability and security — in order of importance for Nordea and for you in this. How would you rank them? What comes first? You have performance, security, maintainability.

P: I would still say performance is probably — well, important overall. Security is of course important as hell. But I think in this loading context I do not think security is what you should hang up on because I think it will turn out fairly similar. So I would put security last. Maintainability is of course important. But performance somehow becomes the important one for creating a good user experience out of the three for end users — and I think that is still the most important thing. Performance, maintainability, security last.

I: When you look at these numbers, was there anything you reacted to, something you did not expect? Something that made you surprised — something was faster or better than you thought.

P: Something that was interesting is this personalization time — because it counts when the personalization happens during this difference between these. And here it is already personalized in the BFF.

I: Yes, that is correct. So it becomes essentially the total time.

P: That is a nice thing I think — you can get something personalized directly. I think that is expected though.

I: Here we have this going directly against Angular and here on CDN — the difference for in-app is 491 versus 49.

P: Exactly. Just that CDN might be most interesting to compare with really, feeling what it would be like in reality with CDN in front, and which it would be. And that is an insanely large difference. The Mycannat feels quite similar — these were just load tests then. Then it is just server-side load without starting to render any JavaScript or anything. They feel fairly similar.

I: Response times really.

P: They feel quite similar. It does not feel like there is any major difference you could explain to anyone — spontaneous feeling at least.

I: No, the biggest difference is around 11%. That one we ran directly against Angular for total requests.

P: Exactly. Yes. If you think that it is a bit slow — I can imagine that, okay. It is how many requests you can manage. Okay. And in-app is better.

I: Yes, before hitting a threshold it would be. We should keep in mind that everything was done on one computer. Always.

P: Okay. Nothing specific I react to that feels unexpected. Exactly. Well, that figure is the one I find most interesting. Personalization time.

I: If you could modify either of them — is there anything specific where you feel the solutions are lacking? You can combine them in some way or do something completely different. Move logic somewhere else.

P: I am spontaneously thinking of this. If you see that you have a tracking service, for example — like many do — then that is actually a quite good place to get data from for doing personalization later. So I think that if you were to make some new spin on this, that you would have something small beside the BFF — some small analytics service that the client talks to and the BFF, or the matching database, waits for information from. That it is divided in that way. But otherwise, maybe not. As I think about it.

I: Thank you for your participation.

P: Thank you.

Swedish Original / Svensk originaltext

Intervjutranskript

I = Intervjuare | **P** = Deltagare

I: Kan du kort förklara vad du jobbar med och din erfarenhet av Angular, CMS-integration och servicearchitektur?

P: Jag har varit IT-konsult i ungefär tre och ett halvt år, lite mer. Det är i princip min heltidskarriär, och jag har dessutom jobbat med fullstackutveckling. Den större delen av min konsultkarriär har också handlat om fullstackutveckling med lite sidospår in i mobilutveckling. Men stor del av den tiden har varit kopplad till Angular-utveckling och kretsat kring det på många sätt. Framförallt front-end, och lite back-end också, men jag skulle säga framförallt front-end.

I: Är du bekant med BFF-mönstret och liknande middleware-mönster?

P: Det skulle jag säga, ja.

I: Okej, då börjar vi. Först går vi in på in-app-varianten och sedan gör vi det mot BFF:en. Sedan går vi lite på säkerhetsspåret och sedan kör vi en liten jämförelse.

I: På en skala 1 till 10, hur lätt skulle det vara för en ny utvecklare att förstå vad man ska göra och börja arbeta med in-app-lösningen? Skalan är 1 väldigt svårt, 10 väldigt lätt. Tanken är att vi har applikationen och hemsidan. Nu vill vi introducera personalisering via det här designmönstret.

P: Och specifikt kopplat till Nordeas behov?

I: Du behöver inte specificera det. Jag tänker att det beror ganska mycket på vilken förkunskap den nya utvecklaren har.

P: Visst, en ny utvecklare som kommer in i projektet men som har erfarenhet. Det är en rätt komplex snurra egentligen. Den kan bli väldigt komplex i alla fall. Men det är kanske inte det absolut mest komplexa.

I: 1 till 10 – vad gör det svårt?

P: 6 eller 7, va?

I: Det beror på om man tänker att 1 är att ändra färg på en knapp, eller 1 är väldigt svårt och 10 väldigt lätt. Vilken väg tar du?

P: 10 är lätt, 1 är svårt.

I: 10 är svårt.

P: 3 eller 4.

I: Finns det någon speciell del som du tänker skulle vara mest utmanande? Till något sådant här kan man behöva jobba lite med CDN:et. Det är externt, men det är vad man vill ha. Och sedan finns det segmenteringen också.

P: Specifikt för en publik applikation ser man verkligen problemet: vilken typ av användare är du och vad får du? Hur ska den matchningen gå till? Nu sätter vi matchningen, databasen och allt det åt sidan?

I: Det är en del av hela grejen.

P: Jag skulle ändå beskriva det som komplicerat, men också... specifikt: att identifiera användaren unikt, och sen på något sätt försöka hitta mönster för den användaren och personalisera utifrån det. Det är där jag känner att jag har minst koll på hur man skulle bygga något sådant. Men det är också baserat på min erfarenhet — jag har helt enkelt inte den erfarenheten ännu.

Men jag tänker framförallt på att tillhandahålla det contentet på ett bra sätt. Hur det presenteras och när. Och med server-side rendering finns det fördelar och nackdelar. Men specifikt behöver server-side en server som faktiskt renderar HTML. Och sen på något sätt göra det skalbart, för om varje request måste renderas av en server skapar det väldigt mycket tryck mot servern. Därav CDN och så vidare. Hur segmenterar man det i ett CDN? Om man vill att resurser ska delas mellan användare på något sätt så att man kan serva cachade requests.

I: Precis — i den här lösningen sker personaliseringen alltid på klientsidan. Så server-side-renderat är bara default-contentet. Matchningsreglerna sitter fortfarande i matchningsdatabasen. Det är där vi håller ditt state. Så ingenting SSR-renderar är personaliserat.

P: Då är det mer ett problem för den andra lösningen. Men då är det ändå betydligt enklare. Man kan skala bort ganska mycket — applikationen delas bland alla andra användare. Man har ett CDN som serverar generisk data. Det är inte alls samma mängd data som behöver sparas, och segmentering är inte problemet. Då handlar det egentligen om att klienten får ett slags ID med vilket den kan hämta data från matchningsdatabasen. Den biten tycker jag är enklare i den här lösningen.

Men det lägger mer logik på front-end och klienten, som måste hantera personaliseringen och hämta contentet och så vidare. Jämfört med BFF-metoden där man skulle kunna...

I: På samma ämne — på en skala 1 till 10, där 1 är svårt och 10 är väldigt lätt, hur lätt skulle det vara att modifiera den här lösningen eller lägga till mer personalisering — fler content-saker att ändra på?

P: Det finns mycket saker som kopplas ihop då. Jag tänker på hur Angular fungerar på serversidan och klientsidan — de är i samma applikation och sen körs det egentligen två gånger, eller det hydratiseras en extra gång och kör igenom allt på nytt i klienten. Det blir mer logik som måste finnas i klienten för att hantera personaliseringen. När det finns mycket logik i klienten påverkas den om man behöver göra en ändring i matchningsdatabasen.

Det känns som att det finns många fördelar med SSR-delen av alltihop. Att serva SSR-delen känns enklare. Så det blir inte lika svårt att göra ändringar. Det blir mer kopplat — klienten måste uppdateras om man vill göra förändringar till personaliseringen.

I: Om vi vill byta ut saker behöver vi gå igenom hela sidan och veta vad vi ska leta efter och vad vi ska ersätta.

P: Allt det. Autentiseringen mot matchningsdatabasen. Att hämta den datan. Rendera ny data igen ifall vi redan har hämtat och börjat visa något som inte är personaliserat. Det är väldigt mycket logik som måste finnas i klienten.

I: Om vi gör en ändring i appen för personalisering skulle vi behöva deploya om i princip hela appen. Om vi placerar det någonstans på skalan — det är svårt att göra en förändring här. Om vi behöver lägga till ett nytt segment, eller på något sätt behöver göra om eller ändra hur personaliseringen fungerar på det här sättet.

P: Det beror självklart på förändringens omfattning. Jag skulle ändå säga att det involverar fler delar — både matchningsdatabasen i sig och eventuellt CMS:et. Klienten behöver framförallt tänka om logiken för var personalisering sker. Jag skulle säga samma oavsett. Det här är något som generellt är svårt.

I: 10 är lätt. Det stannar så — 10 är lätt, 1 är svårt. Högre poäng desto bättre.

P: Jag förstår. Kanske en 4:a. Om jag bara ska säga något. Om man säger något generellt, om man vill göra en stor förändring — hur matchningsdatabasen fungerar, hur man kan byta ut den helt och hållet, byta interface mellan hur klienten pratar med matchningsdatabasen, hur man kan byta ut CMS:et — jag skulle ändå säga att det finns många bitar som måste samspela. Personaliseringen kopplas ganska hårt in i klienten. Om man vill ta bort personaliseringen eller ändra den helt och hållet involveras väldigt många delar. 4 då. Om man säger att en 1:a är att byta ut ett gammalt COBOL-system på en bank som tar 20 år — absolut.

I: Vi går vidare med lite testning. Hur testbar skulle du säga att den här lösningen är? 1 är väldigt svårt, 10 är väldigt lätt. Ge mig ett betyg och sen kan vi diskutera.

P: Jag tänker spontant att det ändå finns tydliga roller som de olika delarna av systemet tar, som skulle kunna testas individuellt. CMS:et kan man testa individuellt. Man kan testa matchningsdatabasen individuellt. Klienten skulle man också kunna testa individuellt. Jag skulle ändå säga att det är testbart. Högt nummer är bra. Kanske 3? Nej, 7. Det beror på vad det är för testning.

I: Kan man testa personaliseringslogiken isolerat? Logiken är uppdelad.

P: Matchningsdatabasen säger ändå vilken data man ska hämta — den har reglerna. Då kan man testa reglerna där. Sedan sker testningen av personaliseringen i klienten. Jag tänker också att man skulle kunna testa mycket av den logiken där. Och även testa hela flödet från klientens perspektiv — fejka en slags cookie och ha någon form av state man kan testa med i matchningsdatabasen i en testmiljö eller något liknande. Jag tänker ändå att det är ganska hyfsad.

I: Har du någon känsla för driftkostnaden för att köra den här arkitekturen? Inte att implementera den, utan att driva den.

P: Spontana känslan — jag kan inte jämföra dem emellan. CMS är CMS. Det kommer behövas. Det är en kostnad. Jag förstår att CMS har ett CDN också. Och för en skala som Nordea, som är rätt stor, tänker jag ändå att det är ganska hanterbart. I princip allt i CMS kan bli cachat. Jag tror inte den kostnaden blir superdyr.

Eftersom allt sker client-side behöver Angular-appen i sig inte heller ha en jättestor server-kapacitet. För då skickar CDN:et inte vidare till Angular-servern om det inte är cachat. Allt kan ske i användarens browser. Så det kostar inte lika mycket. Man behöver inte så många Angular-servrar. Det är bara CDN:et som måste lagra mycket data — eller kanske det blir den dyra posten. Med matchningsdatabasen kommer requests att gå direkt från klienten. Om man har väldigt många klienter som ska personalisera content, kommer det bli mycket requests till matchningsdatabasen. Det är där vi behöver skala upp med antalet användare på ett sätt som Angular-servern och CMS

med ett CDN framför sig inte behöver. Det är troligtvis där hostingkostnaderna kommer ligga. Det beror lite på hur matchningen görs och hur mycket som behöver beräknas i realtid.

I: Men med fokus på matchningsdatabasen — om man tänker på att jämföra att ha applikationen utan personalisering kontra med personalisering, är det som behöver skala upp och blir den nya, dyrare delen matchningsdatabasen.

P: Det är där jag ser en kostnad som korrelerar med antalet användare och skalans storlek.

Sedanfinns det andra kostnader inblandade. Beroende på vilket CMS man har kan det finnas licenskostnader och allt möjligt. Det är svårt att säga att det kommer bli den dyraste kostnaden.

I: Om man tänker på att vi lägger till personalisering, var kommer kostnaden att öka?

P: Jag skulle säga att om vi lägger till personalisering är det matchningsdatabasen som kostnaden hamnar på. Givet att vi har en SSR-renderad Angular-app och CMS redan på plats, och att vi inte gör några ändringar där.

I: Om du skulle uppskatta det på något sätt — om man tänker på att personalisera och väger nyttan mot kostnaden att personalisera?

P: Det är lite av en affärsfråga egentligen.

I: Om vi säger att man måste avväga var man lägger processkraften, var behöver vi skala? Hosting och servrar? Om vi säger att det kostar — jag drar ett tal ur luften — 200 miljoner om året för att vi ska kunna personalisera alla Nordeas öppna sidor. Är det värt det jämfört med att ha personaliserat och det kostar 100 miljoner?

P: Min spontana känsla är att det skulle kosta mindre än så. Men det är en kostnadskalkyl. Något man också måste testa på vissa användare — får vi en högre konverteringsgrad? Får vi fler kunder eller tjänar vi mer pengar på att kunder får personaliserat content? Det är en avvägning som måste göras statistiskt. Man måste göra någon form av analys på det — går det ihop att vi tjänar tillräckligt mycket mer pengar på att göra personaliseringen att det är värt det, eller räknar vi bara pengar, eller har vi andra mål? Vi vill bara göra upplevelsen mycket bättre, att kunderna känner att det är mycket bättre. Man måste undersöka vad effekterna av personaliseringen faktiskt är. Men personalisering i sig är vanligtvis värt det. Det är min spontana känsla. Det blir ungefärligt och approximativt. Det är orimligt att göra exakt. Men du vill ha ett tal?

I: En uppskattning av det. Vad tror du det kostar? Din känsla. Mellan 1 och 10 — väldigt hög kostnad vid 1 och väldigt låg kostnad vid 10.

P: Jag trodde du ville ha ett tal.

I: Inte såhär. Det kostar 20 miljoner — det är otroligt svårt att säga.

P: 10 är låg kostnad i alla fall. Då skulle jag säga att det här kanske är... 4, 3, 4. Det beror helt på vad man sätter på skalan. En 10:a är att köpa ett kaffe. En 1:a är att byta ut all infrastruktur. Det är svårt hur den skalan går.

I: Vi går vidare till BFF-arkitekturen. I princip samma frågor fast vi tittar på den. Skala 1 till 10, hur lätt tror du att en ny utvecklare skulle förstå och börja arbeta med den här arkitekturen? 10 är väldigt lätt.

P: Jag ser fördelen med att det finns en mer logisk uppdelning. Klienten behöver inte bry sig på samma sätt om content — om att personalisera content. Man kan kanske på ett annat sätt sätta sig in i... Säg att en ny utvecklare behöver sätta sig in i personaliseringen och utveckla en feature kopplad till personaliseringen av appen. I det här fallet blir det ett ställe. Så jag behöver bara fokusera på allt det — om man räknar med databasen om den krävs. Men den behövs. Ja, man

behöver förstå en mindre del av systemet. Jag tänker om man skulle hoppa in i den andra lösningen och göra något motsvarande, kanske man också behöver förstå Angular-appen och hur den är uppstrukturerad. Det kan innebära att man måste ändra delar av personaliseringen. Medan det här blir väldigt icke-centraliserat. Eller snarare... ja, det är icke-centraliserat på det sättet att det finns en del som bara hanterar BFF:en.

I: Ja, personaliseringen är på ett ställe.

P: Exakt. Jag skulle uppskatta att den är lättare. Och sen får vi se — det är min spontana känsla.

I: Du satte 3–4 på in-app-grejen.

P: Precis. Säg att det skulle vara en 6:a kanske?

I: Och skulle du säga att separationen här är en del av vad som gör det lättare?

P: Absolut, det skulle jag säga. Jag kan återkoppla lite till det. Så vi diskuterar kring testningen. Vi pratade om att här kan man testa både matchningsdatabasen och klienten. Det blir lite samma sak. Om jag fokuserar på personalisering måste jag testa saker på flera olika system, medan det här bara är ett. BFF:en är kanske inte mer komplex än... och kanske lika komplex som om man slår ihop matchningsdatabasen och klienten. Men det är fortfarande separerat. Och jag tror att det är — generellt sett blir det lättare med en lösning som är uppdelad. Man kan ändra personaliseringen utan att deploya om klienten.

I: På en skala 1 till 10, 1 svårt, 10 lätt, hur lätt skulle den här lösningen vara om man behöver modifiera eller utöka personaliseringslogiken eller inkludera fler segment?

P: Då tänker jag att det kanske inte krävs något arbete i klienten för att göra det. Det är framförallt BFF:en som påverkas. Och sen beror det på om man segmenterar...

I: Angular är helt omedveten om personaliseringen egentligen.

P: Ja, precis. Och sedan kanske CDN:et också påverkas om man segmenterar olika sidor där. Så det kanske krävs lite förståelse för CDN:et som är mer komplext.

I: Du satte en 4:a på in-app.

P: Var det en fråga om testbarhet eller modifierbarhet?

I: Modifierbarhet.

P: Ja, men då... Sätt en 6:a där också.

I: Och tänker du att separationen mellan logiken är en anledning till det talet också?

P: Absolut. Att ha ett dedikerat ställe. Det tror jag. Och specifikt — säg att man har olika team som jobbar med det också. Man har ett team som bara gör personalisering och jobbar med BFF:en. Eller man behöver synka två olika team där ett team jobbar med klienten, eller hur det nu kan vara. Så det finns sådana aspekter i det också. Men jag skulle absolut säga att separationen är det viktigaste.

I: Så finns det någon form av risk för mer komplexitet för att den är separerad?

P: Det kanske finns... om man tänker att BFF:en skulle bli ett nytt... att den ersätter CMS:et som interface någonstans. Eller att BFF:en blir det nya interface mot CMS:et. Då kanske man måste... och man vill inte ändra något i klienten överhuvudtaget. Så kanske man blir mer begränsad i vad man kan göra. För då måste man anpassa sig efter den befintliga klienten och hur den strukturerar data och så vidare. Det måste fungera enligt dess principer. Om man bygger det här genom att lägga till en BFF och låta klienten vara i stort sett orörd, kan man behöva lägga till lite saker för att få någon slags cookie eller någon form av identifierare för användaren. Men det kanske är en risk med

möjligheten att göra större förändringar. Men så länge man har allt till sitt förfogande, bygger det på principen att vi ska lägga till features för personalisering. Vi får bara röra BFF:en. I den andra lösningen finns en tydlig koppling — där får man röra båda. Men har man samma tillgång till att göra förändringar i klienten ser jag inga större risker jämfört med den andra. Det beror på scenariot. Men jag skulle säga att det kanske finns färre risker i förvaltningsdelen. Men kanske större risker i vad man kan bygga om man ser det som en separat del och begränsar sig till att bara göra förändringar där.

Här kanske finns mer risker i förvaltningsbarheten. Men större möjligheter om man har ett team som jobbar med båda.

I: Nu går vi till testbarheten. Hur testbar skulle du uppskatta att den här arkitekturen är, 1 till 10?

P: Det är ganska testbart. Jag skulle säga att testbarheten är ganska likartad. Det kanske är en liten vinst i och med att det är ännu mer separation och ansvar. Nu lägger vi till personalisering. Då kanske vi inte behöver förändra testningen av klienten i sig överhuvudtaget, och det bara blir att testa BFF:en. Jag skulle säga samma. Men man kan fortfarande testa olika delar här också. Det är likadant med det här — man kan även testa det på liknande sätt. End-to-end kanske är svårare att testa på ett annat sätt. Det måste vara mer komplext. Jag skulle sätta samma som den andra. Kanske ett poäng högre. Ja, samma.

I: I BFF-lösningen kanske det är lite lättare att testa hela personaliseringen.

P: Det är kanske en fördel om man bara fokuserar på att testa personaliseringen. I den första lösningen måste man testa personalisering på två ställen. Och sedan kanske hela grejen också. Medan det här kanske bara är att man testar det på ett ställe. Och sedan får en helhetsnivå med end-to-end-testning. Är det lite bättre, sätt en 8:a istället.

I: Ja. Om vi tänker lite på kostnadsgrejen — samma skala 1 till 10 — vad känner du på den fronten?

P: Jag tänker att i och med att allt från BFF:en också blir cachat och klienten bara hämtar data från CDN:et, finns det inte samma skalningsfaktor som krävs på BFF:en per användare. Men istället kanske CDN-lagret behöver vara betydligt mer komplext och ha mycket mer data. Så CDN-kostnaden kanske ökar betydligt här. Det skulle vara min gissning.

Men om jag ska uppskatta kostnaden..

I: Det är lite missvisande. Den går alltid till BFF:en och kollar ditt state i min nuvarande implementation. Den kringgås bara via den här proxy-grejen. Och det hade den kunnat göra i den andra lösningen också. Men här är det onödigt, för det finns ingenting som kan bli cachat. Om man behöver uppdatera sitt state — om man tänker att man vill skicka tracking-data eller något sådant.

P: Det är svårt att komma runt. Man måste bearbeta tracking någonstans. Och data som används för att personalisera måste uppdateras för att ge rätt personaliserad data. Men jag tänker att det är en betydligt mindre del. Istället för att klienten hämtar data från matchningsdatabasen hämtar den det från cachat CDN. Förstår jag rätt? I det här fallet, i det första fallet, hämtar klienten... den får veta vilken data den ska hämta från matchningsdatabasen och sedan hämtar den det från CDN:et. Medan det i det här fallet ändå är samma sak någonstans — den måste ändå få något form av ID. Men den behöver inte veta vilken data den ska hämta. Så det kanske ändå blir mindre logik för klienten i slutändan. Och det kommer ändå att gå igenom CDN:et.

I: Varje segmentändring på en sida blir en hel cachad sida.

P: Precis.

I: Medan vi här kan ha en sida och bara segmenten i in-app-lösningen. Så vi får en hel del mer minne eller lagring som går åt här.

P: Ja, precis. Jag tänker att BFF:en ändå måste hänga med på något sätt. Om klienten gör ett sido-request förbi CDN:et för att uppdatera information i BFF:en måste det kunna hanteras. Men spontant känns det ändå som att det är mindre arbete. Om man betänker att databasen kontinuerligt måste beräkna personalisering för varje användare i den första lösningen — det skalar mer med antalet besök. Ju fler besökare, desto mer requests går till matchningsdatabasen, och den måste beräkna vad varje användare ska ha.

I: Ja. Det är mer en affärsfråga — ska det vara segment-of-one och vill vi ändra din personaliseringssegmentering i realtid?

P: Då kanske det finns en skalningsfråga på BFF-nivån också, för att klienten ändå måste uppdatera — eller BFF:en ändå måste veta hur den ska personalisera för det här segmentet. Och det kommer i slutändan via någon form av request från klienten som går förbi CDN-nivån till BFF:en. Så den måste ändå kunna bearbeta nya requests som sker.

I: Ja, den behöver styra reglerna — orkestrera allt. Bygga ihop JSON-objekten.

P: Men det blir också cachat, tänker jag, den delen. Och det gör det i den här första lösningen också. Blir det cachat eller kommer det vara mer individuellt?

I: Det cachas till CDN:et som ligger mellan klienten där. Så man går till matchningsdatabasen och frågar — här kommer jag med den här identifieraren, vad ska jag ha? Det ska du ha. Sedan kan jag skicka iväg requesten mot CMS:et, men den kan fastna i CDN:et på vägen. Och det fungerar likadant i BFF:en. Så man frågar BFF:en, är det här jag ska ha? Ja, det är det. Här är allt. Men här, om vi har 10 sidor och vi personaliserar på två ställen och vi har 10 varianter på varje del vi kan personalisera, har man 10 gånger 10 gånger 2 på en sida varianter. Så många sidor behöver man ha cachade i CDN:et.

P: Det kan bli en sjuk kostnad — det tänker jag också. Så kostnadsmässigt kanske inte... Det är svårt att jämföra om skalningen av matchningsdatabasen motsvarar hur mycket BFF:en måste skala.

I: Givet att de ändå bara behöver skala liknande, eller möjligen mer för BFF:en för att den kräver mer processkraft.

P: Ja, det är det jag undrar. För om requesten vanligtvis går till CDN:et från klienten, behöver klienten bara gå och hämta ett nytt ID — förstår jag rätt? För när man gör sin första request får man ett ID. Och sedan måste tracking och sådant också skickas till BFF:en. Så det kanske blir ungefär detsamma. Svårt att jämföra utan att gå in på supernoga detaljer.

I: Men du satte en 4:a på in-app-lösningen. Lågt var dåligt. Lågt är dåligt, högt är billigt. Är din spontana känsla att det här skulle vara billigare eller dyrare?

P: Svårt att säga för CDN:et — om den extra logiken som krävs för CDN kostar mycket mer, eller om lagringen i CDN kostar mycket mer. Svårt att uppskatta. Säg... ja, spontant lite högre då kanske. En 5:a då.

I: Billigare?

P: Nej, sämre. En 3:a då. Jag tror samtidigt att det... att det ligger inom samma storleksordning kostnadsmässigt.

I: Vi går vidare till lite säkerhetstänk och går tillbaka till in-app-lösningen. Man skickar sessionskakan tillbaka till basen, ser vilket user-segment man ska ha, skickar tillbaka det till klienten — den får i princip tillbaka endpointsen den ska skicka requests till. På en skala 1 till 10, 1 är låg säkerhet, 10 är hög säkerhet — introducerar den här lösningen nya säkerhetsrisker? Man tänker på att det är öppna sidor vi jobbar med också.

P: Ja, precis. Det jag ser är att vi lägger till någon form av data som vi sparar. Det kanske är data vi redan har egentligen. Vi trackar fortfarande användare och använder det för att förbättra sidan och så vidare. Så att lägga till personaliseringen utökar vad vi gör med den informationen. Det kanske ger lite fler risker i compliance-bitar. Och där, ja — plötsligt har vi, kanske om man också utökar det, mer information som vi behöver spara om användare. Som skulle kunna vara känslig, jag vet inte. Även om det är öppna sidor blir det ändå att man plockar ut en person och ger den unik data anpassad för den. Det känns som att man måste skydda det.

Men jag ser inte det som en jättestor risk eller att det blir stora säkerhetsproblem med det här. Det jag tänker på är klienten, speciellt när man har mycket client-side. Och hela personaliseringen som sker client-side innebär att något exponeras utåt — all logik om hur vår personalisering fungerar. Matchningsdatabasens regler är dolda. Så det kanske inte spelar någon större roll att resten sitter i klienten. Men det blir mer transparent vad som händer med personaliseringen. Men jag vill helt ärligt säga att det inte finns några jättestora säkerhetsproblem med lösningen generellt. Det är cookie-baserat, det är ingen superkänslig data vad jag kan föreställa mig.

I: Och det enda jag använder är GDPR-skyddad data.

P: Och vi har ändå en databas som skyddar allt. Vi kör förmodligen secure cookies och best practice för att lagra tokens och så vidare för klienter. Så det är svårt att se någon större risk — jämförbart med vilket annat system som helst.

I: Ganska säkert alltså?

P: Ganska säkert skulle jag säga. Men det är inte...

I: Är vi uppe på 7–8?

P: Jag skulle säga 8. Vill du ha nästa också?

I: Vi kommer dit. Tänk på det här — känner du att det finns något... eftersom personaliseringen sker i browsern på klienten behöver vi kommunicera med... ökar vi attackytan om man tänker på att vi utsätts för en hackerattack eller något? Är vi säkra mot förfalskning eller vad det nu kan vara. Gör vi den ytan större?

P: Attackvektorer, menar du?

I: Ja, till exempel har vi öppnat en endpoint mot matchningsdatabasen mot internet. Precis.

P: Det är en känslig punkt och sedan CDN:et i sig — om man jämför med laddningen idag måste man säga. Om man inte har matchningen, den kommer ändå att vara där. Där finns det bara hämtningar. Så vad som öppnas upp i miljön är att vi nu skickar någon form av information till matchningsdatabasen. Och när man kan skicka information till något sådant finns det någon form av attackvektor där. Beroende på vad det är. Men det öppnar upp för något — någon form av attack kanske. Beroende på... det är en risk. Men sedan — och den måste underhållas, att man skyddar matchningsdatabasen enligt bästa principer. Så jag ser ingen direkt risk att det skulle vara känsligt. Men det finns möjligheter att till exempel köra en DDoS-attack. Man måste skydda sig mot sådana saker. Det är inget problem att ha den, men det kräver att man tänker på det och skyddar sig mot det. Och att klienten exponerar lite mer hur logiken fungerar — det är svårt att se hur det skulle kunna utnyttjas för... requesten kommer ändå att komma från CDN:et och man kan tänka sig att det skulle vara skyddat mot överbelastningsattacker och så vidare. Och matchningsdatabasen — ja, ger information om vad man ska ha för något, men kräver ändå att ha en session. Och den är rätt anonym ändå givet att det är öppna sidor. Nej, det öppnar upp en extra attackvektor som vi var inne på, men.

I: Om man tänker på risken kring cache-relaterade problem... alltså att man faktiskt visas fel content för fel segment — ser du någon risk i det? Eller struntar vi i det för att det inte är vårt ansvar, det exponerar inget personligt till någon annan. Allt är publikt, öppet.

P: Det skulle vara om man på något sätt försökte korsa den här informationen med att vara inloggad eller något, men det tänker jag inte riktigt är i scope. Då kanske det finns en risk att... det rimliga, om man skulle göra något sådant, är att matchningsdatabasen gav information och om det inte finns en cache där, finns det ingen risk att det skulle läcka till någon annan direkt. Ja. Nej, jag tänker att det borde vara lugnt, och om det bara är öppen data i CDN:et är det ingen större fara att det läcker.

I: Om vi tar Principle of Least Privilege — är du bekant med det?

P: Ja, prata gärna om det för mig.

I: Det är i princip bara att varje komponent i ett system har så lite information som den behöver. Och hur man tänker kring det i den här lösningen — har klienten mer åtkomst än den faktiskt behöver? Om man tänker — skulle en illvillig klient kunna extrahera data från matchningsdatabasen på något sätt?

P: Givet att vi alltid opererar från en begränsad förståelse av detta — och givet att man ändå skyddar den endpoint:en, att man har sin unika identifierare och sin cookie som säkerställer att man är den man är — är det svårt att se hur man skulle kunna hämta någon annans data. Och om allt sitter publikt i CDN:et — den faktiska data man sedan hämtar — matchningsdatabasen säger ju bara vilken data man ska hämta. Sedan sitter det publikt på annat ställe. Så klart att någon annan skulle kunna hämta data som inte var avsedd för dem. Men det börjar inte betyda att det är något dåligt för att det ändå är öppen data. Och ja... Klienten får i slutändan information om vad den ska hämta, kanske på ett lite annat sätt än i den andra lösningen. Och den datan måste ändå landa i klienten till slut.

I: Ja, den får i princip exponerade endpoints som är publicerade. En av de endpoints som redan sitter publikt på nätet.

P: Så ingen större risk. Men klienten har ju lite större ansvarsområde än vad den andra lösningen kommer ha.

I: Lite mer?

P: Lite mer om man tänker på hur mycket ansvar den har. Men egentligen har den inte mer åtkomst än vad den andra lösningen har till något.

I: Finns det några andra säkerhetsgrejer med det här, eller något kring säkerhet som man skulle oroa sig för? Bara kopplat till den första på något sätt.

P: Nej, inget jag kan komma på på rak arm. Alltså, givet att det är öppen data och... Den spontana känslan. Jag fick inte jämföra dem emellan. Bra, och det finns ändå cookies och saker som är secure och hela den biten. Det ger känslan av att man följer best practice någonstans för den typen av data. Så jag känner ingen stor... jag känner ingen större risk.

I: Vidare, vidare. Vi går vidare till BFF:en. Skala 1 till 10, hur upplever du säkerheten?

P: Jag skulle säga ungefär samma som den förra.

I: En 8:a?

P: Det är i princip samma information som behövs från klienten — den behöver någon form av identifierare, cookie eller liknande.

I: Så det finns viss risk för båda men matchningsdatabasen och BFF:en är exponerade — inte fullt optimerade mot internet.

P: Men det får man med BFF:en också. Ja. I och med att... och båda blir ju så. Ja, precis. Det är därför jag tänker att de är ganska likvärdiga säkerhetsmässigt. Så kanske det finns sätt... ja, kanske. Beroende på hur det är strukturerat kanske det öppnar upp för olika typer av attacker eller det finns säkert andra sätt att attackera det jämfört med det andra. Men jag skulle ändå säga att det finns design-by-obscurity — att BFF:en blir lite mer så, det är lite mer black box vad som händer eller hur personaliseringen fungerar. Här får man endpoints som svarar. Här är vad du ska hämta. Det gör det tydligare för någon som vill attackera — så här fungerar vårt system, så här fungerar vår personalisering. Vi kan se att vi skickar en cookie här, vi får tillbaka vad vi ska hämta. Så vi vet att den här tjänsten vi skickar den här datan till har det ansvaret.

Och på det här sättet skickar vi också någon data och vi får tillbaka en cookie antar jag. Men vi vet inte vad som händer därinne utöver det. Vi vet att nästa gång vi hämtar något från vår CDN kommer det vara annorlunda baserat på vad vi fick tillbaka. Så här blir det lite mer obskurt vad som faktiskt sker, och därigenom kanske lite mer skyddat. Men eftersom det ändå måste finnas en liten sidokanal här för att uppdatera sin cookie eller få ny information om vad man ska hämta, finns det fortfarande samma vektor — överbelastningsattack eller hitta ett kryphål för att slå ner matchningsdatabasen eller BFF:en. Men jag skulle ändå betygsätta dem ganska lika.

I: Om vi tar de cache-relaterade riskerna.

P: Jag tänker att klienten i det fallet är ansvarig för att bygga upp sidan och det finns egentligen bara en sida vi får från CDN som är färdig, server-side-renderad. Så länge det är okej påverkas den inte av någon personalisering, tänker jag. Så det finns ingen större risk där. Och det kanske öppnar upp för några fler CDN-relaterade attacker här — någon skulle kunna... Jag är ingen cybersäkerhetsexpert men jag tänker att det blir mer — eftersom CDN:et blir mer avancerat, finns det också fler saker som kan gå fel i den här lösningen kanske, när man har väldigt mycket data som ska lagras i CDN:et och segmenteras för olika användare och så. Kanske att det skulle kunna finnas en cache poisoning-vektor. Jag kan inte tänka mig hur det skulle gå till i praktiken. Men kanske att det är där man behöver tänka lite mer på hur det ska fungera. Att det ska finnas mer säkerhetstänk kring CDN:et i den här lösningen än i den. Annars skulle jag ändå betygsätta dem ganska lika faktiskt, båda.

I: Om vi tar Principle of Least Privilege i relation till BFF:en — hur känns det kring det?

P: Kring alla lösningar, menar du?

I: Ja.

P: Som vi pratade om där har klienterna ju mindre vetskap om hur personaliseringen fungerar. Det blir lite mer black box vad den får tillbaka från BFF:en, och den är bara ansvarig för att rendera det. Men utan vetskap om vad som är personaliserat och inte finns det något i det — och att BFF:en, att all logik för hur saker är personaliserade hamnar där, medan här sipprar det lite ut till klienten också, ger lite mer information till någon som kanske vill attackera, vilket är öppet egentligen. Så det är lite bättre i den här då. Det är bra, men jag skulle också säga igen — vad gör den informationen? Den är verkligen inte känslig. Den är inte personlig på något sätt och den är helt öppen. Det innebär inte att någon skulle få tag i den. Men den finns fortfarande. Den har ändå mindre ansvar för det. Och inget mer att behöva känna till.

I: Är det något annat kring säkerhet som man vill ta upp?

P: Egentligen inte. Jag känner det. Nej.

I: Vi går vidare till jämförelsedelen då, lite mer. Om man skulle behöva välja en av dem, vilken hade du rekommenderat för att utveckla och implementera på Nordeas öppna sidor? Det kan vara en av dem eller att det beror på.

P: Jag gillar nog BFF-mönstret mer. För i klienten behöver man inte göra lika mycket. Man kan ha den logiken helt kopplad till ett eget system och hantera det — ja. Det blir inte så ihopkluttat överallt och kräver inte lika mycket integrering. Och jag tänker också på hur det ser ut i hur vi jobbar nu — matchningsdatabasen och hela den biten, vad vi ska presentera för våra användare, skulle troligtvis ha hamnat på ett annat team än de som jobbar med klienten. Och då tänker jag ungefär att den logiska biten borde vara...

I: Skulle du säga BFF:en då?

P: Ja, men det skulle vara mer spontant. Kanske funderar jag för mycket.

I: Nej, nej, det är bara bra. På en skala 1 till 10, hur säker är du på den rekommendationen? 1 är osäker, 10 är väldigt säker.

P: Jag skulle säga en 7 kanske, 8. Jag känner starkt för det, men som vi diskuterade finns det fördelar och nackdelar med båda. Och beroende på vad man vill få ut i slutändan — vill man ha segment-of-one, då kanske det är lättast att personalisera i klienten. Medan om man kan ha lite bredare och större segment för många användare, om de får samma content, tror jag det finns stora fördelar med BFF-lösningen.

I: Om man skulle ranka de tre dimensionerna — prestanda, förvaltningsbarhet och säkerhet — i viktighetsordning för Nordea och för dig i det här. Hur hade du rankat dem? Vad kommer först? Man har prestanda, säkerhet, förvaltningsbarhet.

P: Jag skulle ändå säga att prestanda är nog viktig överlag. Säkerhet är självklart viktig som fan. Men jag tänker i det här laddningskontextet — jag tror inte säkerhet är det man ska hänga upp sig på för jag tror det kommer bli ganska likt. Så jag skulle sätta säkerhet sist. Förvaltningsbarhet är självklart viktigt. Men prestanda blir på något sätt det viktiga för att skapa en bra användarupplevelse av de tre för slutanvändarna — och det tror jag ändå är det viktigaste. Prestanda, förvaltningsbarhet, säkerhet sist.

I: När du tittar på de här siffrorna, var det något du reagerade på, något du inte trodde? Något som förvånade dig — att något var snabbare eller bättre än du trodde.

P: Något som var intressant är den här personaliseringstiden — för den räknar med när personaliseringen sker under den här skillnaden mellan dessa. Och här är den redan personaliserad i BFF:en.

I: Ja, det stämmer. Så det blir i princip den totala tiden.

P: Det är en bra grej tänker jag — man kan få något personaliserat direkt. Jag tänker att det är ändå väntat.

Här har vi det här direkt mot Angular och här på CDN — skillnaden för in-app är 491 mot 49.

I: Precis. Just att CDN kanske är mest intressant att jämföra egentligen, att känna vad det skulle bli i verkligheten med CDN framför, och vilket det är. Och det är en sjukt stor skillnad. The Mycannat känns ganska likt — dessa var bara loadtester. Då är det bara server-side load utan att börja rendera JavaScript eller något. De känns ganska lika.

I: Svarstider egentligen.

P: De känns rätt lika. Det känns inte som att det finns någon större skillnad man kan förklara för någon — spontan känsla i alla fall.

I: Nej, den största skillnaden är runt 11%. Den körde vi direkt mot Angular för totala requests.

P: Precis. Ja. Om man tänker att det är lite långsamt — jag kan föreställa mig det, okej. Det är hur många requests man hinner göra. Okej. Och in-app är bättre.

I: Ja, innan man når ett tröskel. Vi ska ha i åtanke att allt gjordes på en dator. Alltid.

P: Okej. Inget specifikt jag reagerar på som känns oväntat. Just det. Det är väl den siffran som är den jag tycker är mest intressant. Personaliseringstiden.

I: Om man skulle kunna modifiera någon av dem — finns det något specifikt där man känner att lösningarna fallerar? Man kan kombinera dem på något sätt eller göra något helt annat. Flytta logik någon annanstans.

P: Jag tänker spontant på det där. Om man ser att man har en trackingtjänst, till exempel — som många har — så är det faktiskt ett ganska bra ställe att hämta data ifrån för att göra personalisering senare. Så jag tänker att om man skulle göra någon ny variant på det här — att man skulle ha något litet bredvid BFF:en, en liten analytics-tjänst som klienten pratar med och BFF:en, eller matchningsdatabasen, väntar information ifrån. Att det är uppdelat på det sättet. Men annars kanske inte. Som jag tänker det.

I: Tack för din medverkan.

P: Tack.

F Interview transcript – P4

Interview Transcript

I = Interviewer P = Participant

I: So we'll start with, if you could just briefly describe your role and your experience with Angular, CMS integration, or backend service or architecture.

P: Right, well in general I have like over 10 years of experience working with like web apps, especially with the backend stuff, however with Angular and this isomorphic Angular architecture it's like the last two and a half years, like working on it on a daily basis essentially.

I: And do you feel like you're familiar with the BFF pattern or similar middleware patterns?

P: I would say so, maybe I didn't call it BFF, but yeah.

I: Okay, so on a scale of 1 to 10, 1 being very difficult and 10 being very easy, how easily do you think a new developer could understand and start working with this architecture — and we'll start with the in-app. So not new from school, but new to the project.

P: I think, to be honest, this is maybe, like integration-wise, I would say this is maybe a little bit easier to — not like to set up, but maybe easier to work with — because you're just like integrating with a separate service, right? That's how I would like view it. You get your recommendation service or this matching DB or anything essentially and you plug it into your app and there you control essentially what gets replaced and how. So I would think this is pretty standard integration towards a third-party service that can do something with your content one way or another. So I think this would be like, yeah, pretty standard-ish, like maybe eight. Like more control, but obviously you would need to actually touch the codebase of your app to integrate with it. But yeah, that's like standard.

I: Yeah, do you find any part of the design more challenging to onboard? Because are you assuming that the matching database or this service would be like a third party?

P: Yeah, exactly. So no, I would treat it as like any other third-party integration, essentially almost the same as like fetching data from CMS — just give us the API, basically spec what to call with what parameters and what is the output and then we deal with that. Yeah, the only downside is essentially you need to actively modify your application and integrate with it and maybe install like some libraries or just do it like raw HTTP or whatever the system supports. So yeah, a bit more labor, but standard third-party integration.

I: And on a scale of one to ten, how easily do you think this approach could be like modified or extended if you wanted to include more segments and change more — like, oh, we don't just want to change banners, we want to change...

P: Yeah, that's like, I would say that's kind of hard to say in general, but that's maybe the answer — because that would very much depend on like what the system gives back, the output of the system, but also like how the app itself works, like how it works with the components and with like the whole UI. So there I would go with like something lower, like maybe three or four, because it's just like very unknown and very much dependent on the app itself.

I: Yeah, because now my current implementation, the matching database returns the REST APIs where if you are a part of this segment, you should contact this to get... So this is aware of what the CMS contains, the matching database, and also the client or the application needs to be aware of what should I change.

P: So I guess that would make it like more labour-intensive to extend it.

I: I guess you could make one of them just aware of what components you're changing out, but yeah. Where would you see the biggest risk for technical debt as the personalization logic grows?

P: Well, essentially introducing a bunch of... Yeah, you would need to — integrating in this way, you would need to essentially do active changes in your web app codebase. So there is a risk that you would need to make sure that you keep your app up to date with anything that changes in the matching system. If there is like some backward-incompatible change, then suddenly you need to force redeployment on your app with the changes. So yeah, I would say the maintenance is not great because you would need to actively keep supporting it. Or if your app needs to be updated with something new, and then for some reason the matching system wouldn't be up to date in the same way, then you would need to force that system to be updated. So this bidirectional relationship would be a bit more problematic. Yeah, this system would need to make sure that it's always compatible — to make sure that it can work with the system even if it would not be updated, for example.

I: And if you go on to like testability of this architecture on a scale of 1 to 10, what would you rate?

P: Yeah, that depends a bit, but I would maybe put it somewhere in the middle — 5, 6. Since it would be direct integration, it feels like you could do all these things like mocking the data from the system. And you can make, I believe quite easily, you can make sure that your app will keep working unless the interface changes to this matching system. But then you would probably still want to do proper integration testing or proper end-to-end testing. And if that system doesn't have a problem running live tests with it, then that should just keep working, I would say. But I would put it somewhere in the middle because you would need to make sure that your tests are respecting the fact that you're integrating with this third-party system. And it's baked into your application, in a sense, that integration part.

I: Do you reckon you could test the personalization logic in isolation?

P: If maybe — I mean, the web app itself would need to have something for that. Sometimes systems, they allow you to use these hidden parameters here and there. You can use query params or some special cookies that activate certain paths here and there. But again, you would need to bake that into the web app so it respects the testing mode. So maybe something like that can be done, but again, that's extra work. If that would be super important to test and make sure that that works, then yeah, that needs to be done essentially — so some kind of testing mode for this.

I: Because one thing is the logic, the rules that make the personalization, that lives here. So you divide the logic and the thing.nAnd then on a scale of 1 to 10, how reasonable do you find — if you can make some sort of evaluation — the operational costs of running this architecture in production? Not implementing it, but just running costs.

P: Yeah, I guess that would be completely up to the pricing of the system. Because if it would be used as a third-party system, then yeah, whatever the payment model is — per thousand requests, per million requests or something like that.

I: But if you compare — you have a stock application with no personalization, and then if you want to introduce this — if you scale that from 1 to 10, how much would the cost rise? Do you think the Angular servers need to be beefed up?

P: Not really. I think integrating with the system, apart from paying the labor to actually integrate, I don't think there would be a big difference. Of course, then there is the operational cost of querying the data from the system, but whatever the model would be there. So I don't think it would be more expensive to run the application with this, unless this system would be super expensive. I don't know if you would pay per request or something like that.

I: A pretty high number then.

P: Yeah, I would say it would be completely dependent on the pricing model of the system. If it would be for free, then it's almost like no money to run it in the long term. But if it would be really expensive, or if it would be cheap in the beginning but then suddenly extremely expensive — because, I don't know, what happens quite often is that you have these systems. Even Google Maps is a good example. Under a certain limit it's for free, very reasonable, but suddenly your system just spikes and it goes viral because of something, something happens. Then suddenly overnight you can get thousands and thousands of dollars of charges because you just did not anticipate the traffic. You have nothing in the web app that kind of cuts off the request. So I would say potentially it can be extremely expensive if stuff is not put in place to prevent that happening. Maybe something like seven, eight.

I: We'll move on to the BFF now — it's essentially the same questions for this implementation. So one to ten, how easily do you think you could onboard a developer, working with this architecture, maintaining it, implementing it?

P: Yeah, because here there's essentially almost nothing needed, right? It's a thing that is in front of the application. So there is almost like — as I understood it, there would just be a need for a little bit of infra, essentially making sure that this CDN is in front of the application. So no direct integration in a sense in the web app. So I would say like the regular developer wouldn't even almost touch it. It would be more like a DevOps task to basically configure the thing so that it works.

I: If you take that to implement the BFF — because that's the only thing you really need to do, you just change where the application fetches from.

P: So yeah, I would say like this is way simpler. Because this is still like a third-party service, right?

I: It depends where I think it would be, but you could collect the data maybe for a third party. Okay. But then we might want to have our own rules.

P: All right, so you mean the developers in the project itself would need to actively operate in the BFF to maybe configure it or?

I: Yeah, or maybe it would be a whole different team. But yeah, it's an in-house development and maintainability.

P: Okay, because from the integration point of view, that would be kind of like almost instant. But yeah, if you would count that you would need to completely maintain the whole BFF solution, then yeah, you would need essentially a separate team — or like someone or team of someones to do that. So yeah, that would be quite a lot of work. But then did I understand it correctly? Not the same would be needed for the first solution because that would be just like a service provided — you would get this matching data back. So there would be like maybe no team maintaining this.

I: I guess it would be an internal team maintaining it, and maybe you could have a third party that collects the data and maybe gives it to this — this creates the segments, this applies the segments to you.

P: Yeah, okay. But then the situation is kind of similar, I would say, because the work would need to be done on one or the other system. But then integrating here, it's way more laborious for the first solution — you would need to spend more work to actually utilize it. Here you would just configure it in front of your web app essentially.

I: If this would be the matching database, the other team would be responsible for it. Then the communication — because this would be like the web app team that does the

personalization of the app and then another team that does the matching database or what you would call it.

P: But then the answer is for like the whole solution including the BFF, or for only integrating with the web app?

I: For the whole solution.

P: All right, but then it feels like this is more complex — like the BFF solution is more complex, but the integration part is more simple. Here this is a bit more simple, but this is a bit more complex. So I don't know, like maybe then the answer would be the same, because it's hard to say — at least for me it's hard to say like how complex this would become.

I: Yeah and I guess the most complex part of it is the decision making, the rule logic, business logic or what you would call it.

P: All right, but then I would maybe put seven on this one just to make it a little bit simpler because of the integration aspect of it.

I: Yeah simpler, so you would put a nine?

P: Oh sorry, the nine, yeah.

I: How does the separation of responsibilities affect the mental model of the system?

P: Yeah, so I would say the separation is like better here because it's pretty independent — in a sense that this system is independent. The application or the web app is then working kind of in isolation if it needs to be, and then there is this like CDN essentially in front doing the magic. If something goes wrong or something happens, that can always be taken out. So it seems like the web app is pretty independent, but the experience for the client is like the mix of both. So I would say the separation is like a bit better here because again the CDN and this system would need to do a bit more heavy lifting to make sure that they always work. The web app can live kind of on its own, compared to this — here the web app and this system would need to be in sync more, they will need to be aware of each other a little bit more. So yeah, this in my head is a better separation.

I: Yeah and then one to ten, if you think about — if you're going to modify or extend the system.

P: Yeah, it feels like this would be like somewhere like eight or something like that, because essentially extensions would happen here mostly (in the BFF), which is like quite independent. Yeah, of course like the web app thing would need to maybe somehow help out or make sure that everything works together overall, but it feels like it would be quite independent and can also be tested quite in isolation. This CDN can be configured in ways that — yeah — pass the traffic only for a certain percentage, or with these magic cookies or params do not release that yet to everyone. Stuff like that. So it feels like there is more control, way more control.

I: Yeah, where do you see the risks of added complexity?

P: Oh yeah, the added complexity is that you would need to make sure that it works together, and someone would need to make sure that it works together. If this would be like two separated teams then that's like a human — like management, team management kind of problem. So maybe not a technical problem. But then you would need to probably have someone that would make sure everything always works together, that there is nothing that breaks horribly, and then the line gets blurred or a bit more blurry — like who takes that responsibility. Usually like, is it the web team, is it like this team, or is there like someone above? So I think it's like a bit more like team management, software management kind of issue. Then of course this whole system and the CDN would need to make sure that it really works fast — you don't want to put a big system in front of your web app or anything else

that has a possibility that it slows down or something happens to it. So the system would really need to be on top of the performance since that would be like the point of contact for the client.

I: Yeah, one thing I can mention from my tests is that the storage on the CDN is quite a bit larger for the BFF than — because since this catches the whole page and app for each — yeah, essentially each page and each segment. And here you can just have the page, the default page, and then the segments separate. And if you would rate the testability of this architecture?

P: I think the testability is maybe a little bit better than the previous solution, maybe like one point better, because again this separation is like quite good. Yeah, the web app can be completely fully tested on its own, and then if this personalization kicks in then you can do like one of these things like these magic cookies, params and stuff like that. You can very easily turn it on and off. Even if that system would go down you can shield from that a bit more easily since it's like this whole component in front of the web app instead of it being part of the web app internally. So I think the testability is actually yeah pretty good — kind of like plug in, plug out.

I: I think you put a five or six on the in-app?

P: Yeah, so this one I would put like seven.

I: And if you rate the operational costs of running this?

P: Yeah, this would be — I would say this would be maybe like similar. Or maybe it depends — yeah, again, this system, if that would require — well, this one we treated as like some kind of service that can cost some money, so that component would be the same. But also the CDN would be like kind of a big part. For low traffic usage these are usually quite cheap or essentially for free. But if the amount of variants of content — like the storage — if that explodes for some reason, that can get extremely expensive. But on the other hand the CDNs are really good also at saving money and essentially bandwidth, because you can cache a lot of stuff there and then instead of hammering your infrastructure your web servers you just get content cached all around the world. So it feels like this might be a bit better operationally, especially because of this heavy use of CDN. But it depends like how much content or how much data would be there. But those are things that can be calculated in advance, those can be estimated. So it feels like maybe there is better control over costs in this — or that's what I would maybe expect.

I: You put a seven or eight on the in-app solution

P: Yeah. But then this one will go like nine.

I: Yeah okay, so we'll move on to more of the security aspect, and we'll go back to the in-app application architecture. How secure do you consider the architecture overall on a scale one to ten, ten is very secure?

P: Yeah, so here I would say it depends a little bit, but I think it might be a little bit more secure in a sense that the application directly integrates to the system, so whatever the security model for the web app is, you would inherit a bit of that in this solution because the web app would process the data directly. So I don't know like if there is some garbage in the data that is unsafe or something, that can be filtered out if you already have these things in the web app. So you would get a bit of a benefit. And then you have way more control over what you use from the system since you need to directly integrate with it. So this leaking some stuff unintentionally — it feels like it would be a harder thing to happen. Yeah, more control essentially here since it's a direct integration. But yeah, again, it also depends like what the data is produced here, but let's say that should be good. So yeah, I would maybe rate this like seven, eight.

I: Does the attack surface increase with this solution?

P: I would say not really, because for the client you don't even know like this happens kind of under the hood. The only thing is that yeah, like cookies or stuff that is used to trigger the personalization part — if there's like a possibility to get out some personalized data without some kind of authorization and stuff like that, if you can use that to harvest essentially some personalized content and that can be used in some harmful ways — I would consider that to be like one extra thing there. But otherwise it's just content.

I: So yeah, it's just the client could potentially see what you're changing out.

P: Yeah, exactly.

I: So yeah, but you would have no insight in what segments you are or...

P: Yeah. So I don't think this is bad in any sense or insecure. Like if this data is not sensitive or anything like that, then it's perfectly fine. Because of course like the malicious clients, they can try to enumerate — yeah, whatever method — if the system would be using for example cookies or these params or something like that, then they can start bashing it and bombarding the system and trying to get out anything useful. But in the end it's still just content. So yeah, and there can be protections put against that. So yeah, that's fine. And you still need to go to the application server to get the proper content back. So I think this is pretty fine — it's like integrating directly with something. If this something is safe then it should be good.

I: Do you see any risks with the cache and the CDN for this application?

P: Yes — not just 'oh we're giving you the wrong segment'. Well, I mean technically it can happen, but that would need to be like some misconfiguration. That's when malicious clients can try to do so-called cache poisoning where they try to fetch a bunch of data and then just forcefully keep it in the cache so everyone else gets the same garbage back. But that's on the team or the developers or the DevOps to make sure — that's on the CDN provider or like for them as well. But you need to make sure that you are protected against something like that.

I: Are you familiar with the principle of least privilege?

P: Yeah, yeah.

I: How do you evaluate this — does the browser have more data than it actually needs access to, or exposes information?

P: Not really. It doesn't seem like it. You would get served by the web app, possibly something would come from the CDN. So yeah, you would get back your content with your personalization baked into that. And since the integration point is deep in the application server itself, then yeah, if the app doesn't expose anything you would only be getting back whatever the app allows you to get back.

I: Do you have any other security concerns regarding this implementation?

P: The only thing — now I'm looking at this like arrows — so there is also the arrow from the client directly to the matching DB. What is that used for?

I: The session handling — both of them. You need to maintain your session. And both of the implementations do that. And right now the BFF — it communicates via the CDN, it just gets bypassed, just used as a proxy? Didn't do that for the application.

P: Because I would consider the direct connection there being like more problematic — because then yeah, just like the basic thing, if someone wanted to DDoS that thing then without CDN it's way simpler.

I: Then yeah, you would want the CDN to provide a shield for that.

P: Exactly, exactly. So I would want to make sure that nothing internal is exposed directly, because that can be misused one way or another. So that's the one thing.

I: Yeah, we'll move on to the BFF. If you rate the security of this architecture overall?

P: Yeah, so this — I would say this all depends mostly on the CDN. The thing is, yeah, that would need to be properly maintained since that would be doing quite some heavy lifting. But yeah, that would be fully in control — like the DevOps or the team or the CDN provider itself. So that is all good. Yeah, so I think this is — especially because of this connectivity thing — this feels like it's a bit more secure, like more shielded. But more importantly, the personalized service is kind of transparent to the web app, the web app doesn't know that much about it, which is good. It doesn't seem like you would be able to misuse that since it is behind CDN. So I would say this is a bit more secure. But especially because of the arrow — this connectivity here — it can be handled. Here I would go with like nine something.

I: Do you think cross-site request forgery — reference forging — would that be a risk for this implementation?

P: Well yes, absolutely — if someone would take over like this, so the moment that is somehow compromised, so someone would essentially — if someone would be able to intercept this line then it's like big big problem in a sense. But it depends — it would be kind of similar here as well. If someone would be able to take over this system then you have this direct integration that is maybe even more scary, because then the application server itself will be exposed to some malicious third party behind it. Here it would be mainly CDN that would get garbage — your web app would be more shielded. But if this would be running like fully internally then yeah, that would not be that big of an issue. Of course, here the CDN provider would need to make sure that this line is not compromised..

I: From the CDN to the BFF.

P: Yeah, exactly. Because you don't want to end up with like some kind of man-in-the-middle and these kinds of situations.

I: Yeah. But you reckon that applies to the...

P: Yeah, the same applies here as well. Here again it feels a bit more scary because there is this direct integration to it. So here what can happen — in both cases the clients would get compromised content. But here the CDN would get possibly the biggest hit of the garbage, whereas here you would get your application server exposed to that. So it feels like this would still be a bit better.

I: But if you would take away this line between the CDN and the BFF and you would essentially proxy it and authenticate it somehow via the application server — would that eliminate the risk? So it works just as it does now on the SSR, but then when it's client-based it still just proxies and authenticates it through the server application?

I: Would it minimize the risks?

P: Well, then it would be kind of similar like here. So it feels like this is actually a good thing that it can happen like that — because yeah, then you're shielding your application server from an extra component here. But there is also this direct line — and this is used for what exactly at the moment?

I: For the SSR.So if it's a cache miss, yeah, it would go there.

P: So yeah, then it's quite similar, especially in the case of that SSR.

I: Yeah, but maybe I would like need to have some sort of layer that shields this service from like DDoS or...

P: Yeah, but then here you would have your CDN anyway. Yeah, the whole thing just relies on the fact that you don't want this to be somehow hijacked or like this, and if that would happen then you are in big trouble anyway.

I: I think we covered the cache-related security risks. And the least privilege aspect for this?

P: Yeah, so here it gets like a little bit blurry because then the CDN gets access here that would need to be handled separately — because you don't want to end up with the CDN having access to too many things. So in a sense this is more shielded from that, because here with this direct integration you control it a little bit more. But here, this CDN — you would need to make sure there is an extra thing, you would need to make sure that everything is good from the CDN to the BFF.

I: But do you think that's the same problem with the in-app solution?

P: Yeah, basically. But here it's even like worse because the client can be like anyone — there would be nothing between there. At least the CDN — like you can believe that big CDN providers are doing like due diligence and they have experience like how to do this stuff correctly. And then usually as a client you have some access to some kind of management platform or some management toolkit on their end. So you can set up all these warnings and all these levers and — you know — breaks essentially. If something goes like terribly wrong you can cut it off via some management. If you would expose it directly here, then you have no such thing — you would need to somehow do something in this system, and that's usually more problematic. So yeah, this feels like a good place to have all this — you know, like a big red button: something goes wrong, okay, turn it off.

I: Any further security concerns regarding this architecture?

P: Yeah, the main thing is that since we are talking about content manipulation, the super important thing is that whatever this system does — and this is in both cases — it needs to be secure and safe. Because okay, there is a CMS, and that presumably has its own rules how to manage content and what can be exposed through the CMS, there is some user management, who can do what. And that provides the content primarily. But then if now the content can be swapped out — still the content comes from the CMS right, but you can switch around who sees what depending on these segmentation rules and whatnot — then you need to have the same scrutiny essentially also here, because you don't want to end up exposing something to someone that is not supposed to see that. So with this content manipulation, yeah, you just need to make sure that you are following similar practices. And then of course with this CDN being in front, you need to make sure for example that this segmented content — if we are talking about public websites — it wouldn't be a problem if someone else can actually get that segmented content out. Because again you can end up in this situation with cache poisoning, or if the data is in those CDN caches then anyone can eventually query that data. So you don't want someone just seeing something they are not supposed to see ever. If we are talking about unauthenticated channels — and that's what we are working with, like with the websites — then it's not a problem if there would be like some authentication going on, that would be way more complex. So yeah, but just this fact that the data can end up accumulated in the CDN — for public data, that should not be a problem. That's fine.

I: Yeah, we'll go on to the comparative assessment. If you would choose one architecture to put into deployment, which one would you choose?It could depend, you could change things like small things around.

P: But yeah, I mean it feels like yeah, the BFF would be maybe the easier to start with — especially because the integration would be presumably simpler, also based on the numbers. The performance numbers seem like it performs a bit better. Then yeah, the kind of like plug in, plug out aspect of it — it's simpler to actually experiment with the data coming via the BFF, so that would be a good benefit. But mainly the part that you wouldn't need to

directly integrate or modify the codebase in massive ways to actually get the data out of it. So that would be good to start fast with and then figure out like if that works well or not.

I: How would you rate your confidence with this recommendation?

P: Yeah, seeing the numbers, seeing essentially all these materials — I would go somewhere with like seven. I'm personally not a super big fan of all these like modification to content solutions, especially if it's not transparent like how and why it happens. But like on a personal level, in kind of a general sense — because for example for web development so far the super popular way to do this stuff is to use all these tag managers essentially, where you let the web load like in a normal way but then you have a bunch of JavaScript running on your site, usually like Google Tag Manager and all that, and then there is a bunch of random stuff that can happen to your website after the fact, after it was loaded for the first time. And then you lose all the control of what happens in your app — if something goes wrong you never know. But which is not really a problem here, because this content loading happens way before and it's kind of compatible with what the app should be doing. So it's more specifically these random pieces of JavaScript that run on your website that you have no idea who made, how they made it, what they load and stuff like that. So yeah, but in general I'm a bigger fan when these kind of features are directly incorporated in the systems that are providing the data, like directly — like the CMSs. For example, obviously those systems are quite more rigid and super hard to extend like most of the times, so that's why I assume these types of solutions are made — so it's easier to integrate. So yeah, in the best case scenario the CMS itself would have some kind of easy way to build the personalization into it, like very customized personalization.

I: Yeah, yeah. Because maybe if you want to have like a segment-of-one personalization, the cache would be big.

P: Yeah.

I: Yeah, if you would have to rank the three quality dimensions — performance, maintainability, and security — by importance, how would you rank them?

P: Well, all of them are quite important of course. But I would say security would be number one, because something going wrong with the security — well, you don't want essentially someone unauthorized to alter the content that is then shown to millions of people. So that would be the biggest thing. Then it would be the performance I would say, because this needs to be blazing fast always, otherwise people are going away — no one wants to wait even like two seconds for stuff to load. And then yeah, the quality aspect. But that's — I mean, that obviously still needs to be good, but it's kind of a hidden thing — you don't want to accumulate technical debt too much. There needs to be a good balance between the quality and performance. Well, I would say the security is like an absolute must, where it needs to be really really on point, because even small mishaps can end up being disasters, especially if someone would get access to your content management or content altering capabilities. And then the performance and quality is like a bit of a trade-off — the performance obviously needs to be good and quality needs to be good, but you cannot have everything at 100. But I would say that the performance should be maybe a little bit more important than the quality.

I: When you're looking at the performance data, is there something that surprised you?

P: When I first just looked at yeah — essentially here this BFF versus in-app with the CDN, the caching, or the impact on this personalization time — essentially this median time here being so massively different, which is like 10 times almost, like a 90 percent difference. So that's like one thing here. Nothing like super crazy, but yeah, this one is the one that pops out the most.

I: And if you could modify either architecture somehow, what specific change would make the biggest impact on your overall assessment?

P: Well, the only thing is that I find this — way, way, not like better but nicer in a way to work with — and also the BFF solution. But then thinking about the fact of how willing the team would be from a technical point of view to put the system in front of their CMS, and with some kind of customized CDN — because it can be that most likely they already have some CDN already in place. Would that require some customization? Would it need to be this specific CDN from like you or from the whole BFF package? So yeah, that would be one thing that would be a little bit to think about. And then as I was saying previously, if this type of system would be easily like plug and play or some kind of plugin for certain CMSs, that would be pretty much a no-brainer, I would say, for many people that would be deciding what to use. As it is, this would be super generic — it can be used anywhere — but then yeah, you would need to still make sure to configure it or something like that. But if this integration would be somehow part of the CMS and if there would be some kind of personalization plugin that would be part of the CMS, still hooking up to this separate BFF system, all this whatever needs to be running here — that would be like even better for us, for example, I would say. But then of course that's more work because it would need to be thought into the CMS. But again, if the CMS would support some easy way to install it as a plugin or have it configured, then why not. And a lot of the time these systems — not just personalization systems — there is some generic solution that you can reasonably simply put in front of something, but then slowly over time they start coming up with these connectors and plugins for specific systems that many many clients are using. And that's usually quite nice from an integration point of view if you can just take it into use, configure it a little bit, and then if it works like that then it's perfect.

I: Is there anything else you would like to add that we haven't covered?

P: Yeah, maybe the only thing — now looking a bit more at this — what would happen... That's the one thing, a little bit scary — not scary thing — but one thing for the BFF solution is: what would happen if this whole system somehow went down? Or like the performance would be compromised — hardware fails, whatever, some networking issue, something like that happens.

I: Yeah, I guess you could have in the application like a fallback — if this doesn't work, oh the response time is now 400 milliseconds, you go straight to the CMS.

P: Yeah, because something like that would definitely need to be taken care of.

I: Because the app needs to be working without the personalization as well.

P: Yeah, exactly. Yeah, so that's maybe the only other thing.

I: All right, thank you.

P: Right.